Laporan Tugas Besar 1

IF2211 Strategi Algoritma

Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Battlecode 2025

Semester II 2025/2026

Disusun oleh:

Josh Reinhart Zidik — 13524048
Arina Azka — 13524049
Anngelina Andra Alana — 13524079

Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

1 Deskripsi Tugas **3**
1.1 Latar Belakang . . . 3
1.2 Komponen Permainan . . . 3
1.2.1 Objectives . . . 3
1.2.2 Aturan Tiebreaker . . . 3
1.2.3 Peta . . . 4
1.2.4 Jarak Pandang dan Pergerakan . . . 4
1.2.5 Tower dan Ruins . . . 4
1.2.6 Pola Sumber Daya Khusus . . . 4
1.2.7 Cat . . . 5
1.2.8 Chips . . . 5
1.2.9 Robot . . . 5
1.2.10 Jenis Robot . . . 6
1.2.11 Towers . . . 6
1.2.12 Serangan . . . 7
1.2.13 Penambangan . . . 7
1.2.14 Komunikasi . . . 7
1.2.15 Action dan Cooldown . . . 8
1.3 Batasan Program . . . 9
1.3.1 Bytecode . . . 9
1.3.2 Package . . . 10
1.3.3 Memory . . . 10
1.3.4 Execution Time . . . 10

2 Landasan Teori **11**
2.1 Algoritma Greedy . . . 11
2.2 Elemen-Elemen Algoritma Greedy . . . 12
2.2.1 Himpunan Kandidat (C) . . . 12
2.2.2 Fungsi Seleksi . . . 12
2.2.3 Fungsi Kelayakan . . . 12
2.2.4 Himpunan Solusi (S) . . . 12
2.2.5 Fungsi Solusi . . . 12
2.2.6 Fungsi Objektif . . . 13
2.3 Gambaran Umum Permainan Battlecode . . . 13
2.3.1 Robot . . . 13
2.3.2 Menara . . . 14
2.3.3 Resource: *Paint* dan *Chips* . . . 14
2.3.4 Mekanisme Turn . . . 14
2.4 Cara Kerja Program Secara Umum . . . 15
2.4.1 Alur Eksekusi Bot . . . 15
2.4.2 RobotController sebagai Antarmuka Engine . . . 15
2.4.3 Batasan Bytecode . . . 15
2.4.4 Independensi dan Keterbatasan Penginderaan Robot . . . 16
2.4.5 Sistem Komunikasi Antar Unit . . . 16
2.4.6 Penerapan Greedy pada Bot Battlecode . . . 16

3 Aplikasi Strategi Greedy **17**

3.1 Pemetaan Permasalahan Battlecode ke Algoritma Greedy 17
3.1.1 Main 17
3.1.2 Alternatif 1 19
3.1.3 Alternatif 2 20
3.2 Alternatif Strategi Greedy 22
3.2.1 Main 22
3.2.2 Alternatif 1 24
3.2.3 Alternatif 2 26
3.3 Analisis Efisiensi dan Efektivitas Strategi 30
3.3.1 Main 30
3.3.2 Bot Alternatif 1: 31
3.3.3 Bot Alternatif 2: 31
3.4 Strategi Greedy yang Dipilih 33

Lampiran **35**
Tautan Repositori GitHub 35

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode merupakan permainan strategi waktu nyata yang menuntut pemain untuk mengembangkan program yang dapat mengendalikan robot secara otonom. Dalam permainan ini, setiap tim mengendalikan sekumpulan robot yang harus bekerja sama secara strategis untuk mengalahkan tim lawan. Melalui permainan ini, peserta dapat mempelajari berbagai konsep dalam bidang ilmu komputer seperti kecerdasan buatan, algoritma pencarian jalur, komunikasi terdistribusi, serta pengambilan keputusan dalam sistem yang memiliki keterbatasan sumber daya komputasi.

Pada Tugas Besar 1 mata kuliah Strategi Algoritma (IF2211), mahasiswa diminta untuk mengimplementasikan sebuah bot yang dapat bermain dalam permainan Battlecode 2025. Bot tersebut harus dirancang menggunakan pendekatan algoritma greedy untuk menentukan aksi terbaik pada setiap kondisi permainan. Bot yang dibuat akan dipertandingkan dengan bot milik kelompok lain dalam suatu kompetisi yang diselenggarakan sebagai bagian dari tugas besar ini.

Melalui tugas ini, mahasiswa diharapkan dapat memahami konsep algoritma greedy secara lebih mendalam serta mampu mengaplikasikannya pada permasalahan yang melibatkan pengambilan keputusan secara dinamis dalam permainan.

1.2 Komponen Permainan

Permainan Battlecode 2025 memiliki berbagai komponen utama yang mempengaruhi jalannya permainan. Komponen-komponen ini meliputi tujuan permainan, aturan peta, sumber daya, jenis robot, serta mekanisme pembangunan menara. Berikut penjelasan masing-masing komponen.

1.2.1 Objectives

Tujuan utama permainan adalah mewarnai peta sebanyak mungkin. Tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai akan memenangkan permainan. Selain itu, kemenangan juga dapat dicapai dengan menghancurkan seluruh robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% wilayah peta, maka permainan berakhir dan pemenang ditentukan menggunakan aturan tiebreaker berikut:

- Luas area yang telah diwarnai di peta
- Jumlah menara sekutu yang masih hidup
- Total jumlah chips yang dimiliki tim
- Total jumlah paint yang dimiliki oleh robot dan menara
- Jumlah robot sekutu yang masih hidup

- Jika masih seri, pemenang dipilih secara acak

1.2.3 Peta

Permainan berlangsung pada peta berbentuk grid dua dimensi dengan ukuran antara 20×20 hingga 60×60 petak. Koordinat peta dimulai dari titik kiri bawah $(0, 0)$ dan meningkat ke arah timur (kanan) dan utara (atas). Lokasi di peta direpresentasikan dengan objek MapLocation yang menyimpan nilai x dan y. Untuk menjaga keseimbangan permainan, peta dijamin memiliki sifat simetris baik melalui rotasi maupun refleksi.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat lingkungan sekitar dalam jarak $\sqrt{20}$ petak, kira kira 4.47. Robot dapat bergerak ke salah satu dari delapan arah di sekelilingnya selama petak tujuan tidak ditempati robot lain atau menara. Peta juga dapat memiliki tembok (walls) yang tidak dapat dilewati maupun diwarnai. Tembok maksimal hanya 20% dari seluruh peta. Tembok tidak bisa dihapus atau dibangun oleh pemain.

1.2.5 Tower dan Ruins

Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh. Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (ruins), zona 5×5 yang sudah ditentukan di peta.

Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan. Pusat reruntuhan berjarak minimal 5 petak satu sama lain, dan zona 5×5 di sekitar pusat reruntuhan dijamin bebas tembok. Setiap tim memulai permainan dengan 2 menara: 1 menara cat dan 1 menara uang. Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

- Money Tower
- Paint Tower
- Defense Tower

Secara default, setiap tim memulai permainan dengan 1 menara penghasil uang dan 1 menara penghasil cat, keduanya sudah di-upgrade ke level 2.

1.2.6 Pola Sumber Daya Khusus

Mewarnai pola khusus tertentu di peta dapat meningkatkan produksi sumber daya tim. Pola ini disebut Special Resource Pattern (SRP). Setiap pola aktif yang berhasil diwarnai akan membuat semua menara tambang sekutu mendapatkan +3 sumber daya tambahan per giliran.

Robot bisa menandai pola ini menggunakan fungsi markResourcePattern(), lalu menyelesaikannya dengan completeResourcePattern() (biaya 200 chips). Setelah pola selesai dibuat, pola tersebut harus tetap utuh (tidak rusak warnanya) selama 50 ronde agar menjadi aktif. Pola ini sama untuk semua peta dan permainan.

1.2.7 Cat

Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor. Setiap robot dan menara menyimpan cat secara individu, dengan kapasitas maksimal tergantung tipenya. Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi. Cat bisa diisi ulang dari mopper atau ditarik dari menara penghasil cat. Setiap tipe robot memiliki serangan berbeda yang bisa:

- Menggunakan cat untuk meletakkan warna di petak, atau
- Menghapus cat milik lawan dari petak.

Robot bisa memilih antara warna utama atau warna sekunder tim untuk mewarnai petak. Kedua warna ini dianggap sama untuk keperluan skor, tapi dibedakan saat mengecek penyelesaian pola cat.

1.2.8 Chips

Chips merupakan sumber daya yang digunakan untuk memproduksi robot dan membangun menara. Chips dihasilkan oleh Money Tower setiap ronde dan dapat digunakan oleh seluruh unit dalam tim.

1.2.9 Robot

Semua robot dapat melakukan aksi seperti bergerak, merasakan (sensing), dan berkomunikasi satu sama lain. Dalam setiap pertempuran, robot milikmu akan berhadapan dengan satu tim musuh. Permainan berjalan berbasis giliran (turn-based) dan dibagi menjadi ronde. Pada setiap ronde, setiap robot mendapat giliran untuk menjalankan kode dan melakukan aksi. Kode yang dijalankan robot menghabiskan bytecode (ukuran sumber daya komputasi, dijelaskan lebih lanjut pada section bytecode limit). Setiap robot hanya memiliki jumlah bytecode tertentu per giliran; jika habis, gilirannya langsung berakhir dan akan dilanjutkan pada giliran berikutnya. Jika robot sudah selesai menjalankan aksinya pada giliran tersebut, ia harus memanggil Clock.yield() untuk menunggu giliran berikutnya dimulai.

Semua robot memiliki HP (hitpoints / nyawa). Ketika HP mencapai nol, robot langsung dihapus dari permainan. Setiap robot memiliki ID unik yang diacak dan tidak kurang dari 10.000. Robot hanya dapat berinteraksi dengan lingkungan terdekat melalui sensing, pergerakan, dan kemampuan khusus. Setiap robot menjalankan salinan kode Anda secara independen. Robot tidak bisa berbagi variabel statis (masing-masing punya salinannya sendiri) karena dijalankan di JVM terpisah.

Dua atau lebih robot tidak boleh berada di petak yang sama. Ketika cooldown pergerakan di bawah 10, robot dapat bergerak ke salah satu dari 8 petak tetangga. Semua robot memiliki cadangan cat (paint) dengan kapasitas maksimum tertentu. Semakin rendah cadangan cat, semakin tinggi cooldown pergerakan dan aksi yang dihadapi.

Aturan penalti cat rendah:

- Jika cadangan cat hanya X
- Jika cadangan cat habis sama sekali selama giliran, robot tidak bisa bergerak atau

melakukan aksi apa pun (kecuali self-destruct) sampai mendapat cat lagi, dan akan kehilangan 20 HP per ronde.

Robot spawn dengan cadangan cat 100Kehilangan cat saat mengakhiri giliran:

- Di wilayah netral : kehilangan 1 cat
- Di wilayah musuh : kehilangan 2 cat
- Di wilayah sekutu : tidak kehilangan cat

Penalti cat tambahan setiap ronde adalah 1× jumlah robot sekutu yang bersebelahan (dalam 8 petak sekitar). Penalti ini berlipat ganda (×2) saat berada di wilayah musuh. Robot dapat di-spawn dalam radius aksi $\sqrt{4}$ ($\approx$ 2 petak) dari menara sekutu, dengan biaya cat dan chips sesuai jenisnya.

1.2.10 Jenis Robot

Dalam permainan ini terdapat tiga jenis robot utama yang dapat diproduksi oleh tim, yaitu *Soldier*, *Splasher*, dan *Mopper*. Setiap jenis robot memiliki karakteristik, kemampuan, serta peran strategis yang berbeda dalam permainan.

Soldier merupakan unit dasar yang digunakan untuk menyerang dan mewarnai petak secara langsung. Robot ini memiliki kemampuan menyerang satu petak dalam radius aksi tertentu serta dapat memberikan damage kepada menara musuh.

Splasher adalah unit yang berfokus pada serangan area. Robot ini dapat menyerang area berbentuk lingkaran dengan radius tertentu dari titik pusat serangan, sehingga efektif untuk mewarnai banyak petak sekaligus dan memberikan damage kepada menara musuh dalam area tersebut.

Sementara itu, *Mopper* memiliki peran utilitas yang berbeda dari dua robot lainnya. Robot ini dapat menghapus cat musuh pada petak tertentu, mentransfer cat kepada robot atau menara sekutu, serta melakukan ayunan mop yang dapat mengurangi jumlah cat yang dimiliki robot musuh.

Setiap robot memiliki atribut yang berbeda seperti kesehatan (*health*), kapasitas cat maksimum (*paint capacity*), biaya produksi, radius serangan, damage serangan, serta waktu jeda aksi (*cooldown*). Ringkasan statistik dari masing-masing jenis robot ditunjukkan pada Tabel berikut.

Name	Health	Paint Capacity	Paint Cost	Money Cost	Attack Cost	Attack Radius	Attack Damage	Attack Cooldown
Soldier	250	200	200	250	5	3	50 health to towers	10
Splasher	150	300	300	400	50	2	100 health to towers	50
Mopper	50	100	100	300	0	$\sqrt{2}$	-10 paint to robots	30

Tabel 1: Statistik dan atribut utama setiap jenis robot

1.2.11 Towers

Dalam permainan ini terdapat tiga jenis menara utama, yaitu *Money Tower*, *Paint Tower*, dan *Defense Tower*. Setiap jenis menara memiliki fungsi yang berbeda dalam mendukung strategi tim. *Money Tower* berfungsi untuk menghasilkan chips setiap giliran, sedangkan *Paint Tower* menghasilkan sumber daya paint yang digunakan oleh robot untuk berbagai aksi dalam permainan. Sementara itu, *Defense Tower* berperan sebagai struktur pertahanan yang memiliki kemampuan serangan lebih kuat dibandingkan jenis menara lainnya.

Setiap menara memiliki sejumlah atribut yang menentukan performanya, antara lain biaya pembangunan atau peningkatan level (*build/upgrade cost*), jangkauan serangan (*attack range*), kekuatan serangan satu target (*single-block attack strength*), kekuatan serangan area (*AoE attack strength*), laju produksi sumber daya (*base mining rate*), serta tingkat kesehatan (*health*). Nilai dari atribut-atribut tersebut berbeda untuk setiap jenis menara dan meningkat seiring dengan peningkatan level menara. Rincian atribut untuk masing-masing jenis dan level menara ditunjukkan pada Tabel berikut.

Name	Build/Upgrade Cost	Attack Range	Single-block Attack Strength	AoE Attack Strength	Base Mining Rate	Health
Money Lv1	1000 chips	3	20	10	20 chips/turn	1000
Lv2	2500 chips	3	20	10	30 chips/turn	1500
Lv3	5000 chips	3	20	10	40 chips/turn	2000
Paint Lv1	1000 chips	3	20	10	5 paint/turn	1000
Lv2	2500 chips	3	20	10	10 paint/turn	1500
Lv3	5000 chips	3	20	10	15 paint/turn	2000
Defense Lv1	1000 chips	4	40	20	20 chips/attack*	2000
Lv2	2500 chips	4	50	25	30 chips/attack*	2500
Lv3	5000 chips	4	60	35	40 chips/attack*	3000

Tabel 2: Spesifikasi berbagai jenis menara dan levelnya

1.2.12 Serangan

Menara dapat menyerang unit musuh. Menara memiliki dua jenis serangan:

- Serangan satu petak (single-block attack) yang menyerang satu unit musuh di satu petak.
- Serangan area efek (AoE / area of effect) jarak jauh yang memberikan kerusakan kepada semua unit musuh di dalam jangkauannya.

Setiap giliran, satu menara dapat melakukan satu serangan satu petak dan satu serangan area efek.

1.2.13 Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan secara pasif menambang sumber daya.

- Cat disimpan di dalam menara dan dapat diambil oleh robot sekutu.
- Uang (chips) tersedia untuk semua unit sekutu.

Setiap menara memulai dengan 500 cat untuk digunakan spawn berbagai jenis unit. Menara dapat memunculkan robot dalam jarak 2 petak dari posisinya. Jika cat di menara habis, menara tidak akan bisa spawn unit lagi sampai diisi ulang atau menghasilkan cat sendiri. Cat dapat diisi ulang menggunakan cat yang dikumpulkan oleh mopper.

1.2.14 Komunikasi

Robot hanya bisa melihat lingkungan di sekitarnya secara langsung dan dikendalikan secara independen oleh salinan kode yang sama, sehingga koordinasi antar-robot sangat sulit. Anda tidak dapat berbagi variabel apa pun di antara mereka; bahkan variabel static pun tidak akan dibagikan karena setiap robot memiliki salinannya sendiri.

Komunikasi dilakukan dengan dua cara:

- Pesan (Messages) Pesan dapat berisi informasi apa saja, tetapi dibatasi hanya 4 byte.
- Penanda (Markers) Penanda menyatakan warna apa yang seharusnya digunakan untuk mewarnai ulang suatu petak, dan penanda ini terlihat oleh semua robot sekutu. Penanda tidak memberikan efek lain apa pun dalam permainan.

1. **Pesan**
2. **Penanda**

1.2.15 Action dan Cooldown

Robot melakukan berbagai aksi untuk berinteraksi dengan dunia permainan. Beberapa aksi tidak bisa dilakukan berkali-kali dalam satu giliran atau dalam waktu singkat karena terikat cooldown. Berikut daftar aksi utama:

- Menyerang/mewarnai
  Menyerang dan mewarnai dilakukan dengan fungsi yang sama. Aksi ini hanya bisa dilakukan jika action cooldown robot < 10. Action cooldown robot dapat dicek dengan fungsi isActionReady(). Dilakukan dengan memanggil metode attack() yang akan menaargetkan satu petak yang dapat dilihat oleh robot tersebut. Efeknya tergantung tipe robot (seperti yang dijelaskan di bagian Robot): memberikan damage atau mewarnai petak. Setelah dilakukan, action cooldown robot bertambah sesuai nilai yang ditentukan oleh tipe robot.
- Bergerak (moving)
  Hanya bisa dilakukan jika movement cooldown < 10. Movement cooldown dapat dicek dengan isMovementReady(). Petak tujuan harus kosong (tidak ditempati unit lain) dan bisa dilewati (passable). Dilakukan dengan memanggil move() dengan arah yang diinginkan. Setelah bergerak, movement cooldown bertambah 10. Untuk memeriksa apakah gerakan valid, gunakan canMove().
- Menandai (marking)
  Semua robot bisa menandai petak tempat robot sedang berdiri, atau Menghapus penanda sekutu dari petak tempat robot berdiri. Biaya dari aksi ini adalah 1 cat. Aksi ini tidak menambah action cooldown sama sekali. Aksi dapat dilakukan dengan mark() atau removeMark(). Penanda tidak memengaruhi kepemilikan wilayah (tidak membuat petak menjadi wilayah sekutu). Penanda hanya bisa dirasakan oleh robot sekutu melalui senseNearbyMapInfos(). Penanda secara umum tersebut hanya boleh ditempatkan pada petak yang dapat diwarnai (tidak ada tembok atau reruntuhan). Selain itu, terdapat Fungsi khusus penanda, yakni markTowerPattern() dan markResourcePattern()menggunakan 25 cat dan menandai area 5×5 dengan pola yang sesuai.
- Mentransfer Cat (Transferring)
  Hanya dapat dilakukan oleh robot mopper, dan hanya dapat dilakukan jika action cooldown < 10 (dapat dicek dengan isActionReady()). Aksi ini dapat dilakukan dengan memanggil prosedur transferPaint(). Jarak maksimal: $\sqrt{2}$ (~ 1.41 petak). Mentransfer jumlah cat tertentu ke target (robot atau menara sekutu). Jika jumlah yang diberikan negatif, maka mopper justru mengambil cat dari target. Setelah dilakukan, action cooldown bertambah sesuai nilai yang disebutkan di deskripsi mopper.

- Menarik cat dari menara (withdrawing)
  Semua robot bisa menarik cat dari menara sekutu untuk mengisi ulang cadangan cat mereka. Memerlukan action cooldown < 10. Setelah dilakukan, action cooldown bertambah 10. Jarak maksimal: $\sqrt{2}$ (~ 1.41 petak). Dilakukan dengan transferPaint() menggunakan nilai negatif untuk menunjukkan jumlah yang ingin ditarik.
- Berkomunikasi (communitating)
  Robot menggunakan sendMessage() untuk mengirim pesan ke menara sekutu dalam jangkauan. Menara menggunakan sendMessage() untuk mengirim pesan ke robot sekutu dalam jangkauan. Robot menggunakan readMessages() untuk membaca pesan baru yang diterima dalam 5 ronde terakhir dari buffer. Kedua metode ini tidak menambah action cooldown. Syarat komunikasi: kedua unit harus berada di petak yang terhubung melalui jalur cat sekutu.
- Menghancurkan Diri Sendiri (Disintegrating)
  Robot apa pun bisa memanggil disintegrate(). Menghancurkan robot tersebut secara instan. Tidak ada cooldown khusus untuk aksi ini.

1.3 Batasan Program

1.3.1 Bytecode

Jumlah komputasi yang boleh dilakukan oleh robot pada setiap giliran sangat terbatas. Bytecode menjadi ukuran yang praktis untuk menghitung komputasi di bahasa pemrograman seperti Java, di mana satu bytecode Java kira-kira setara dengan satu operasi dasar seperti pengurangan atau pengambilan field, dan satu baris kode biasanya mengandung beberapa bytecode (untuk detail lebih lanjut bisa dilihat di tautan tersebut). Karena bytecode merupakan bagian dari kode yang sudah dikompilasi, program yang sama akan selalu menghasilkan bytecode yang identik dan karena itu membutuhkan jumlah komputasi yang persis sama pada input yang sama. Keunggulan ini sangat membantu karena kita bisa menghindari penggunaan waktu sebagai ukuran komputasi, yang sering menimbulkan masalah seperti perilaku tidak deterministik. Dengan adanya batas bytecode, menjalankan ulang pertandingan yang sama antara bot yang sama akan selalu menghasilkan hasil yang persis identik, fitur yang sangat berguna untuk proses debugging.

Setiap ronde, setiap robot mengambil gilirannya secara berurutan. Jika sebuah robot mencoba melebihi batas bytecode-nya (biasanya terjadi tanpa disengaja, misalnya karena ada loop yang terlalu besar atau hal serupa), komputasinya akan langsung dihentikan sementara dan kemudian dilanjutkan kembali tepat dari titik itu pada giliran ronde berikutnya. Kode akan berjalan lagi dengan normal, tetapi hal ini bisa menimbulkan masalah. Misalnya, robot sempat memeriksa apakah sebuah petak masih kosong, lalu komputasinya terpotong, robot lain bergerak dulu, dan ketika giliran robot itu dilanjutkan ia mencoba bergerak ke petak yang sekarang sudah ditempati orang lain. Untuk menghindari situasi seperti ini, gunakan fungsi Clock.yield() untuk mengakhiri giliran robot secara sengaja. Fungsi ini akan menghentikan komputasi tepat di tempat yang kamu inginkan, lalu melanjutkannya dari baris kode berikutnya pada giliran ronde selanjutnya.

Batas bytecode untuk semua robot adalah 17500, sedangkan batas bytecode untuk semua menara adalah 20000.

1.3.2 Package

Pemain dapat menggunakan kelas dari paket mana pun yang tercantum dalam AllowedPackages.txt, kecuali kelas yang tercantum dalam DisallowedPackages.txt. File-file ini dapat ditemukan di sini. Selain itu, pembatasan berikut juga berlaku:

- Object.wait, Object.notify, Object.notifyAll, Class.forName, dan String.intern tidak diperbolehkan.
- java.lang.System hanya mendukung out, arraycopy, dan getProperty.
- Selain itu, getProperty hanya boleh digunakan untuk mengambil properti dengan nama yang diawali dengan "bc.testing.".
- java.io.PrintStream tidak boleh digunakan untuk membuka file.

Perlu diperhatikan bahwa pelanggaran terhadap salah satu pembatasan di atas akan menyebabkan robot meledak saat dijalankan, meskipun file sumber berhasil dikompilasi tanpa masalah.

1.3.3 Memory

Robot harus menjaga penggunaan memori tetap wajar. Jika sebuah robot menggunakan lebih dari 8 MB heap space selama turnamen atau pertandingan scrimmage, robot tersebut dapat meledak.

1.3.4 Execution Time

Robot harus menjaga waktu eksekusinya tetap wajar. Walaupun penghitungan bytecode memang dimaksudkan untuk tujuan ini, beberapa penggunaan Java yang sangat canggih dapat menyebabkan robot berjalan sangat lambat meskipun masih berada dalam batas bytecode.

Untuk mengurangi beban pada server, diberlakukan batas total waktu eksekusi per tim. Waktu ini dihitung secara kumulatif untuk semua robot dalam satu pertandingan, dan akan di-reset antar pertandingan dalam satu game. Jika sebuah tim melebihi batas ini, tim tersebut akan otomatis menyerah dan kalah dalam game. Nilai batas ini dapat diakses melalui konstanta game, dan Anda dapat mengecek berapa banyak waktu yang telah berlalu melalui kelas Clock.

Hal ini seharusnya tidak mempengaruhi sebagian besar pemain. Jika Anda mendapati bahwa Anda mencapai batas tersebut, kemungkinan ada optimasi yang perlu dilakukan pada kode Anda.

2 Landasan Teori

2.1 Algoritma Greedy

Algoritma greedy merupakan salah satu metode yang paling populer untuk memecahkan masalah optimasi (optimization problems), dapat berupa maksimasi atau minimasi. Secara harfiah, "*greedy*" berarti rakus atau loba. Dalam konteks komputasi, algoritma greedy membangun solusi dari suatu permasalahan secara langkah per langkah sedemikian sehingga ia mengambil pilihan yang menawarkan pilihan paling jelas dan langsung pada setiap langkah. Dengan kata lain, ia memilih solusi terbaik secara lokal pada setiap langkah dengan harapan pilihan tersebut akan menghasilkan solusi global yang optimal.

Ciri khas dari algoritma greedy adalah sifatnya yang iteratif dan sederhana. ia tidak pernah menyesali pilihan yang telah dibuat. Sekali sebuah pilihan diputuskan pada suatu langkah, maka pilihan tersebut tidak dapat diubah atau dibatalkan pada langkah selanjutnya. Proses pemilihan ini terus dilakukan hingga tidak ada lagi kandidat yang dapat dipilih atau hingga solusi akhir terbentuk.

Pendekatan greedy banyak digunakan dalam berbagai permasalahan optimasi karena implementasinya relatif sederhana dan memiliki efisiensi waktu yang baik. Beberapa algoritma greedy yang populer adalah Fractional Knapsack, algoritma Dijkstra, algoritma Kruskal, pengkodean Huffman, dan algoritma Prim. Namun demikian, algoritma greedy tidak selalu menjamin diperolehnya solusi optimal. Apabila solusi optimal tidak tercapai, maka solusi yang dihasilkan disebut sebagai solusi sub-optimum atau pseudo-optimum. Terdapat beberapa alasan mengapa algoritma greedy tidak selalu menghasilkan solusi optimal.

- Algoritma greedy tidak melakukan evaluasi terhadap seluruh kemungkinan solusi yang tersedia sebagaimana yang dilakukan pada metode exhaustive search atau brute force.
- Dalam suatu permasalahan dapat terdapat beberapa kemungkinan fungsi seleksi (*selection function*) yang berbeda. Apabila fungsi seleksi yang dipilih tidak tepat, maka solusi yang dihasilkan dapat menjadi tidak optimal.

Agar algoritma greedy dapat menghasilkan solusi optimal, permasalahan tersebut harus memiliki dua sifat utama berikut.

- ***Greedy Choice Property*** - Solusi optimal dapat dibangun dengan membuat pilihan lokal terbaik pada setiap langkah
- ***Optimal Substructure*** Solusi optimal dari suatu masalah mengandung solusi optimal untuk submasalahnya.

Meskipun demikian, algoritma greedy tetap sering digunakan dalam praktik karena mampu menghasilkan solusi hampiran (approximation) yang cukup baik dalam waktu komputasi yang relatif singkat. Hal ini menjadi sangat berguna pada permasalahan dengan ukuran data yang besar, di mana pencarian solusi optimal secara eksak memerlukan waktu komputasi yang sangat lama. Sebagai contoh, pada permasalahan Travelling Salesman Problem (TSP) dengan jumlah simpul yang besar, penggunaan algoritma brute force untuk menemukan tur dengan bobot minimum membutuhkan waktu komputasi yang bersifat eksponensial. Dengan menggunakan algoritma greedy, meskipun solusi optimal tidak selalu diperoleh, solusi yang dihasilkan dapat dianggap sebagai pendekatan

terhadap solusi optimal atau pseudo-optimum.

Selain itu, apabila suatu algoritma greedy mampu menghasilkan solusi optimal pada suatu permasalahan tertentu, maka keoptimalan tersebut harus dapat dibuktikan secara matematis. Pembuktian optimalitas algoritma greedy sering kali menjadi tantangan tersendiri dalam analisis algoritma. Jika pembuktian optimalitas tidak dapat dilakukan, maka salah satu cara untuk menunjukkan keterbatasan algoritma greedy adalah dengan memberikan counterexample, yaitu contoh kasus yang memperlihatkan bahwa solusi yang dihasilkan tidak selalu optimal. Contoh kasus seperti ini dapat ditemukan pada beberapa variasi persoalan penukaran uang (coin change problem), di mana algoritma greedy terkadang menghasilkan solusi optimal pada beberapa kasus, tetapi tidak pada kasus lainnya.

2.2 Elemen-Elemen Algoritma Greedy

Untuk mencapai tujuannya, sebuah algoritma greedy disusun oleh beberapa elemen utama yang membentuk kerangka kerjanya. Elemen-elemen tersebut digunakan untuk menentukan cara kandidat dipilih dan cara solusi dibangun secara bertahap.

2.2.1 Himpunan Kandidat (C)

Himpunan kandidat berisi elemen-elemen yang berpotensi menjadi pembentuk solusi pada setiap langkah. Pada setiap langkah, satu buah kandidat dipilih dari himpunan ini untuk dipertimbangkan.

2.2.2 Fungsi Seleksi

Fungsi ini dinyatakan dengan predikat seleksi. Fungsi seleksi bertugas untuk memilih kandidat yang paling memungkinkan mencapai solusi optimal pada setiap langkah. Kandidat yang sudah dipilih pada suatu langkah tidak akan dipertimbangkan lagi pada langkah selanjutnya.

2.2.3 Fungsi Kelayakan

Fungsi ini dinyatakan dengan predikat layak. Fungsi kelayakan merupakan fungsi yang memeriksa apakah suatu kandidat yang telah dipilih dapat memberikan solusi yang layak, yaitu kandidat tersebut bersama-sama dengan elemen yang sudah ada dalam himpunan solusi tidak melanggar batasan (*constraint*) yang ada. Kandidat yang memenuhi syarat dimasukkan ke dalam himpunan solusi, sedangkan kandidat yang tidak memenuhi syarat akan dibuang dan tidak dipertimbangkan lagi.

2.2.4 Himpunan Solusi (S)

Himpunan solusi berisi kandidat-kandidat yang terpilih sebagai solusi persoalan. Artinya, himpunan solusi adalah himpunan bagian dari himpunan kandidat yang memenuhi kriteria tertentu.

2.2.5 Fungsi Solusi

Fungsi solusi merupakan fungsi yang menentukan apakah himpunan solusi yang sedang dibangun sudah memenuhi kondisi yang diinginkan atau belum. Fungsi ini berperan

untuk memeriksa apakah proses pemilihan kandidat dapat dihentikan karena solusi yang dihasilkan sudah dianggap cukup atau lengkap sesuai tujuan permasalahan. Jika kondisi solusi telah terpenuhi, maka proses konstruksi solusi dihentikan. Sebaliknya, jika belum terpenuhi, proses pemilihan kandidat akan dilanjutkan dengan menambahkan elemen berikutnya ke dalam himpunan solusi.

2.2.6 Fungsi Objektif

Fungsi objektif merupakan fungsi yang bertujuan untuk mengoptimalkan nilai solusi, baik dengan cara memaksimumkan atau meminimumkan nilai tersebut.

Dengan memanfaatkan komponen-komponen tersebut, dapat disimpulkan bahwa algoritma greedy bekerja dengan mencari suatu himpunan bagian S dari himpunan kandidat C. Himpunan S harus memenuhi sejumlah kriteria yang telah ditentukan, yaitu membentuk sebuah solusi yang valid serta menghasilkan nilai yang optimal berdasarkan fungsi objektif yang digunakan.

2.3 Gambaran Umum Permainan Battlecode

2.3.1 Robot

Terdapat tiga jenis robot yang dapat diproduksi: Soldier, Splasher, dan Mopper. Setiap robot berjalan secara independen dengan salinan kode yang sama, tetapi tidak dapat berbagi variabel satu sama lain karena dijalankan pada JVM terpisah. Masing-masing robot memiliki atribut HP, kapasitas cat, biaya produksi, dan *cooldown* yang berbeda. Cat adalah sumber daya vital bagi setiap robot — robot dengan cadangan cat rendah akan mengalami *cooldown* yang lebih panjang, dan jika cat habis sepenuhnya, robot tidak dapat bergerak maupun beraksi serta kehilangan 20 HP per ronde. Setiap robot hanya bisa melihat lingkungan dalam radius $\sqrt{20}$ ($\approx$4.47 petak) dari posisinya.

Gambar 1: Radius penglihatan robot

2.3.2 Menara

Menara berperan ganda: sebagai tempat *spawn* robot dan sebagai unit tempur yang dapat menyerang musuh. Terdapat tiga jenis menara: *Money Tower* (menghasilkan *chips* per ronde), *Paint Tower* (menghasilkan cat per ronde), dan *Defense Tower* (memiliki serangan lebih kuat serta memberikan bonus serangan pasif ke seluruh menara sekutu). Menara hanya bisa dibangun di atas reruntuhan (*ruins*) dengan terlebih dahulu mewarnai pola 5×5 yang sesuai di sekitarnya, kemudian memanggil `completeTowerPattern()`. Jumlah menara per tim dibatasi maksimal 25. Setiap menara dapat di-*upgrade* hingga level 3 menggunakan `upgradeTower()` dengan mengorbankan sejumlah *chips*.

2.3.3 Resource: *Paint* dan *Chips*

Permainan menggunakan dua jenis sumber daya utama: cat (*paint*) dan koin (*chips*).

Chips bersifat *shared* di tingkat tim — seluruh *chips* yang dihasilkan *Money Tower* langsung masuk ke pool bersama dan dapat digunakan oleh semua unit dalam tim tanpa perlu ditransfer secara manual. *Chips* digunakan untuk memproduksi robot baru dan meng-*upgrade* menara.

Cat (*paint*) bersifat individual — setiap robot dan menara menyimpan cat dalam kapasitas masing-masing secara terpisah. *Paint Tower* menghasilkan cat dan menyimpannya di dalam tower itu sendiri, sehingga tidak langsung tersebar ke seluruh tim. Robot harus secara aktif mendatangi tower dan menarik cat menggunakan `transferPaint()` dengan nilai negatif. Semua jenis robot dapat mengambil cat dari tower sekutu manapun, sedangkan hanya Mopper yang dapat memberikan cat kepada robot lain.

Cat dikonsumsi setiap ronde bergantung pada jenis wilayah yang ditempati robot:

- Di wilayah sekutu: tidak ada penalti
- Di wilayah netral: -1 cat per ronde
- Di wilayah musuh: -2 cat per ronde

Khusus Mopper, seluruh penalti cat dilipatgandakan ($\times 2$) saat berada di wilayah musuh. Selain itu, terdapat penalti tambahan sebesar 1 cat per robot sekutu yang berada di 8 petak sekitar, yang mendorong robot untuk tidak berkerumun. Jika cat habis sepenuhnya, robot tidak dapat bergerak maupun beraksi dan kehilangan 20 HP per ronde hingga mendapat cat kembali.

2.3.4 Mekanisme Turn

Permainan berjalan secara *turn-based* di mana setiap robot mendapat satu giliran per ronde. Dalam satu giliran, robot dapat melakukan berbagai aksi seperti bergerak, menyerang atau mewarnai petak, berkomunikasi dengan menara, serta mentransfer cat — namun setiap aksi terikat oleh *cooldown* tersendiri yang membatasi seberapa sering aksi tersebut dapat dilakukan. Batas *bytecode* per giliran adalah 17.500 untuk robot dan 20.000 untuk menara; jika terlampaui, eksekusi dipotong dan dilanjutkan pada ronde berikutnya. Setiap giliran harus diakhiri dengan `Clock.yield()`.

2.4 Cara Kerja Program Secara Umum

2.4.1 Alur Eksekusi Bot

Bot Battlecode ditulis dalam bahasa Java dan dikompilasi menggunakan Gradle sebagai build tool. Seluruh kode bot berada di dalam direktori `src/` dengan masing-masing bot di dalam package tersendiri. Untuk mengompilasi dan menjalankan pertandingan, cukup jalankan perintah `./gradlew run` dari direktori root proyek. Gradle akan mengompilasi kode secara otomatis, kemudian engine Battlecode akan menjalankan simulasi pertandingan antara dua bot yang telah dikonfigurasi di berkas `build.gradle`. Hasil pertandingan disimpan sebagai berkas replay yang dapat ditonton melalui aplikasi client yang disertakan dalam repositori.

Saat simulasi berjalan, engine Battlecode akan meng-instansiasi setiap robot sebagai proses yang berjalan di atas Java Virtual Machine (JVM). Setiap robot mendapatkan salinan kode yang sama, namun masing-masing berjalan secara sepenuhnya independen dan tidak dapat berbagi memori atau variabel satu sama lain. Ketika sebuah robot pertama kali di-spawn, engine memanggil fungsi `run(RobotController rc)` satu kali. Fungsi ini kemudian berjalan dalam loop tak terbatas yang mewakili keseluruhan hidup robot, di mana setiap iterasi merepresentasikan satu ronde permainan. Di akhir setiap iterasi, robot wajib memanggil `Clock.yield()` untuk menyerahkan giliran kembali ke engine, sebelum loop dilanjutkan ke iterasi berikutnya pada ronde berikutnya.

2.4.2 RobotController sebagai Antarmuka Engine

Satu-satunya cara robot berinteraksi dengan dunia permainan adalah melalui objek `RobotController` yang diinjeksikan oleh engine ke dalam fungsi `run()`. Objek ini menyediakan seluruh API yang dibutuhkan robot untuk mengindera lingkungan sekitar, bergerak, menyerang, membangun tower, mengirim pesan, dan mentransfer cat. Beberapa contoh metode yang tersedia antara lain `rc.senseNearbyRobots()` untuk mendeteksi robot di sekitar, `rc.senseNearbyMapInfos()` untuk membaca kondisi tile-tile yang terlihat, `rc.move()` untuk bergerak, `rc.attack()` untuk menyerang atau mewarnai tile, serta `rc.sendMessage()` untuk mengirim pesan ke menara sekutu. Robot tidak bisa mendapatkan informasi apapun tentang kondisi permainan selain melalui metode-metode yang disediakan oleh `RobotController` ini.

2.4.3 Batasan Bytecode

Salah satu mekanisme paling krusial dalam Battlecode adalah pembatasan bytecode per giliran. Setiap robot diberi jatah maksimal 17.500 bytecode per ronde, sedangkan menara mendapat jatah 20.000 bytecode. Bytecode dalam konteks ini adalah instruksi-instruksi Java yang dieksekusi oleh JVM, sehingga setiap operasi seperti perulangan, pemanggilan fungsi, dan akses array mengonsumsi sejumlah bytecode. Jika jatah bytecode habis sebelum `Clock.yield()` dipanggil, engine akan memotong eksekusi secara paksa dan melanjutkannya pada ronde berikutnya. Hal ini berarti sebuah robot yang menjalankan komputasi terlalu berat bisa saja tidak sempat melakukan aksi penting seperti bergerak atau menyerang pada giliran tersebut. Keterbatasan ini mendorong pengembang bot untuk menulis kode yang efisien dan memprioritaskan operasi yang paling penting di awal setiap giliran.

2.4.4 Independensi dan Keterbatasan Penginderaan Robot

Setiap robot hanya dapat mengindera kondisi dalam radius penglihatan yang terbatas, yaitu $\sqrt{20}$ petak atau sekitar 4,47 petak dari posisinya. Robot sama sekali tidak mengetahui apa yang terjadi di luar radius tersebut. Karena setiap robot berjalan di JVM yang terpisah, tidak ada mekanisme berbagi variabel secara langsung antar robot. Variabel static dalam kode Java yang biasanya bersifat shared antar instance, dalam Battlecode berlaku per-robot karena setiap robot memiliki salinan kode dan memorinya sendiri. Konsekuensinya, setiap robot harus membuat keputusan secara mandiri hanya berdasarkan informasi yang dapat ia indera pada giliran tersebut, ditambah informasi yang tersimpan dalam variabel-variabel statisnya sendiri dari ronde-ronde sebelumnya.

2.4.5 Sistem Komunikasi Antar Unit

Meskipun robot beroperasi secara independen, Battlecode menyediakan mekanisme komunikasi terbatas melalui sistem pesan. Robot dapat mengirim pesan berukuran 4 byte ke menara sekutu menggunakan `rc.sendMessage()`, dengan syarat keduanya terhubung melalui jalur cat sekutu yang tidak terputus. Menara dapat membaca pesan yang masuk menggunakan `rc.readMessages()` dan kemudian meneruskan informasi tersebut ke robot lain yang berada dalam jangkauannya. Mekanisme ini memungkinkan terjadinya koordinasi sederhana antar unit, misalnya robot yang menemukan tile musuh dapat melaporkan lokasinya ke menara, kemudian menara meneruskan informasi tersebut ke robot lain yang lebih sesuai untuk menanganinya. Selain pesan, robot juga dapat membaca penanda visual yang ditinggalkan pada tile-tile peta untuk koordinasi lebih sederhana tanpa perlu koneksi langsung.

2.4.6 Penerapan Greedy pada Bot Battlecode

Karakteristik permainan Battlecode yang mengharuskan setiap robot mengambil keputusan secara lokal, cepat, dan tanpa informasi global menjadikannya sangat cocok untuk pendekatan greedy. Tidak ada waktu maupun informasi yang cukup bagi robot untuk mengevaluasi semua kemungkinan aksi dan dampak jangka panjangnya. Oleh karena itu, setiap robot menerapkan heuristik lokal untuk memilih aksi yang tampak paling menguntungkan pada kondisi saat itu. Keputusan diambil secara hierarkis, di mana kondisi paling mendesak seperti kehabisan cat atau kehadiran musuh diperiksa terlebih dahulu, sebelum robot melanjutkan ke tindakan yang lebih bersifat ekspansif. Setiap aksi yang dipilih tidak dapat dibatalkan, dan robot langsung melanjutkan ke giliran berikutnya tanpa mengevaluasi ulang pilihan yang sudah dibuat, yang merupakan ciri khas dari algoritma greedy.

3 Aplikasi Strategi Greedy

3.1 Pemetaan Permasalahan Battlecode ke Algoritma Greedy

Dalam implementasi pada kode program, persoalan Battlecode 2025 dipetakan sebagai persoalan pengambilan keputusan lokal terbaik di setiap giliran. Bot tidak menghitung seluruh kemungkinan strategi sampai akhir permainan, tetapi memilih aksi yang saat itu paling menguntungkan berdasarkan kondisi sekitar, seperti ruin terdekat, sebaran paint, musuh yang terlihat, jumlah paint, dan posisi robot.

3.1.1 Main

1. **Himpunan Kandidat (*Candidate Set*)**
   Himpunan kandidat adalah semua aksi yang mungkin dilakukan oleh setiap unit pada setiap giliran berdasarkan kondisi yang dapat diindera dalam radius pandang. Untuk **soldier**, kandidatnya adalah seluruh tile yang dapat diindera: tile kosong (*empty*), tile musuh, tile berisi ruin, dan lokasi tower musuh. Untuk **tower**, kandidatnya adalah seluruh jenis unit yang dapat di-*spawn* (soldier, mopper, splasher) beserta seluruh tile di sekitar tower sebagai lokasi *spawn*. Untuk **mopper**, kandidatnya adalah seluruh tile musuh yang terlihat dan koordinat *dispatch* yang diterima melalui pesan. Untuk **splasher**, kandidatnya adalah seluruh lokasi tower musuh yang dilaporkan melalui pesan dan tile musuh yang terlihat.
2. **Fungsi Seleksi (*Selection Function*)**
   Fungsi seleksi digunakan untuk menentukan kandidat terbaik secara lokal dari himpunan kandidat dengan harapan bahwa kandidat tersebut merupakan kandidat terbaik secara global. Untuk algoritma unit **soldier**, fungsi seleksi digunakan untuk mencari tile berisi ruin, kemudian memilih tile kosong yang paling dekat dengan unit jika tidak terdapat tile berisi ruin. Untuk **tower**, fungsi seleksi memilih jenis unit berdasarkan kondisi tile di sekitarnya, jika terdapat tile musuh di dalam jangkauan penglihatan maka tower akan *spawn* unit mopper dan jika jumlah tile musuh lebih dari 12 maka tower akan *spawn* unit splasher. Jika tidak terdapat tile musuh di sekitar tower, maka *spawn* unit akan dilakukan secara bergilir yaitu unit soldier secara default, unit mopper setiap 5 unit yang di-*spawn* dan unit splasher setiap 25 unit yang di-*spawn*. Untuk unit **mopper**, fungsi seleksi mengecek pesan dari tower yang masuk terlebih dahulu sebagai tujuan utama dari perpindahan, jika tidak terdapat pesan maka unit akan mencari tile musuh sebagai tujuan utama. Terakhir, unit **splasher** memiliki fungsi seleksi yang mencari tile dengan jumlah tile musuh yang bersebelahan paling banyak.
3. **Fungsi Kelayakan (*Feasibility Function*)**
   Fungsi kelayakan menentukan jika kandidat yang dipilih dapat dijalankan pada giliran saat ini berdasarkan kondisi robot saat ini. Kandidat terbaik untuk target eksplorasi global (target simetris setiap unit dan target dari pesan yang diterima mopper) akan dinyatakan tidak layak jika sudah berada dalam jangkauan penglihatan unit, jika hal ini terjadi maka unit akan menentukan target eksplorasi yang baru. Selanjutnya, kandidat terbaik untuk target eksplorasi lokal (target di dalam jangkauan) dianggap tidak layak jika di antara target dan unit terdapat tembok berurutan yang lurus, hal ini diperiksa dalam fungsi blockedByWall. Selain kedua fungsi kelayakan tersebut, digunakan juga fungsi kelayakan yang berasal

dari library battlecode seperti canBuildRobot, canUpgradeTower, canSenseRobotAtLocation, canMarkTowerPattern, canCompleteTowerPattern, canSendMessage, canAttack, canMove, canMopSwing, canTransferPaint.

4. **Himpunan Solusi (*Solution Set*)**
   Himpunan solusi adalah satu aksi yang dipilih dan dieksekusi per giliran per unit. Karena setiap robot hanya dapat melakukan satu aksi per giliran akibat batasan *action cooldown* Battlecode, solusi selalu berupa elemen tunggal, misalnya soldier menyerang tile musuh di koordinat (x, y), tower men-*spawn* MOPPER ke arah NORTHEAST, mopper bergerak menuju koordinat *dispatch* (x, y), atau splasher menyerang tower musuh di koordinat (x, y).
5. **Fungsi Solusi (*Solution Function*)**
   Fungsi solusi menentukan kapan proses seleksi berhenti dan solusi dianggap lengkap untuk satu giliran. Untuk setiap unit, solusi dianggap selesai segera setelah **satu aksi berhasil dieksekusi** atau **satu kondisi `return` tercapai** dalam rantai prioritas. Tidak ada *backtracking* — begitu soldier memilih untuk menyerang tile musuh dan berhasil, giliran itu selesai tanpa mempertimbangkan apakah ada tile lain yang lebih menguntungkan secara global.
6. **Fungsi Objektif (*Objective Function*)**
   Fungsi objektif adalah ukuran keberhasilan yang ingin dimaksimalkan oleh keseluruhan strategi bot. Dalam Battlecode 2025, kondisi kemenangan adalah menguasai lebih banyak tile pada akhir permainan, sehingga fungsi objektif bot ini adalah:

$$\max \sum_{\text{giliran}} \Delta\text{tile sekutu} \tag{1}$$

   yaitu memaksimalkan pertambahan jumlah tile yang dicat dengan warna sekutu secara kumulatif sepanjang permainan. Setiap aksi greedy yang dipilih — mengcat tile kosong terdekat, membersihkan tile musuh, membangun tower baru — berkontribusi langsung pada nilai fungsi objektif ini tanpa perlu merencanakan urutan global.

Tabel 3: Ringkasan Elemen Algoritma *Greedy* Bot `andraimprove`

Elemen Greedy	**Realisasi dalam Bot `andraimprove`**
Himpunan kandidat	Semua tile dan unit dalam radius pandang yang dapat diindera tiap giliran
Fungsi seleksi	Cari tile berisi ruin, jika tidak ada maka pilih tile kosong dengan jarak terkecil. Prioritas *spawn* dilakukan berdasarkan kondisi tile sekitar tower
Fungsi kelayakan	Cek jarak unit dengan target eksplorasi global dan lokal sebelum bergerak
Himpunan solusi	Satu aksi per giliran per unit (dibatasi *action cooldown*)
Fungsi solusi	Berhenti setelah satu aksi berhasil dieksekusi (*early return*)
Fungsi objektif	Maksimalkan jumlah tile sekutu secara kumulatif hingga akhir permainan

3.1.2 Alternatif 1

1. **Himpunan Kandidat (*Candidate Set*)**
   Himpunan kandidat adalah semua aksi yang mungkin dilakukan oleh setiap unit pada setiap giliran berdasarkan kondisi yang dapat diindera dalam radius pandang. Untuk **soldier**, kandidatnya adalah seluruh tile yang dapat diindera: tile kosong (*empty*), tile musuh, tile berisi ruin, dan lokasi tower musuh. Untuk **tower**, kandidatnya adalah seluruh jenis unit yang dapat di-*spawn* (soldier, mopper, splasher) beserta seluruh tile di sekitar tower sebagai lokasi *spawn*. Untuk **mopper**, kandidatnya adalah seluruh tile musuh yang terlihat dan koordinat *dispatch* yang diterima melalui pesan. Untuk **splasher**, kandidatnya adalah seluruh lokasi tower musuh yang dilaporkan melalui pesan dan tile musuh yang terlihat.
2. **Fungsi Seleksi (*Selection Function*)**
   Fungsi seleksi menentukan kandidat mana yang dipilih sebagai aksi terbaik pada giliran saat ini, tanpa mempertimbangkan konsekuensi jangka panjang. Untuk **soldier**, fungsi seleksi memilih tile kosong dengan jarak kuadrat terkecil dari posisi saat ini. Untuk **tower**, fungsi seleksi memilih jenis unit berdasarkan urutan prioritas bertingkat: soldier jika ronde < 250 dan jumlah soldier < 10, splasher jika ada intel dan *cooldown* terpenuhi, mopper jika rasio soldier/mopper > 3 atau tidak ada mopper, dan soldier sebagai *fallback*. Untuk **mopper**, fungsi seleksi memilih tile musuh terdekat yang terlihat, atau koordinat *dispatch* dari tower jika tidak ada tile musuh yang terlihat secara langsung.
3. **Fungsi Kelayakan (*Feasibility Function*)**
   Fungsi kelayakan memeriksa apakah suatu kandidat dapat dieksekusi pada giliran saat ini berdasarkan kondisi robot. Untuk semua unit, kandidat dinyatakan **tidak layak** jika paint terlalu rendah untuk beraksi (soldier: < 15 untuk kondisi darurat, < 50 untuk refuel terlebih dahulu), HP terlalu rendah dan ada ancaman musuh (soldier: $HP \leq 100$ dan ada mopper atau tower musuh di sekitar sehingga harus retreat), sumber daya tidak mencukupi untuk *spawn* (tower: pengecekan `canAffordSoldier`, `canAffordMopper`, `canAffordSplasher`), atau aksi fisik tidak dapat dilakukan (`canAttack`, `canMove`, `canBuildRobot` mengembalikan `false`). Kandidat dinyatakan **layak** hanya jika semua syarat tersebut terpenuhi.
4. **Himpunan Solusi (*Solution Set*)**
   Himpunan solusi adalah satu aksi yang dipilih dan dieksekusi per giliran per unit. Karena setiap robot hanya dapat melakukan satu aksi per giliran akibat batasan *action cooldown* Battlecode, solusi selalu berupa elemen tunggal, misalnya soldier menyerang tile musuh di koordinat (x, y), tower men-*spawn* MOPPER ke arah NORTHEAST, mopper bergerak menuju koordinat *dispatch* (x, y), atau splasher menyerang tower musuh di koordinat (x, y).
5. **Fungsi Solusi (*Solution Function*)**
   Fungsi solusi menentukan kapan proses seleksi berhenti dan solusi dianggap lengkap untuk satu giliran. Untuk setiap unit, solusi dianggap selesai segera setelah **satu aksi berhasil dieksekusi** atau **satu kondisi `return` tercapai** dalam rantai prioritas. Tidak ada *backtracking* — begitu soldier memilih untuk menyerang tile musuh dan berhasil, giliran itu selesai tanpa mempertimbangkan apakah ada tile lain yang lebih menguntungkan secara global.
6. **Fungsi Objektif (*Objective Function*)**

Fungsi objektif adalah ukuran keberhasilan yang ingin dimaksimalkan oleh keseluruhan strategi bot. Dalam Battlecode 2025, kondisi kemenangan adalah menguasai lebih banyak tile pada akhir permainan, sehingga fungsi objektif bot ini adalah:

$$\max \sum_{\text{giliran}} \Delta\text{tile sekutu} \tag{2}$$

yaitu memaksimalkan pertambahan jumlah tile yang dicat dengan warna sekutu secara kumulatif sepanjang permainan. Setiap aksi greedy yang dipilih, misalnya mengcat tile kosong terdekat, membersihkan tile musuh, membangun tower baru berkontribusi, langsung pada nilai fungsi objektif ini tanpa perlu merencanakan urutan global.

Tabel 4: Ringkasan Elemen Algoritma *Greedy* Bot `andraimprove`

Elemen Greedy	**Realisasi dalam Bot** `andraimprove`
Himpunan kandidat	Semua tile dan unit dalam radius pandang yang dapat diindera tiap giliran
Fungsi seleksi	Pilih tile kosong terdekat (*distSq* minimum); prioritas bertingkat untuk *spawn*
Fungsi kelayakan	Cek paint, HP, resource, dan `canXxx()` sebelum eksekusi aksi
Himpunan solusi	Satu aksi per giliran per unit (dibatasi *action cooldown*)
Fungsi solusi	Berhenti setelah satu aksi berhasil dieksekusi (*early return*)
Fungsi objektif	Maksimalkan jumlah tile sekutu secara kumulatif hingga akhir permainan

3.1.3 Alternatif 2

1. **Himpunan Kandidat (*Candidate Set*)**
   Himpunan kandidat merupakan seluruh aksi yang mungkin dipilih oleh setiap unit pada suatu giliran berdasarkan kondisi lingkungan yang berada dalam radius penglihatan robot. Pada strategi alternatif 2, kandidat ditentukan oleh informasi lokal seperti keberadaan ruin, sebaran paint, posisi musuh, serta resource yang dimiliki robot.
   - Untuk soldier, kandidat meliputi seluruh target aksi yang terlihat di sekitarnya, yaitu ruin yang dapat dibangun, ruin musuh yang sedang dalam proses pembangunan, tower musuh yang terlihat, unit musuh di sekitar, serta tile kosong atau tile musuh yang dapat dicat. Selain itu soldier juga memiliki kandidat target eksplorasi seperti target simetri peta pada early game dan waypoint eksplorasi pada fase permainan selanjutnya.
   - Untuk tower, kandidat adalah seluruh jenis unit yang dapat di-spawn yaitu soldier, mopper, dan splasher, serta seluruh arah di sekitar tower yang dapat dijadikan lokasi spawn.
   - Untuk mopper, kandidat berupa seluruh tile yang memiliki enemy paint dalam radius penglihatan, unit musuh yang berada di sekitar tower sekutu, serta soldier sekutu yang membutuhkan transfer paint.

- Untuk splasher, kandidat adalah seluruh lokasi serangan yang dapat memberikan dampak area terbesar, yaitu area dengan konsentrasi enemy paint tertinggi yang berada dalam jangkauan serangan.

2. **Fungsi Seleksi (*Selection Function*)**
   Fungsi seleksi menentukan kandidat mana yang dipilih sebagai aksi terbaik pada giliran saat ini. Pada strategi alternatif 2, pemilihan aksi dilakukan menggunakan heuristik prioritas lokal yang mempertimbangkan kondisi sekitar tanpa menghitung konsekuensi jangka panjang.
   - Untuk tower, fungsi seleksi menentukan jenis unit yang akan di-spawn berdasarkan kondisi sekitar seperti keberadaan musuh, jumlah enemy paint, komposisi unit sekutu, dan fase permainan.
   - Untuk mopper, fungsi seleksi memprioritaskan pembersihan enemy paint pada area strategis seperti sekitar ruin, kemudian menyerang unit musuh atau membersihkan tile musuh terdekat.
   - Untuk splasher, fungsi seleksi memilih lokasi serangan yang menghasilkan jumlah tile musuh terbesar dalam area splash.
3. **Fungsi Kelayakan(*Feasibility Function*)**
   Fungsi kelayakan memeriksa apakah kandidat yang dipilih dapat dieksekusi pada giliran tersebut berdasarkan kondisi robot dan aturan permainan.Pada strategi ini, kandidat dinyatakan tidak layak apabila memenuhi kriteria berikut.
   - Jumlah paint robot berada di bawah batas minimum untuk melakukan aksi tertentu.
   - Target berada di luar jangkauan serangan atau gerakan robot.
   - Tile yang dituju tidak dapat dilalui atau tidak dapat dicat.
   - Ruin yang menjadi target sudah ditempati robot lain atau telah diklaim oleh unit lain.
   - Resource yang diperlukan untuk spawn unit atau upgrade tower tidak mencukupi.

   Kandidat hanya dinyatakan layak apabila seluruh kondisi tersebut terpenuhi sehingga aksi dapat dijalankan secara valid pada giliran tersebut.
4. **Himpunan Solusi (*Solution Set*)**
   Himpunan solusi adalah aksi yang akhirnya dipilih dan dieksekusi oleh robot pada setiap giliran. Karena setiap robot hanya dapat melakukan satu aksi utama per giliran akibat batasan mekanisme permainan Battlecode, maka solusi pada setiap langkah selalu berupa satu aksi tunggal.
5. **Fungsi Solusi (*Solution Function*)**
   Fungsi solusi menentukan kapan proses seleksi berhenti dan aksi dianggap selesai pada giliran tersebut. Dalam strategi alternatif 2, proses seleksi berhenti segera setelah satu aksi yang layak berhasil dijalankan.Dengan demikian, tidak terdapat proses backtracking atau evaluasi ulang terhadap kandidat lain yang mungkin lebih optimal secara global.
6. **Fungsi Objektif(*Objective Function*)**
   Fungsi objektif merupakan ukuran keberhasilan yang ingin dimaksimalkan oleh ke-

seluruhan strategi bot. Dalam Battlecode 2025, kemenangan ditentukan oleh jumlah tile yang berhasil dikuasai pada akhir permainan. Fungsi objective code adalah sebagai berikut.

$$\max \sum_{\text{giliran}} \Delta\text{tile sekutu} \tag{3}$$

yaitu memaksimalkan jumlah tile yang berhasil dicat oleh tim secara kumulatif sepanjang permainan. Setiap aksi greedy yang dipilih, seperti mengecat tile kosong, membersihkan enemy paint, membangun tower baru, serta menyerang area dengan konsentrasi musuh agar secara langsung berkontribusi terhadap peningkatan kontrol wilayah dan nilai fungsi objektif tersebut tanpa memerlukan perencanaan global yang kompleks.

Tabel 5: Ringkasan Elemen Algoritma *Greedy* Bot `andraimprove`

Elemen Greedy	**Realisasi dalam Bot `andraimprove`**
Himpunan kandidat	Semua target aksi dalam radius penglihatan, yaitu ruin, musuh, tile paint, target eksplorasi, dan pilihan spawn unit
Fungsi seleksi	Pemilihan aksi berdasarkan prioritas lokal seperti build ruin, serang musuh, bersihkan paint, atau eksplorasi
Fungsi kelayakan	Pengecekan resource (paint, chip), jarak target, kondisi tile, serta validitas ruin atau target
Himpunan solusi	Satu aksi yang dipilih dan dijalankan pada setiap giliran robot (dibatasi *action cooldown*)
Fungsi solusi	Proses seleksi berhenti setelah satu aksi berhasil dijalankan
Fungsi objektif	Memaksimalkan jumlah tile yang dikuasai tim melalui ekspansi, kontrol paint, dan pembangunan tower

3.2 Alternatif Strategi Greedy

Dalam menyelesaikan persoalan Battlecode, terdapat beberapa alternatif strategi greedy yang dapat digunakan. Setiap alternatif menggunakan heuristic yang berbeda.

3.2.1 Main

Tower
: Tower memiliki heuristik greedy yang menentukan prioritas 3 aksi berbeda. Pada prioritas tertinggi, tower akan mencoba untuk melakukan spawn unit robot pada direction yang dibuat selalu berubah untuk setiap unitnya. Spawn unit robot tersebut memiliki ketentuan sebagai berikut:

- Tower akan selalu spawn unit robot saat tersedia. Namun, jika jumlah tower sekutu sudah bernilai 6 atau lebih, tower akan mengurangi frekuensi spawn unit agar menghemat jumlah money dan paint yang dimiliki.
- Jumlah unit yang sudah di-spawn ditandai oleh *spawnedBot*, variabel tersebut menjadi penentu unit yang akan di-spawn.

- Pada kondisi default (jika kondisi di bawah tidak ada yang terpenuhi), tower akan spawn unit Soldier.
- Unit Mopper akan di-spawn jika $0 \equiv spawnedBot \pmod 5$ atau variabel $mustSpawnMopper$ bernilai true.
- Unit Splasher akan di-spawn jika jumlah round permainan sudah mencapai 1500 dan jika $0 \equiv spawnedBot \pmod{25}$ atau variabel $mustSpawnSplasher$ bernilai true.

Selain spawn unit robot, tower akan selalu membaca pesan yang datang untuk memberikan unit Mopper lokasi tujuan. Variabel $mustSpawnMopper$ akan bernilai true jika terdapat pesan ($messages$ tidak kosong) selama 3 round berturut-turut dan jika terdapat tile dengan paint musuh yang tidak terhalang oleh tembok. Variabel $mustSpawnSplasher$ akan bernilai true jika terdapat lebih dari 12 tile musuh di sekitarnya. Pada urutan prioritas kedua, tower akan mendeteksi robot di sekitarnya. Jika tower mendeteksi 1 robot musuh, maka tower akan melakukan serangan single-block. Namun jika tower mendeteksi lebih dari 1 robot musuh, maka tower akan melakukan serangan AoE. Pada urutan prioritas terakhir, tower akan melakukan upgrade. Aksi tersebut hanya akan dilakukan jika tower sudah berada dalam peta selama 500 round (untuk level 2) dan 1000 round (untuk level 3).

Soldier

Unit soldier memiliki 2 aksi utama, yaitu aksi attack dan aksi move. Aksi attack memiliki 3 kondisi dengan urutan prioritas yang berbeda, yaitu:

1. **Tile kosong dengan mark**: Unit soldier akan mengutamakan pembangunan tower dengan mengisi tile kosong yang memiliki mark di atasnya.
2. **Tile kosong pada jarak $\sqrt{2}$**: Unit soldier akan memeriksa dan mengecat tile yang langsung bersebelahan dengan tile posisinya saat ini jika tile tersebut kosong.
3. **Robot dan tower dengan tim yang berbeda**: Jika unit soldier mendeteksi musuh di dalam jangkauan attack dan kedua kondisi sebelumnya tidak terpenuhi, maka unit akan menyerang musuh yang pertama kali terdeteksi.

Selain aksi attack, unit soldier juga memiliki daftar urutan prioritas untuk aksi move. Tile yang menjadi tujuan unit ini ditentukan oleh kasus-kasus berikut:

1. **Unit berada dalam state "kembali"**: Unit soldier hanya akan memasuki state ini jika mendeteksi tile dengan paint musuh yang lebih dekat daripada tile kosong terdekat. Pada state ini, unit akan kembali ke tower yang terakhir dilalui (ditandai dengan $nearestTower$) untuk mengirimkan pesan lokasi tile musuh tersebut. Jika unit sudah mencapai tower terdekat, maka state unit akan berubah menjadi normal dan gerakan akan mengikuti urutan prioritas selanjutnya.
2. **Unit mendeteksi ruin**: Jika unit mendeteksi ruin dalam jangkauan penglihatannya dan jaraknya dengan ruin lebih dari 2, maka unit akan bergerak ke arah ruin tersebut.
3. **Tile kosong terdekat**: Unit mendeteksi setiap tile kosong yang ada di sekitarnya kemudian memilih tile dengan jarak yang paling kecil. Namun jika tile kosong yang terdeteksi terhalang oleh tembok lurus, maka tile tersebut dianggap *unreachable* dan akan diabaikan.
4. **Target simetris dari spawn**: Saat unit di-spawn pertama kali, unit akan mene-

tapkan 3 target tile simetris dari posisi spawn. Unit akan bergerak ke arah target ini hanya jika seluruh kondisi di atas tidak terpenuhi. Jika unit sudah mencapai target simetris, maka unit akan memasang target simetris selanjutnya.

Mopper
Pada setiap round, unit mopper hanya akan bergerak jika pada seluruh tile yang bersebelahan dengannya sudah tidak ada lagi tile musuh. Algoritma target move pada unit mopper memiliki 3 kasus berbeda, berikut adalah urutan prioritas kasus tersebut:

1. **Pesan dari tower**: Unit mopper akan membaca pesan yang datang sebagai arah utama pergerakan. Pesan yang diterima mopper berasal dari tower asalnya dan mengandung koordinat tile dengan paint musuh. Unit akan berhenti mengikuti pesan tersebut jika sudah berjarak kurang dari $\sqrt{20}$ petak.
2. **Tile musuh**: Unit mopper akan mencari tile dengan paint musuh di dalam jangkauan penglihatannya kemudian menjadikan tile tersebut sebagai target move.
3. **Target simetris dari spawn**: Serupa dengan algoritma unit soldier, jika unit mopper tidak menemukan tile musuh di sekitarnya dan tidak memiliki pesan maka unit akan bergerak ke arah target simetris yang sudah ditetapkan saat spawn.

Selain algoritma perpindahan, unit mopper juga akan mendeteksi unit-unit lain di sekitarnya. Unit akan melakukan mop swing jika terdapat robot musuh tepat di sebelah tilenya. Selain itu, unit akan mendeteksi robot dengan jumlah paint paling sedikit kemudian melakukan transfer paint dengannya.

Splasher
Unit splasher memiliki algoritma paling sederhana, yaitu mencari dan bergerak ke tile dengan lebih dari 6 tile musuh di sekitarnya. Jika unit tidak mendeteksi tile musuh, maka unit akan bergerak ke arah target simetris, serupa dengan unit soldier dan mopper. Karena resource splasher yang cenderung mahal, algoritma splasher didesain untuk melakukan damage sebesar mungkin kepada area musuh.

Pola Umum
Setiap unit didesain untuk bergerak secara agresif mengambil alih tile kosong di dalam map secepat mungkin. Jika unit tidak mendeteksi tile kosong dalam jangkauan, maka secara otomatis akan bergerak ke arah target simetris yang ditentukan sejak spawn pertama kali. Tower pada algoritma ini didesain untuk bersifat defensif dengan spawn unit mopper dan splasher jika terdapat tile musuh di sekitarnya dan menyerang musuh dalam jangkauan, sebaliknya jika tidak terdapat ancaman di sekitarnya maka tower akan terus menerus spawn unit sesuai dengan urutannya.

3.2.2 Alternatif 1

Tower
: Tower menggunakan heuristik greedy sederhana. Pertama, serang musuh yang terlihat di range of vision. Kedua, Upgrade dirinya sendiri apabila resource mencukupkan. Ketiga membaca, menyimpan, dan meneruskan pesan, baru dengan informasi yang dia miliki menspawn unit soldier yang sesuai. Keputusan soldier diprioritiskan sampai batas kuantitas atau ronde tertentu. Sedangkan mopper hanya muncul jika rasio atau informasi mengenai tile musuh muncul. Splasher muncul jika memenuhi syarat ketika ada informasi mengenai tower enemy yang memiliki health rendah. Pada algortima ini, tower difokuskan untuk tidak mengeluarkan banyak unit yang belum dibutuhkan.

Soldier
: Soldier memiliki daftar prioritas yang paling panjang, dan merupakan fokus dari startegi bot ini dikarenakan biaya spawnnya yang paling murah diantara kedua jenis robot lain (Mopper dan Splasher). Urutan dari priotitas adalah:

1. **Duration paint (<15)**: Langsung balik ke tower tanpa melakukan apapaun. Ini adalah heuristik survival: lebih baik tidak melakukan apapun daripada mati di tengah jalan.
2. **Refuel (<50)**: coba tarik paint dari tower terdekat, atau bergerak mendekat jika belum dalam jangkauan. Threshold 50 dipilih agar soldier selalu punya cukup paint untuk beberapa aksi ke depan.
3. **Raid Tower Musuh**: jika ada tower musuh dalam jangkauan penglihatan dan jumlah soldier sekutu >= 3, langsuing serang. Heurstik ini menghindari serangan sendirian yang berisiko. Bot menunggu kekuatan yang cukup sebelum menyerang tower.
4. **Bangun tower di ruin**: Jika ada ruin yang belum diklaim oleh soldier dan ronde sudah > 50, soldier mengambil ahli dan membangun. Threshold ronde 50 memberi waktu untuk menyebar dulu, sebelum mulai membangun.
5. **Retreat jika ada ancaman**: Jika HP <= 40 dan ada mopper atau tower musuh dalam jangkauan pandang, soldier munudur ke arah berlawanan. Heuritstik ini hanya aktif ketika soldier sudah lemah, bukan sebagai respons default ketika melihat musuh.
6. **Pilih tile kosong terdekat**: dari semua tile kosong yang terlihat, pilih yang paling dekat. Ini menjadi heursitik greedy inti, yaitu selalu ambil tile yang paling mudah dijangkau, tanpa mempertimbangkan apakah ada tile yang lebih strategis di tempat lain.
7. **Eksplorasi determinsitik**: jika tidak ada tile kosong yang terlihat, bergerak ke salah satu dari 8 kandidat titik yang tersebar di sekitar tengah peta, bergiliran setiap kali titik lama sudah dicapai.

Mopper
: Mopper menerapkan heuristik berbasis dispatch. Prioritasnya adalah: baca pesan dari tower dan perbarui target jika ada koordinat yang lebih dekat, lalu refuel jika paint < 20, lalu lakukan swing jika ada musuh di sekitar, lalu transfer paint ke soldier yang kekurangan, lalu serang tile musuh terdekat yang terlihat, lalu bergerak ke koordinat dispatch jika tidak ada tile musuh yang terlihat, lalu eksplorasi jika tidak ada keduanya. Heuristik kuncinya adalah bahwa mopper tidak pernah mencari musuh secara mandiri. Mopper selalu bergerak ke koordinat yang sudah diketahui, baik dari penglihatan langsung maupun dari pesan tower.

Splasher
: Splasher juga berbasis dispatch, dengan tambahan fase pasca-serangan. Setelah menerima koordinat tower musuh dari pesan, splasher bergerak menuju tower tersebut dan menyerangnya. Ketika tower sudah hancur, splasher tidak langsung pergi. Ia mencari posisi terbaik untuk membersihkan sisa paint musuh di sekitar bekas tower, dengan memilih posisi yang memaksimalkan jumlah tile musuh dan tile kosong yang bisa dicakup sekaligus. Setelah 20 ronde tanpa target bersih yang ditemukan, splasher melepaskan target dan beralih ke tile musuh terdekat yang terlihat, atau eksplorasi jika tidak ada.

Pola Umum
: Setiap unit menyelesaikan kewajiban bertahan hidup sebelum melakukan aksi produktif (offense and defensive actions).Tower menyerang musuh sebelum spawn unit robots, soldier mengisi paint dan retreat sebelum ekspolrasi, mopper refuel sebelum mebersihkan tile, spalsher refuel sebelum menyerang tower target. Setelah kebutuhan dasar terpenuhi, setiap unit langsung beralih ke aksi paling ofensif. Tower nenyerang, soldier menyerang tower musuh, atau menguasai tile kosong, mopper membersihkan tile musuh di koordinat dari message dispatch, spalsher menghancurkan tower musuh yang lemah. Jika tidak ada aksi ofensif yang bisa dilakukan, unit baru akan beralih ke eksplorasi sebagai fallback akhir.

Pola ini menunjukkan tiga lapisan yang konsisten:

1. **Surive First**: Jangan sampai mati, jaga paint dan HP
2. **Attack When Possible**: ofensif tiap ada kesempatan.
3. **Explore as Fallback**: Jika tidak ada yang menjadi target, atau yang bida diserang kuasai wilayah baru.

3.2.3 Alternatif 2

Secara umum, strategi alternatif 2 berusaha menjaga keseimbangan antara ketahanan unit, ekspansi wilayah, dan tekanan terhadap musuh. Setiap robot terlebih dahulu memastikan kondisi dasarnya aman, seperti memiliki paint yang cukup untuk melakukan aksi. Setelah itu, robot memprioritaskan objective utama yang sedang aktif, misalnya pembangunan tower, pembersihan area musuh, atau penyerangan target yang terlihat. Jika tidak ada objektif penting yang dapat dilakukan, robot akan beralih ke eksplorasi untuk mencari peluang baru.

Untuk mempermudah implementasi, strategi ini dibangun dengan beberapa komponen utama yang saling mendukung sebagai berikut.

Infrastruktur

Struktur program dirancang secara modular dengan satu kelas utama yang mengatur perilaku setiap jenis robot untuk mengurangi konflik antar robot dan meningkatkan efisiensi ekspansi. Pada setiap ronde permainan, program akan mendeteksi tipe robot yang sedang dijalankan dan memanggil fungsi perilaku yang sesuai.

Beberapa struktur data juga digunakan untuk menyimpan informasi penting selama permainan, seperti sistem claim ruin yang menandai ruin yang sedang dikerjakan oleh soldier tertentu. Sistem ini menyimpan lokasi ruin, pemilik klaim, serta waktu klaim sehingga ruin yang tidak lagi aktif dapat kembali tersedia bagi unit lain.

Eksplorasi

Eksplorasi peta dilakukan menggunakan dua pendekatan utama. Pada fase awal permainan, robot memanfaatkan simetri peta untuk memperkirakan kemungkinan posisi lawan sehingga eksplorasi dapat dilakukan secara lebih terarah.

Setelah fase awal, eksplorasi dilakukan menggunakan sistem waypoint eksplorasi, yaitu sekumpulan titik tetap yang tersebar di berbagai bagian peta. Robot akan memilih waypoint terdekat yang belum pernah dikunjungi dan bergerak menuju titik tersebut. Mekanisme ini membantu robot menjelajah wilayah secara merata dan menghindari per-

gerakan berulang pada area yang sama.

Pergerakan

Pergerakan robot menggunakan heuristik greedy yang mengevaluasi beberapa arah gerakan yang mungkin dilakukan. Robot akan memilih arah yang paling mendekati target dengan mempertimbangkan kondisi tile di sekitarnya. Tile milik sekutu diberi prioritas lebih tinggi, sedangkan tile musuh cenderung dihindari untuk mengurangi risiko. Selain itu, robot juga menyimpan informasi lokasi sebelumnya dan arah gerakan terakhir untuk menghindari gerakan bolak-balik. Jika robot gagal membuat kemajuan selama beberapa ronde berturut-turut, target eksplorasi akan di-reset agar robot dapat keluar dari situasi *stuck*.

Manajemen Resource

Robot memiliki threshold tertentu untuk menentukan kapan harus kembali ke tower untuk melakukan refill. Soldier tidak akan memulai pembangunan tower jika jumlah paint belum mencukupi, sedangkan mopper tetap menyimpan cadangan paint sebelum mentransfer paint kepada soldier yang membutuhkan. Pendekatan ini memastikan bahwa unit tidak kehabisan resource di area depan dan tetap dapat menjalankan aksi penting secara konsisten.

Fase Permainan

Beberapa keputusan strategi juga dipengaruhi oleh fase permainan. Pada early game, produksi unit lebih difokuskan pada soldier untuk mempercepat eksplorasi dan ekspansi wilayah. Pada fase berikutnya, komposisi unit menjadi lebih beragam dengan penambahan mopper untuk menjaga wilayah serta splasher untuk mengendalikan area melalui serangan area.

Tower

Prioritas paling tinggi tower adalah menyerang musuh yang berada di sekitarnya, khususnya target dengan health terendah, agar peluang eliminasi meningkat. Berbeda dengan strategi yang tetap melakukan spawn meskipun ada ancaman, pada alternatif ini tower menghentikan seluruh aksi ekonomi dan produksi ketika musuh masih ada di sekitar. Dengan kata lain, selama area sekitar tower belum aman, tower hanya berfokus pada pertahanan.
Setelah area aman, tower akan mempertimbangkan upgrade, tetapi hanya pada money tower level 1 dan level 2, dengan syarat jumlah chip mencukupi, jumlah tower sekutu sudah minimal dua, dan tower memang bisa di-upgrade. Heuristik ini menunjukkan bahwa upgrade tidak dilakukan secara agresif sejak awal, melainkan ketika fondasi tim sudah cukup stabil.
Untuk produksi unit, tower menentukan jenis spawn melalui fungsi chooseSpawn, yang mempertimbangkan keberadaan musuh, jumlah enemy paint, komposisi pasukan sekutu, jumlah tower, dan ronde permainan. Urutan heuristiknya adalah sebagai berikut.

- Jika tower mendeteksi unit musuh bertipe **mopper**, maka tower langsung memilih spawn mopper sebagai respons untuk melawan unit pembersih lawan dengan unit utilitas yang sesuai.
- Jika ada musuh di sekitar atau terdapat ruin yang sedang terkontestasi oleh enemy paint, tower juga akan memilih mopper untuk mengamankan area sekitar tower atau ruin penting.

- Pada ronde awal dan tidak ada ancaman mendesak, tower memprioritaskan soldier untuk ekspansi, eksplorasi, dan pembangunan struktur.
- Jika jumlah tile musuh di sekitar sudah mencapai ambang tertentu, tower kembali memilih mopper untuk mengembalikan kontrol area.
- Bila jumlah mopper terlalu sedikit dibanding soldier, tower akan menambah mopper.
- Jika jumlah tower sudah cukup banyak dan ronde permainan sudah melewati fase awal, tower mulai mempertimbangkan splasher sebagai unit kontrol area.
- Jika tidak ada kondisi khusus di atas yang terpenuhi, tower akan kembali memilih soldier sebagai unit utama.

Dengan demikian, tower pada alternatif ini berperan sebagai pusat keputusan untuk bertahan ketika terancam, memperkuat ekonomi saat aman, dan memproduksi unit sesuai kondisi area sekitar.

Soldier

Soldier merupakan unit inti dalam strategi ini karena memegang tanggung jawab paling besar, yaitu eksplorasi, pembangunan tower, gangguan terhadap progres musuh di ruin, penyerangan unit lawan, serta kontrol tile secara bertahap. Urutan prioritas soldier adalah sebagai berikut.

- Jika soldier sudah mengunci suatu ruin, ia akan terus membangun sampai selesai selama paint mencukupi.
- Jika paint di bawah batas minimum, soldier kembali ke tower terdekat untuk refill.
- Soldier mengecat tile di bawahnya untuk membersihkan enemy paint atau mengisi pola tower.
- Pada ronde awal, soldier bergerak menuju target simetri untuk menemukan wilayah lawan lebih cepat.
- Jika mendeteksi ruin yang sedang dikerjakan musuh, soldier akan mendekat dan merusak tile di sekitarnya.
- Jika menemukan ruin yang belum diklaim, soldier akan mengklaim dan memulai pembangunan tower.
- Jika ada tower musuh, soldier menyerang atau mendekatinya.
- Jika tidak ada tower, soldier menyerang unit musuh terdekat.
- Jika tidak ada target lain, soldier menjelajah peta untuk mencari ruin atau musuh baru.

Secara keseluruhan, heuristik soldier di alternatif ini berfokus untuk menjaga resource terlebih dahulu, menyelesaikan tugas yang sedang aktif, lalu ambil peluang ofensif dan eksploratif yang paling bernilai di sekitar.

Mopper

Mopper pada alternatif ini berfungsi sebagai unit support, area-cleanup, dan anti-rush untuk menjaga kestabilan area tim, membersihkan enemy paint, dan membantu soldier tetap aktif. Urutan prioritas heuristik mopper adalah sebagai berikut.

- Jika paint rendah, mopper kembali ke tower untuk refill.

- Jika ada soldier sekutu dengan paint rendah, mopper mentransfer sebagian paint.
- Jika ada soldier atau mopper musuh dekat tower sekutu, mopper memprioritaskan menyerang target tersebut.
- Jika ada ruin penting yang terkena enemy paint, mopper membersihkannya.
- Jika ada musuh di sekitar, mopper menyerang unit dengan paint paling sedikit.
- Jika tidak ada target robot, mopper membersihkan enemy paint di sekitar.
- Jika tidak ada target, mopper ikut menjelajah peta.

Heuristik kunci mopper pada strategi ini adalah bahwa mopper menjadi unit yang memastikan soldier tetap punya paint, mempertahankan ruin penting, dan membersihkan pengaruh musuh.

Splasher

Splasher berfungsi sebagai unit pengendali area yang dirancang untuk memaksimalkan dampak setiap serangan. Splasher selalu mencoba mencari pusat serangan dengan nilai area tertinggi. Urutan heuristik splasher adalah sebagai berikut.

- Jika paint rendah, splasher kembali ke tower.
- Menyerang area 3×3 dengan kombinasi enemy paint dan mark terbanyak untuk merusak progress lawan.
- Jika tidak ada pattern break, splasher menyerang area dengan jumlah enemy paint terbanyak.
- Jika berada di tile musuh, splasher bergerak menjauh untuk posisi yang lebih aman.
- Jika tidak ada target yang bernilai tinggi, splasher menjelajah area baru.

Dengan demikian, splasher diarahkan untuk menghasilkan kerusakan area maksimum dan mengganggu pola kontrol wilayah lawan secara efisien.

3.3 Analisis Efisiensi dan Efektivitas Strategi

3.3.1 Main

Tabel 6: Kelebihan dan Kekurangan Strategi Bot `testA`

	Aspek	**Keterangan**
+	Spawn terstruktur	Pola spawn 4 soldier : 1 mopper yang konsisten memastikan ada mopper secara periodik tanpa bergantung pada intel eksternal
+	Respons cepat terhadap tile musuh	Tower langsung men-*spawn* mopper begitu mendeteksi tile musuh di sekitarnya tanpa menunggu laporan dari soldier
+	Cat tile marked diprioritaskan	Soldier memprioritaskan tile yang sudah di-*mark* untuk dicat sebelum tile kosong biasa, sehingga pola tower bisa diselesaikan lebih cepat
+	Deteksi dinding aktif	Fungsi `blockedByWall` mendeteksi dinding yang menghalangi jalur secara lokal, mencegah soldier bergerak ke arah yang terblokir
+	Upgrade tower otomatis	Tower meng-*upgrade* dirinya sendiri setiap 500 giliran tanpa perlu instruksi dari unit lain
+	Throttle spawn antar tower	Ketika jumlah tower > 6, setiap tower hanya *spawn* pada giliran tertentu berdasarkan ID-nya, menghindari pemborosan *resource* akibat semua tower *spawn* serentak
−	Tidak ada mekanisme refuel	Soldier, mopper, dan splasher tidak memeriksa level paint sama sekali. Unit bisa mati kehabisan paint di tengah peta tanpa pernah kembali ke tower
−	Eksplorasi terbatas 4 titik	`endPoint` hanya berisi 4 titik yang dihitung saat *spawn*, sehingga area eksplorasi sangat terbatas dan tidak adaptif terhadap ukuran peta
−	Soldier berhenti bergerak saat lapor	Ketika `botState = 1`, soldier hanya bergerak menuju tower terdekat untuk mengirim pesan tanpa melakukan aksi lain, memperlambat ekspansi wilayah
−	Splasher hampir tidak berkontribusi	Splasher hanya menyerang posisi sendiri jika tile musuh > 6 di sekitarnya, kondisi yang jarang terpenuhi. Sebagian besar waktu splasher hanya bergerak tanpa aksi
−	Mopper tanpa refuel	Mopper tidak memiliki threshold paint untuk kembali ke tower, sehingga rentan mati sebelum membersihkan tile musuh yang menjadi targetnya

“‘

3.3.2 Bot Alternatif 1:

	Aspek	**Keterangan**
+	Ofensif terkoordinasi	Soldier menyerang tower musuh secara berkelompok (minimal 3 unit) sehingga serangan lebih efektif dan tidak gegabah
+	Sustain tinggi	Mekanisme refuel konservatif (threshold 50) dan retreat darurat (threshold 15) menjaga soldier tetap hidup lebih lama
+	Mopper tepat sasaran	Mopper di-dispatch ke koordinat spesifik dari intel, bukan patroli acak
+	Spawn strategis	Mopper dan splasher hanya di-spawn ketika ada kebutuhan konkret yang dibuktikan melalui pesan intel, menghindari pemborosan resource
+	Navigasi obstacle	Rotasi arah sistematis (7 alternatif) memastikan soldier tidak terjebak di koridor sempit
+	Tower tetap produktif	Tower tidak berhenti setelah membaca pesan, selalu melanjutkan ke keputusan spawn
−	Pipeline rapuh	Jika soldier mati sebelum melapor atau berada di luar jangkauan tower, mopper dan splasher tidak pernah di-spawn
−	Eksplorasi lambat di peta besar	8 titik kandidat eksplorasi yang tetap menciptakan blind spot di sudut-sudut peta besar
−	Tidak ada antisipasi	Bot hanya bereaksi setelah tile musuh terlihat, tidak ada mekanisme untuk mengantisipasi pergerakan musuh
−	Splasher pasif di early game	Splasher tidak berkontribusi sebelum intel tersedia meskipun resource mencukupi
−	Serangan tower butuh massa	Membutuhkan minimal 3 soldier sebelum menyerang tower musuh, sehingga respons ofensif lebih lambat di early game
−	Tidak memanfaatkan simetri peta	Bot tidak menyimpan atau menggunakan informasi simetri peta untuk menentukan target eksplorasi. Kandidat eksplorasi selalu dihitung ulang dari scratch setiap giliran berdasarkan posisi saat ini, sehingga tidak ada pengetahuan kumulatif tentang area yang sudah dikunjungi atau area yang belum dijelajahi

Tabel 7: Kelebihan dan Kekurangan Strategi Bot Altenatif 1

3.3.3 Bot Alternatif 2:

+	Distribusi eksplorasi yang merata	Sistem symmetry target pada early game dan waypoint exploration membuat robot menyebar ke berbagai area peta sehingga eksplorasi lebih merata dan tidak menumpuk pada satu area.
+	Pembangunan tower yang efisien	Sistem claim ruin mencegah beberapa soldier membangun ruin yang sama sehingga pembangunan tower menjadi lebih terdistribusi dan menghindari konflik antar unit.
+	Adaptasi spawn unit	Tower memilih jenis unit berdasarkan kondisi sekitar seperti keberadaan musuh, jumlah enemy paint, serta komposisi pasukan sekutu sehingga produksi unit lebih kontekstual.
+	Kontrol wilayah yang stabil	Mopper secara aktif membersihkan enemy paint di sekitar ruin dan wilayah penting sehingga membantu menjaga kontrol area yang telah dikuasai.
+	Serangan area efektif	Splasher memilih posisi serangan dengan jumlah tile musuh terbanyak sehingga setiap serangan menghasilkan dampak area yang maksimal.
+	Navigasi adaptif	Algoritma pergerakan mengevaluasi beberapa arah sekaligus dan mempertimbangkan jarak target, tile ally paint, serta tile enemy paint sehingga pergerakan relatif efisien dan tidak terlalu sering terjebak.
+	Manajemen resource	Penggunaan threshold paint untuk refill dan build commitment menjaga agar unit tidak kehabisan resource saat melakukan aksi penting seperti pembangunan tower atau pertempuran.
−	Koordinasi tim terbatas	Sebagian besar keputusan diambil secara lokal oleh masing-masing unit sehingga koordinasi antar robot masih terbatas tanpa sistem komunikasi yang kuat.
−	Strategi global sederhana	Pemilihan target bersifat heuristik lokal sehingga robot tidak selalu memilih target yang paling optimal secara global.
−	Pathfinding relatif sederhana	Sistem navigasi berbasis evaluasi arah lokal belum menggunakan algoritma pathfinding kompleks sehingga pada peta dengan obstacle kompleks pergerakan bisa kurang optimal.

–	Ketergantungan pada vision	Banyak keputusan seperti pembangunan ruin atau pembersihan tile bergantung pada informasi yang terlihat langsung sehingga beberapa peluang strategis di luar vision dapat terlewat.
–	Efektivitas splash situasional	Serangan splasher sangat efektif ketika enemy paint terkonsentrasi, tetapi kurang optimal jika tile musuh tersebar secara sporadis.

Tabel 8: Kelebihan dan Kekurangan Strategi Bot Alternatif 2

3.4 Strategi Greedy yang Dipilih

Strategi yang digunakan pada implementasi bot ini adalah strategi *greedy*. Pemilihan strategi ini didasarkan pada hasil pengujian beberapa varian bot yang telah dibuat sebelumnya. Dari berbagai percobaan yang dilakukan, implementasi yang terdapat pada file `main_bot` menunjukkan *win rate* yang paling tinggi dibandingkan dengan pendekatan lainnya.

Pendekatan *greedy* bekerja dengan cara memilih aksi yang dianggap paling menguntungkan pada kondisi saat ini tanpa mempertimbangkan konsekuensi jangka panjang secara mendalam. Dalam konteks permainan Battlecode, strategi ini memungkinkan robot untuk mengambil keputusan secara cepat dan efisien berdasarkan informasi lokal yang tersedia, seperti posisi musuh, lokasi menara, serta area yang perlu dieksplorasi atau dicat.

Berdasarkan hasil evaluasi dari beberapa simulasi pertandingan, strategi pada `main_bot` mampu memberikan performa yang lebih konsisten dibandingkan strategi lain yang diuji. Oleh karena itu, strategi tersebut dipilih sebagai pendekatan utama dalam implementasi bot pada tugas ini.

4 Implementasi dan Pengujian

4.1 Implementasi Algoritma Greedy

4.1.1 Pseudocode Strategi Main

Kamus Global — testA

turnCount	:	integer
rng	:	Random (seed = 6147)
directions	:	array[8] of Direction {N, NE, E, SE, S, SW, W, NW}
botSpawnDir	:	integer
spawnedBot	:	integer
towerRoundAge	:	integer
messageStreak	:	integer
enemyLoc	:	MapLocation
endPoint	:	array[4] of MapLocation
corner	:	integer
enemyTileLoc	:	MapLocation
nearestTower	:	MapLocation
botState	:	integer
buildWhat	:	UnitType
buildBrute	:	array[3] of UnitType {MONEY_TOWER, PAINT_TOWER, DEFENSE_TOWER}

Procedure run(rc : RobotController)

{I.S : Robot baru di-spawn oleh tower}
{F.S : Robot menjalankan aksi sesuai tipenya setiap ronde hingga mati}

type	:	UnitType

while true **do** $turnCount \leftarrow turnCount + 1$
$type \leftarrow rc.getType()$
if $type$ = SOLDIER **then** $runSoldier(rc)$
end if
else if $type$ = MOPPER **then** $runMopper(rc)$
end if
else if $type$ = SPLASHER **then** $runSplasher(rc)$
end if
else $runTower(rc)$
end if
$Clock.yield()$
end while

Procedure runTower(rc : RobotController)

{I.S : Tower aktif pada giliran baru}
{F.S : Tower membaca pesan, mendeteksi tile musuh, men-spawn unit sesuai pola, menyerang musuh terdekat, dan upgrade diri setiap 500 ronde}

messages	:	array of Message
nearbyTiles	:	array of MapInfo
nearbyBots	:	array of RobotInfo
nearestEnemyBot	:	MapLocation
enemyCount	:	integer
mustSpawnMopper	:	boolean
robotType	:	integer
towerCount	:	integer
spawnBot	:	boolean
dir	:	Direction
nextLoc	:	MapLocation
blocked	:	array[9][9] of boolean

$towerRoundAge \leftarrow towerRoundAge + 1$
$mustSpawnMopper \leftarrow false$
$messages \leftarrow rc.readMessages(rc.getRoundNum() - 1)$
for each $m \in messages$ **do if** $enemyLoc \neq null$
then$messageStreak \leftarrow messageStreak + 1$
end if
if $messageStreak = 3$ **then**$mustSpawnMopper \leftarrow true$;
$messageStreak \leftarrow 0$
end if
$enemyLoc \leftarrow MapLocation(m.getBytes() >>$
$8,\ m.getBytes()\ \&\ 0xFF)$
end for
$nearbyTiles \leftarrow rc.senseNearbyMapInfos()$
$blocked \leftarrow blockedByWall(rc, nearbyTiles, rc.getLocation())$
for each $tile \in nearbyTiles$ **do if** $tile.paint.isEnemy()$ **and** tile tidak terblokir dinding **then** $enemyLoc \leftarrow tile.location$;
$mustSpawnMopper \leftarrow true$; **break**
end if
end for
$nearbyBots \leftarrow rc.senseNearbyRobots()$
$nearestEnemyBot \leftarrow null$; $enemyCount \leftarrow 0$
for each $bot \in nearbyBots$ where $bot.team \neq rc.getTeam()$ **do**
$enemyCount \leftarrow enemyCount + 1$
if $nearestEnemyBot = null$ **or** bot lebih dekat
then$nearestEnemyBot \leftarrow bot.location$
end if
end for
$dir \leftarrow directions[botSpawnDir\ \%\ 8]$; $botSpawnDir \leftarrow botSpawnDir + 1$
$nextLoc \leftarrow rc.getLocation().add(dir)$
$towerCount \leftarrow rc.getNumberTowers()$; $spawnBot \leftarrow true$
if $towerCount > 6$ **then if**
$rc.getRoundNum()\ \%\ towerCount \neq rc.getID()\ \%\ towerCount$ **then**
$spawnBot \leftarrow false$

Procedure runSoldier(rc : RobotController)

{I.S : Soldier aktif pada giliran baru}
{F.S : Soldier mewarnai tile kosong atau marked di sekitarnya, bergerak menuju tile kosong terdekat atau ruin, dan melaporkan tile musuh ke tower}

currentLoc	:	MapLocation
surroundTile	:	array of MapInfo
curRuin	:	MapInfo
destination	:	MapLocation
minDistance	:	integer
hasAttacked	:	boolean
blocked	:	array[9][9] of boolean
message	:	integer
sendSuccess	:	boolean

$currentLoc \leftarrow rc.getLocation()$
$surroundTile \leftarrow rc.senseNearbyMapInfos()$
$curRuin \leftarrow null$
if $endPoint[0] = null$ **then**$initEndPoint(rc, currentLoc)$
end if
$hasAttacked \leftarrow false$
if $rc.isActionReady()$ **then for** each $tile \in surroundTile$ **do if**
$tile.getMark() \neq tile.getPaint()$ **and** $tile.getMark() \neq$ EMPTY **and**
$rc.canAttack(tile.location)$ **then**
$rc.attack(tile.location,\ tile.getMark() =$ ALLY_SECONDARY);
$hasAttacked \leftarrow true$; **break**
end if
if $tile.paint =$ EMPTY **and** $rc.canAttack(tile.location)$ **and**
$tile.location.distSq(currentLoc) \leq 2$ **then** $rc.attack(tile.location)$;
$hasAttacked \leftarrow true$; **break**
end if
end for
end if
if $rc.isMovementReady()$ **then** $destination \leftarrow null$;
$minDistance \leftarrow \infty$
$blocked \leftarrow blockedByWall(rc, surroundTile, currentLoc)$
for each $tile \in surroundTile$ **do if** ada robot musuh di $tile.location$
and $rc.isActionReady()$ **and** $\neg hasAttacked$ **then**
$rc.attack(tile.location)$
end if
if ada robot sekutu tower di $tile.location$
then$nearestTower \leftarrow tile.location$; **continue**
end if
if $tile.hasRuin()$ **then**$curRuin \leftarrow tile$; **continue**
end if
if $tile.isWall()$ **thencontinue**
end if
if $tile.paint.isEnemy()$ **and** tile tidak terblokir
then$enemyTileLoc \leftarrow tile.location$; $botState \leftarrow 1$
end if
if $tile.paint =$ EMPTY **and** $tile.isPassable()$ **and** tile tidak terblokir

Procedure runMopper(rc : RobotController)

{I.S : Mopper aktif pada giliran baru}
{F.S : Mopper membersihkan tile musuh terdekat, mop swing ke unit musuh, dan mentransfer paint ke sekutu dengan paint terendah}

currentLoc	:	MapLocation
messages	:	array of Message
nearbyBots	:	array of RobotInfo
nearbyTiles	:	array of MapInfo
hasAttacked	:	boolean
enemyTileNearby	:	boolean
lowestPaint	:	integer
lowestPaintBot	:	MapLocation
blocked	:	array[9][9] of boolean

$currentLoc \leftarrow rc.getLocation()$
if $endPoint[0] = null$ **then**$initEndPoint(rc, currentLoc)$
end if
$messages \leftarrow rc.readMessages()$
for each $m \in messages$ **do** $enemyTileLoc \leftarrow$
$MapLocation(m.getBytes() >> 8,\ m.getBytes()\ \&\ 0xFF)$
end for
if $enemyTileLoc \neq null$ **and** $currentLoc.distSq(enemyTileLoc) \leq 20$
then $enemyTileLoc \leftarrow null$
end if
$nearbyBots \leftarrow rc.senseNearbyRobots()$
$nearbyTiles \leftarrow rc.senseNearbyMapInfos()$
$hasAttacked \leftarrow false$; $enemyTileNearby \leftarrow false$
$blocked \leftarrow blockedByWall(rc, nearbyTiles, currentLoc)$
for each $tile \in nearbyTiles$ **do if** $tile.paint.isEnemy()$ **and**
$rc.isActionReady()$ **and** $tile.location.distSq(currentLoc) \leq 2$ **and**
$\neg hasAttacked$ **then if** $rc.canAttack(tile.location)$ **then**
$rc.attack(tile.location)$; $hasAttacked \leftarrow true$
else$enemyTileNearby \leftarrow true$
end if
end if
if $enemyTileLoc = null$ **and** $tile.paint.isEnemy()$ **and**
$tile.location.distSq(currentLoc) > 2$ **and** tile tidak terblokir **then**
$enemyTileLoc \leftarrow tile.location$
end if
end for
if $rc.isMovementReady()$ **and** $\neg enemyTileNearby$ **then**
$robotMove(rc, currentLoc, enemyTileLoc)$
end if
$lowestPaint \leftarrow \infty$; $lowestPaintBot \leftarrow null$
for each $bot \in nearbyBots$ **do if** $bot.team = rc.getTeam()$ **and**
$bot.location.distSq(currentLoc) \leq 2$ **then if**
$bot.paintAmount < lowestPaint$ **then**
$lowestPaint \leftarrow bot.paintAmount$; $lowestPaintBot \leftarrow bot.location$
end if
end if
else if $bot.team \neq rc.getTeam()$ **and**

Procedure runSplasher(rc : RobotController)

{I.S : Splasher aktif pada giliran baru}
{F.S : Splasher menyerang di posisi sendiri jika banyak tile musuh di sekitar, kemudian bergerak menuju tile musuh terdekat}

currentLoc	:	MapLocation
nearbyTiles	:	array of MapInfo
enemyTileCount	:	integer
blocked	:	array[9][9] of boolean

$currentLoc \leftarrow rc.getLocation()$
if $endPoint[0] = null$ **then**$initEndPoint(rc, currentLoc)$
end if
$nearbyTiles \leftarrow rc.senseNearbyMapInfos()$
$blocked \leftarrow blockedByWall(rc, nearbyTiles, currentLoc)$
$enemyTileCount \leftarrow 0$
for each $tile \in nearbyTiles$ **do if** $tile.paint.isEnemy()$ **and** $rc.isActionReady()$ **and** $tile.location.distSq(currentLoc) \leq 2$ **then**
$enemyTileCount \leftarrow enemyTileCount + 1$
end if
if $enemyTileLoc = null$ **and** $tile.paint.isEnemy()$ **and** $tile.location.distSq(currentLoc) > 2$ **and** tile tidak terblokir **then**
$enemyTileLoc \leftarrow tile.location$
end if
if ada robot sekutu tower di $tile.location$
then$nearestTower \leftarrow tile.location$
end if
end for
if $enemyTileCount > 6$ **and** $rc.canAttack(currentLoc)$ **then**
$rc.attack(currentLoc)$
end if
if $rc.isMovementReady()$
then$robotMove(rc, currentLoc, enemyTileLoc)$
end if

Procedure initEndPoint(rc : RobotController, currentLoc : MapLocation)

{I.S : endPoint belum diinisialisasi}
{F.S : endPoint[0..3] diisi berdasarkan dimensi peta dan posisi robot saat spawn}

mapW, mapH : integer

$corner \leftarrow rc.getID()\ \%\ 2$
$mapW \leftarrow rc.getMapWidth()$; $mapH \leftarrow rc.getMapHeight()$
if $mapH > 2 \times mapW$ **then**
$endPoint[0] \leftarrow (currentLoc.x,\ mapH - 1 - currentLoc.y)$
$endPoint[1] \leftarrow (currentLoc.x,\ mapH - 1 - currentLoc.y)$
end if
else if $mapW > 2 \times mapH$ **then**
$endPoint[0] \leftarrow (mapW - 1 - currentLoc.x,\ currentLoc.y)$
$endPoint[1] \leftarrow (mapW - 1 - currentLoc.x,\ currentLoc.y)$
end if
else $endPoint[0] \leftarrow (mapW - 1 - currentLoc.x,\ currentLoc.y)$
$endPoint[1] \leftarrow (currentLoc.x,\ mapH - 1 - currentLoc.y)$
end if
$endPoint[2] \leftarrow (mapW/2,\ mapH/2)$
$endPoint[3] \leftarrow (mapW - 1 - currentLoc.x,\ mapH - 1 - currentLoc.y)$

Procedure markAndBuildTower(rc : RobotController, currentLoc : MapLocation, targetLoc : MapLocation)

{I.S : Soldier berada dalam jarak ≤ 4 dari ruin}
{F.S : Pola tower ditandai dan diselesaikan jika semua tile sudah dicat}

dir : Direction
shouldBeMarked : MapLocation
towerType : UnitType

$dir \leftarrow currentLoc.directionTo(targetLoc)$
$shouldBeMarked \leftarrow targetLoc.subtract(dir)$
if $buildWhat = null$ **then**$buildWhat \leftarrow nextTowerType(rc, targetLoc)$
end if
if $rc.senseMapInfo(shouldBeMarked).getMark() =$ EMPTY **and**
$rc.canMarkTowerPattern(buildWhat, targetLoc)$ **then**
$rc.markTowerPattern(buildWhat, targetLoc)$
end if
for each $towerType \in buildBrute$ **do if**
$rc.canCompleteTowerPattern(towerType, targetLoc)$ **then**
$rc.completeTowerPattern(towerType, targetLoc)$
end if
end for

Function nextTowerType(rc : RobotController, ruinLocation : MapLocation) → UnitType

{Memilih tipe tower berdasarkan jarak ruin ke tengah peta dan jumlah tower yang sudah dibangun}

```
midPoint          :  MapLocation
relativeDistance  :  integer
```

```
midPoint ← endPoint[2]
if rc.getMapHeight() > rc.getMapWidth() then
    relativeDistance ← rc.getMapHeight()/2
else relativeDistance ← rc.getMapWidth()/2
end if
if ruinLocation.distSq(midPoint) < relativeDistance and
 rc.getNumberTowers() > 5 then return
 LEVEL_ONE_DEFENSE_TOWER
end if
if rc.getNumberTowers() % 2 = 0 then
    return LEVEL_ONE_MONEY_TOWER
else return LEVEL_ONE_PAINT_TOWER
end if
```

Procedure robotMove(rc : RobotController, currentLoc : MapLocation, destination : MapLocation)

{I.S : Robot siap bergerak}
{F.S : Robot bergerak satu langkah menuju destination, atau menuju endPoint jika destination null}

```
goTo  :  Direction
```

```
if destination ≠ null then
    goTo ← currentLoc.directionTo(destination)
else if currentLoc.distSq(endPoint[corner]) ≤ 20 then if corner = 2
 then
    corner ← 3
elsecorner ← 2
end if
end if
goTo ← currentLoc.directionTo(endPoint[corner])
end if
if goTo ≠ null and goTo ≠ CENTER then for i ← 0 to 7 while
 ¬rc.canMove(goTo) do goTo ← goTo.rotateRight()
end for
if rc.canMove(goTo) thenrc.move(goTo)
end if
end if
```

Function sendMessageToTower(rc : RobotController, message : integer) → boolean

{Mengirim pesan ke semua tower sekutu dalam jangkauan.
Mengembalikan true jika minimal satu berhasil.}

nearbyBots	:	array of RobotInfo
sent	:	boolean

$nearbyBots \leftarrow rc.senseNearbyRobots(-1, rc.getTeam())$
$sent \leftarrow false$
for each $bot \in nearbyBots$ where $bot.type.isTowerType()$ **do if**
$rc.canSendMessage(bot.location, message)$ **then**
$rc.sendMessage(bot.location, message)$; $sent \leftarrow true$
end if
end for
return $sent$

4.1.2 Pseudocode Strategi Alternatif 1

Kamus Global		
turnsOnCurrentTarget	:	integer
exploreTarget	:	MapLocation
currentTarget	:	MapLocation
SpawnTowerLoc	:	MapLocation
exploreStep	:	integer
lastReportedRound	:	integer
lastTowerReportRound	:	integer
reportCooldown	:	integer = 50
towerReportCooldown	:	integer = 30
msgEnemyTile	:	integer = 1
msgTellMopper	:	integer = 2
msgTellWeakTow	:	integer = 3
msgTellSplasher	:	integer = 4
percantageWeakTower	:	integer = 40
lastSeenEnemyTile	:	MapLocation
mopperTarget	:	MapLocation
splasherTowerTarget	:	MapLocation
towerDestroyed	:	boolean
patternCleanRound	:	integer
enemyTileReportCount	:	integer
weakTowerReportCount	:	integer
lastSplasherSpawnRound	:	integer
lastDispatchedLocation	:	MapLocation
splasherCooldown	:	integer = 50
chosenTowerType	:	UnitType
chosenRuinLoc	:	MapLocation
directions	:	array[8] of Direction {N, NE, E, SE, S, SW, W, NW}
fourDirections	:	array[4] of Direction {N, E, S, W}

Procedure run(rc : RobotController)

{I.S : Robot baru di-spawn oleh tower}
{F.S : Robot menjalankan aksi sesuai tipenya setiap ronde hingga mati}

```
type                    :  UnitType
while true do type ← rc.getType()
if type = SOLDIER then runSoldier(rc)
end if
else if type = MOPPER then runMopper(rc)
end if
else if type = SPLASHER then runSplasher(rc)
end if
else runTower(rc)
end if
Clock.yield()
end while
```

Procedure runTower(rc : RobotController)

{I.S : Tower aktif pada giliran baru}
{F.S : Tower menyerang musuh, upgrade tower sekutu, meneruskan pesan, dan men-spawn unit yang paling dibutuhkan}

```
enemies, allies         :  array of RobotInfo
toSpawn                 :  UnitType
spawnLoc                :  MapLocation
dir                     :  Direction
enemies ← rc.senseNearbyRobots(lawan)
for each enemy ∈ enemies do if rc.canAttack(enemy.location) then
  rc.attack(enemy.location); break
end if
end for
allies ← rc.senseNearbyRobots(kawan)
for each ally ∈ allies do if isAllyTower(ally.type) and
  rc.canUpgradeTower(ally.location) then
  rc.upgradeTower(ally.location); break
end if
end for
collectAndForward(rc)
toSpawn ← decideWhatToSpawn(rc)
if toSpawn ≠ null then for each dir ∈ directions do
  spawnLoc ← rc.getLocation().add(dir)
if rc.canBuildRobot(toSpawn, spawnLoc) then
  rc.buildRobot(toSpawn, spawnLoc); break
end if
end for
end if
```

Procedure runSoldier(rc : RobotController) — Part 1

{I.S : Soldier aktif pada giliran baru}
{F.S : Soldier menjalankan aksi berdasarkan prioritas: darurat paint, refuel, raid tower, bangun tower, retreat, serang tile, eksplorasi}

myLoc, threatLocation, ruin	:	MapLocation
visible	:	array of MapInfo
enemies, allies	:	array of RobotInfo
totalSoldiers, myHP, dist	:	integer
canRisk, threatened, refueled	:	boolean
toward, away	:	Direction
bestScore, score	:	real

$myLoc \leftarrow rc.getLocation()$
$visible \leftarrow rc.senseNearbyMapInfos()$
$enemies \leftarrow rc.senseNearbyRobots(lawan)$
$allies \leftarrow rc.senseNearbyRobots(kawan)$
$updtEnemyTile(rc, visible)$; $reportWeakTower(rc, enemies, allies)$
if $SpawnTowerLoc = null$ **then** $SpawnTowerLoc \leftarrow$ tower sekutu terdekat dari $allies$
end if
$totalSoldiers \leftarrow$ hitung SOLDIER di $allies$
if $rc.getPaint() < 15$ **then if** $SpawnTowerLoc \neq null$ **then**$moveToward(rc, SpawnTowerLoc)$
end if
return
end if
if $rc.getPaint() < 50$ **then** $tryReportTower(rc, allies)$
$refueled \leftarrow tryRefuel(rc)$
if $\neg refueled$ **then if** $SpawnTowerLoc \neq null$ **and** $myLoc.distSq(SpawnTowerLoc) > 4$ **then**
  $moveToward(rc, SpawnTowerLoc)$
else $SpawnTowerLoc \leftarrow null$; $exploreOutward(rc)$
end if
end if
return
end if

Procedure runSoldier(rc : RobotController) — Part 2

```
for each enemy ∈ enemies do if enemy.type.isTowerType() and
 totalSoldiers ≥ 3 then if rc.canAttack(enemy.location) then
    rc.attack(enemy.location)
else moveToward(rc, enemy.location)
end if
return
end if
end for
if rc.getRoundNum() > 50 then
 ruin ← findRuinForBuilder(rc, visible)
if ruin ≠ null then handleRuinBuilding(rc, ruin); return
end if
end if
myHP ← rc.getHealth(); canRisk ← myHP > 100
if ¬canRisk then for each enemy ∈ enemies do if
 enemy.type = MOPPER or enemy.type.isTowerType() then
 threatened ← true; threatLocation ← enemy.location; break
end if
end for
end if
if threatened then toward ← myLoc.directionTo(threatLocation);
 away ← toward.opposite()
if rc.canMove(away) then
    rc.move(away)
else if SpawnTowerLoc ≠ null
 thenmoveToward(rc, SpawnTowerLoc)
end if
end if
cat tile bawah kaki jika belum berwarna sekutu
return
end if
```

Procedure runSoldier(rc : RobotController) — Part 3

```
myHP ← rc.getHealth(); canRisk ← myHP > 100
if ¬canRisk then for each enemy ∈ enemies do if
  enemy.type = MOPPER or enemy.type.isTowerType() then
  threatened ← true; threatLocation ← enemy.location; break
end if
end for
end if
if threatened then toward ← myLoc.directionTo(threatLocation);
  away ← toward.opposite()
if rc.canMove(away) then
    rc.move(away)
else if SpawnTowerLoc ≠ null
  thenmoveToward(rc, SpawnTowerLoc)
end if
end if
cat tile bawah kaki jika belum berwarna sekutu
return
end if
```

Procedure runSoldier(rc : RobotController) — Part 4

```
for each tile ∈ visible where tile.paint.isEnemy() and
  myLoc.distSq(tile) ≤ 16 do currentTarget ← tile.location;
  turnsOnCurrentTarget ← 0
end for
if currentTarget ≠ null then
  turnsOnCurrentTarget ← turnsOnCurrentTarget + 1
if sudah tiba or tile sudah dicat sekutu or
  turnsOnCurrentTarget > 20 then currentTarget ← null;
  turnsOnCurrentTarget ← 0
end if
end if
if currentTarget = null then bestScore ← −1
for each tile ∈ visible where tile.paint = EMPTY do
  score ← 3.0/ max(1, myLoc.distSq(tile))
if score > bestScore then currentTarget ← tile.location;
  bestScore ← score; turnsOnCurrentTarget ← 0
end if
end for
end if
if currentTarget ≠ null then
    if rc.canAttack(currentTarget) then
        rc.attack(currentTarget); currentTarget ← null;
         turnsOnCurrentTarget ← 0
    else moveToward(rc, currentTarget)
    end if
else exploreOutward(rc)
end if
if tile bawah kaki belum dicat sekutu and rc.canAttack(myLoc) then
  rc.attack(myLoc)
end if
tryReportTower(rc, allies)
```

Procedure runMopper(rc : RobotController) — Part 1

{I.S : Mopper aktif pada giliran baru}
{F.S : Mopper membersihkan tile musuh, swing ke musuh, transfer paint ke soldier, atau bereksplorasi}

enemies, allies	:	array of RobotInfo
visible	:	array of MapInfo
messages	:	array of Message
decoded	:	array of integer
dispatchTarget, nearestEnemyTile	:	MapLocation
bestSwing	:	Direction
bestDist, d	:	integer
refueled	:	boolean

$enemies \leftarrow rc.senseNearbyRobots(lawan)$
$allies \leftarrow rc.senseNearbyRobots(kawan)$
$visible \leftarrow rc.senseNearbyMapInfos()$
$messages \leftarrow rc.readMessages()$
for each $m \in messages$ **do** $decoded \leftarrow decodeMsg(m)$
if $decoded[0] = msgTellMopper$ **then**
  $dispatchTarget \leftarrow MapLocation(decoded[1], decoded[2])$
if $mopperTarget = null$ **or** $dispatchTarget$ lebih dekat **then**
  $mopperTarget \leftarrow dispatchTarget$
end if
end if
end for
if $rc.getPaint() < 20$ **then** $refueled \leftarrow tryRefuel(rc)$
if $\neg refueled$ **then**$exploreOutward(rc)$
end if
return
end if
if $enemies.length > 0$ **then**
  $bestSwing \leftarrow getBestSwingDirection(rc, enemies)$
if $bestSwing \neq null$ **and** $rc.canMopSwing(bestSwing)$ **then**
  $rc.mopSwing(bestSwing)$
end if
end if

Procedure runMopper(rc : RobotController) — Part 2

```
if rc.getPaint() > 50 then for each ally ∈ allies do if
 ally.type = SOLDIER and ally.paintAmount < 30 then if
 rc.canTransferPaint(ally.location, 30) then
 rc.transferPaint(ally.location, 30); return
end if
end if
end for
end if
nearestEnemyTile ← null; bestDist ← ∞
for each tile ∈ visible where tile.paint.isEnemy() do
 d ← rc.getLocation().distSq(tile.location)
if d < bestDist thennearestEnemyTile ← tile.location; bestDist ← d
end if
end for
if nearestEnemyTile ≠ null then
    if rc.canAttack(nearestEnemyTile) then
        rc.attack(nearestEnemyTile)
    else moveToward(rc, nearestEnemyTile)
    end if
    mopperTarget ← null
else if mopperTarget ≠ null then
    moveToward(rc, mopperTarget)
    if rc.getLocation().distSq(mopperTarget) < 4
     thenmopperTarget ← null
    end if
else exploreOutward(rc)
end if
end if
```

Procedure runSplasher(rc : RobotController) — Part 1

{I.S : Splasher aktif pada giliran baru}
{F.S : Splasher menyerang tower target, membersihkan pattern bekas tower, atau menyerang tile musuh terdekat}

myLoc, nearestEnemy, bestClean, loc	:	MapLocation
enemies	:	array of RobotInfo
visible	:	array of MapInfo
messages	:	array of Message
decoded	:	array of integer
towerStillExists, refueled	:	boolean
bestCoverage, coverage, bestDist, d	:	integer

$myLoc \leftarrow rc.getLocation()$
$enemies \leftarrow rc.senseNearbyRobots(lawan)$
$visible \leftarrow rc.senseNearbyMapInfos()$
if $rc.getPaint() < 80$ **then** $refueled \leftarrow tryRefuel(rc)$
if $\neg refueled$ **then**$exploreOutward(rc)$
end if
return
end if
$messages \leftarrow rc.readMessages()$
for each $m \in messages$ **do** $decoded \leftarrow decodeMsg(m)$
if $decoded[0] = msgTellSplasher$ **and** dispatch lebih dekat **then**
$splasherTowerTarget \leftarrow MapLocation(decoded[1], decoded[2])$
$towerDestroyed \leftarrow false$
end if
end for
if $splasherTowerTarget \neq null$ **and** $\neg towerDestroyed$ **then**
$towerStillExists \leftarrow$ cek apakah tower masih ada di $enemies$
if $\neg towerStillExists$ **and** $rc.canSenseLocation(splasherTowerTarget)$
then $towerDestroyed \leftarrow true$;
$patternCleanRound \leftarrow rc.getRoundNum()$
end if
if $\neg towerDestroyed$ **then if** $rc.canAttack(splasherTowerTarget)$ **then**
$rc.attack(splasherTowerTarget)$
else $moveToward(rc, splasherTowerTarget)$
end if
return
end if
end if

Procedure runSplasher(rc : RobotController) — Part 2

```
if towerDestroyed and splasherTowerTarget ≠ null then
  bestClean ← null; bestCoverage ← 0
for each tile ∈ visible where tile.isPassable() do loc ← tile.location
if loc.distSq(splasherTowerTarget) > 16 or myLoc.distSq(loc) > 4
  thencontinue
end if
coverage ← hitung tile musuh (×3) + tile kosong (×1) di radius 1 dari
  loc
if coverage > bestCoverage thenbestClean ← loc;
  bestCoverage ← coverage
end if
end for
if bestClean ≠ null and bestCoverage > 0 then if
  rc.canAttack(bestClean) then
    rc.attack(bestClean)
elsemoveToward(rc, splasherTowerTarget)
end if
return
end if
if rc.getRoundNum() − patternCleanRound > 20 then
  splasherTowerTarget ← null; towerDestroyed ← false
end if
return
end if
nearestEnemy ← null; bestDist ← ∞
for each tile ∈ visible where tile.paint.isEnemy() do
  d ← myLoc.distSq(tile.location)
if d < bestDist thennearestEnemy ← tile.location; bestDist ← d
end if
end for
if nearestEnemy ≠ null then
    if rc.canAttack(nearestEnemy) then
        rc.attack(nearestEnemy)
    elsemoveToward(rc, nearestEnemy)
    end if
else exploreOutward(rc)
end if
```

Function decideWhatToSpawn(rc : RobotController) → UnitType — Part 1

{I.S : Robot belum bergerak pada giliran ini}
{F.S : Robot bergerak satu langkah mendekati target dengan detour otomatis}

round, paint, chips	:	integer
soldierCount, mopperCount, splasherCount	:	integer
canAffordSoldier, canAffordMopper, canAffordSplasher	:	boolean
splasherCooldownOk, hasEnemyIntel	:	boolean
noMoppersButHaveIntel, ratioTooSoldierHeavy	:	boolean
allies	:	array of RobotInfo

{Memutuskan tipe unit yang di-spawn berdasarkan kondisi sumber daya, komposisi armada, dan intel musuh}

$round \leftarrow rc.getRoundNum()$; $paint \leftarrow rc.getPaint()$; $chips \leftarrow rc.getChips()$
$canAffordSoldier \leftarrow paint \geq 200$ **and** $chips \geq 250$
$canAffordMopper \leftarrow paint \geq 100$ **and** $chips \geq 300$
$canAffordSplasher \leftarrow paint \geq 300$ **and** $chips \geq 400$
$allies \leftarrow rc.senseNearbyRobots(kawan)$
hitung $soldierCount$, $mopperCount$, $splasherCount$ dari $allies$
if $soldierCount = 0$ **then if** $canAffordSoldier$ **then**
    return SOLDIER
elsereturn *null*
end if
end if
if $round < 250$ **and** $soldierCount < 10$ **then if** $canAffordSoldier$ **then**
    return SOLDIER
elsereturn *null*
end if
end if
$splasherCooldownOk \leftarrow (round - lastSplasherSpawnRound) \geq splasherCooldown$
$hasEnemyIntel \leftarrow enemyTileReportCount > 0$ **or** $weakTowerReportCount > 0$
if $canAffordSplasher$ **and** $splasherCooldownOk$ **and** $splasherCount < 2$ **and** $hasEnemyIntel$ **then**
$lastSplasherSpawnRound \leftarrow round$; **return** SPLASHER
end if

Function decideWhatToSpawn(rc : RobotController) → UnitType — Part 2

```
if canAffordSplasher and splasherCooldownOk and
  splasherCount < 1 and paint ≥ 350 then
  lastSplasherSpawnRound ← round; return SPLASHER
end if
noMoppersButHaveIntel ← mopperCount = 0 and
  enemyTileReportCount > 0
ratioTooSoldierHeavy ← soldierCount ≥ 3 and
  soldierCount/ max(1, mopperCount) > 3.0
if canAffordMopper and (noMoppersButHaveIntel or
  ratioTooSoldierHeavy) then return MOPPER
end if
if canAffordSoldier thenreturn SOLDIER
end if
return null
```

Procedure moveToward(rc : RobotController, target : MapLocation)

```
  dir, left, right, left2, right2,   :   Direction
  left3, right3
dir ← rc.getLocation().directionTo(target)
if rc.canMove(dir) thenrc.move(dir); return
end if
left ← dir.rotateLeft(); right ← dir.rotateRight()
if rc.canMove(left) thenrc.move(left); return
end if
if rc.canMove(right) thenrc.move(right); return
end if
left2 ← left.rotateLeft(); right2 ← right.rotateRight()
if rc.canMove(left2) thenrc.move(left2); return
end if
if rc.canMove(right2) thenrc.move(right2); return
end if
left3 ← left2.rotateLeft(); right3 ← right2.rotateRight()
if rc.canMove(left3) thenrc.move(left3); return
end if
if rc.canMove(right3) thenrc.move(right3); return
end if
if rc.canMove(dir.opposite()) thenrc.move(dir.opposite())
end if
```

Function tryRefuel(rc : RobotController) → boolean

{I.S : Robot perlu bergerak ke area baru yang belum dijelajahi}
{F.S : Robot bergerak menuju titik kandidat eksplorasi secara deterministik}

myLoc, bestTower	:	MapLocation
allies	:	array of RobotInfo
bestScore, dist, score, paintNeeded	:	integer
t	:	UnitType
isPaintTower	:	boolean

{Mencari paint tower sekutu dengan skor paintAmount/jarak terbaik dan mengambil paint. Mengembalikan true jika berhasil.}

$myLoc \leftarrow rc.getLocation()$; $allies \leftarrow rc.senseNearbyRobots(kawan)$
$bestTower \leftarrow null$; $bestScore \leftarrow -1$
for each $ally \in allies$ **do** $t \leftarrow ally.getType()$
$isPaintTower \leftarrow t \in$ {LEVEL_ONE_PAINT_TOWER, LEVEL_TWO_PAINT_TOWER, LEVEL_THREE_PAINT_TOWER}
if $\neg isPaintTower$ **or** $ally.paintAmount \leq 50$ **thencontinue**
end if
$dist \leftarrow myLoc.distSq(ally.location)$
$score \leftarrow ally.paintAmount / \max(1, dist)$
if $score > bestScore$ **then**$bestScore \leftarrow score$; $bestTower \leftarrow ally.location$
end if
end for
if $bestTower = null$ **thenreturn** $false$
end if
$paintNeeded \leftarrow 100 - rc.getPaint()$
if $rc.canTransferPaint(bestTower, -paintNeeded)$ **then**
  $rc.transferPaint(bestTower, -paintNeeded)$
else $moveToward(rc, bestTower)$
end if
return $true$

Procedure exploreOutward(rc : RobotController)

myLoc, cand	:	MapLocation
mapW, mapH, cx, cy, oppositeX, oppositeY	:	integer
candidates	:	array[8] of MapLocation
farEnough	:	boolean

$myLoc \leftarrow rc.getLocation()$
if $exploreTarget = null$ **or** $myLoc.distSq(exploreTarget) < 8$ **then**
  $mapW \leftarrow rc.getMapWidth()$; $mapH \leftarrow rc.getMapHeight()$
$cx \leftarrow mapW/2$; $cy \leftarrow mapH/2$
$candidates \leftarrow$
  $\{(cx \pm mapW/4,\ cy \pm mapH/4),\ (cx,\ cy \pm mapH/3),\ (cx \pm mapW/3,\ cy)\}$
$exploreStep \leftarrow exploreStep\ \%\ 8$
for $i \leftarrow 0$ **to** 7 **do** $cand \leftarrow candidates[exploreStep\ \%\ 8]$;
  $exploreStep \leftarrow exploreStep + 1$
$farEnough \leftarrow SpawnTowerLoc = null$ **or**
  $cand.distSq(SpawnTowerLoc) > 100$
if $farEnough$ **and** $myLoc.distSq(cand) \geq 8$ **then**
  $exploreTarget \leftarrow cand$; **break**
end if
end for
if $exploreTarget = null$ **or** $myLoc.distSq(exploreTarget) < 8$ **then if**
  $SpawnTowerLoc \neq null$ **then**
    $oppositeX \leftarrow \min(mapW - 1,\ \max(0,\ mapW - SpawnTowerLoc.x))$
    $oppositeY \leftarrow \min(mapH - 1,\ \max(0,\ mapH - SpawnTowerLoc.y))$
    $exploreTarget \leftarrow MapLocation(oppositeX, oppositeY)$
else $exploreTarget \leftarrow MapLocation(cx, cy)$
end if
end if
end if
$moveToward(rc, exploreTarget)$

Procedure collectAndForward(rc : RobotController)

{I.S : Tower menerima pesan dari robot sekutu}
{F.S : Tower meneruskan dispatch ke mopper (tile musuh) atau splasher (tower lemah). Encoding: 4 bit tipe | 6 bit x | 6 bit y}

messages	:	array of Message
allies	:	array of RobotInfo
decoded	:	array of integer
type, x, y, dispatch, sent	:	integer
reported	:	MapLocation

$messages \leftarrow rc.readMessages()$;
$allies \leftarrow rc.senseNearbyRobots(kawan)$
for each $msg \in messages$ **do** $decoded \leftarrow decodeMsg(msg)$
$type \leftarrow decoded[0]$; $x \leftarrow decoded[1]$; $y \leftarrow decoded[2]$
if $type = msgEnemyTile$ **then** $reported \leftarrow MapLocation(x, y)$
if $lastDispatchedLocation \neq null$ **and**
$reported = lastDispatchedLocation$ **thencontinue**
end if
$enemyTileReportCount \leftarrow enemyTileReportCount + 1$
$dispatch \leftarrow encodeMsg(msgTellMopper, x, y)$; $sent \leftarrow 0$
for each $ally \in allies$ where $ally.type =$ MOPPER **and** $sent < 3$ **do if**
$rc.canSendMessage(ally.location, dispatch)$ **then**
$rc.sendMessage(ally.location, dispatch)$; $sent \leftarrow sent + 1$
end if
end for
$lastDispatchedLocation \leftarrow reported$
end if
if $type = msgTellWeakTow$ **then**
$weakTowerReportCount \leftarrow weakTowerReportCount + 1$
$dispatch \leftarrow encodeMsg(msgTellSplasher, x, y)$; $sent \leftarrow 0$
for each $ally \in allies$ where $ally.type =$ SPLASHER **and** $sent < 2$ **do**
if $rc.canSendMessage(ally.location, dispatch)$ **then**
$rc.sendMessage(ally.location, dispatch)$; $sent \leftarrow sent + 1$
end if
end for
end if
end for

Procedure updtEnemyTile(rc : RobotController, visible : array of MapInfo)

{I.S : Robot baru selesai mengindera lingkungan}
{F.S : lastSeenEnemyTile diperbarui dengan tile musuh paling dekat atau paling relevan}

myLoc, nearest	:	MapLocation
bestDist, d	:	integer
newIsCloser, oldTileNowClean	:	boolean

$myLoc \leftarrow rc.getLocation()$; $nearest \leftarrow null$; $bestDist \leftarrow \infty$
for each $tile \in visible$ where $tile.paint.isEnemy()$ **do**
  $d \leftarrow myLoc.distSq(tile.location)$
if $d < bestDist$ **then**$nearest \leftarrow tile.location$; $bestDist \leftarrow d$
end if
end for
if $nearest = null$ **then**return
end if
if $lastSeenEnemyTile = null$ **then**$lastSeenEnemyTile \leftarrow nearest$;
  return
end if
$newIsCloser \leftarrow bestDist < myLoc.distSq(lastSeenEnemyTile)$
$oldTileNowClean \leftarrow rc.canSenseLocation(lastSeenEnemyTile)$ **and**
  $\neg lastSeenEnemyTile.paint.isEnemy()$
if $newIsCloser$ **or** $oldTileNowClean$ **then**
  $lastSeenEnemyTile \leftarrow nearest$
end if

Procedure tryReportTower(rc : RobotController, allies : array of RobotInfo)

{I.S : Soldier mengetahui lokasi tile musuh terakhir}
{F.S : Pesan lokasi tile musuh dikirim ke tower sekutu terdekat jika paint rendah atau tower dalam jangkauan}

myLoc, towerLoc	:	MapLocation
isLowPaint, isTowerInRange	:	boolean
bestDist, d, msg	:	integer

if $lastSeenEnemyTile = null$ **thenreturn**
end if
if $rc.getRoundNum() - lastReportedRound < reportCooldown$ **thenreturn**
end if
$myLoc \leftarrow rc.getLocation()$; $isLowPaint \leftarrow rc.getPaint() < 50$; $towerLoc \leftarrow null$
if $SpawnTowerLoc \neq null$ **then for** each $ally \in allies$ where $ally.location = SpawnTowerLoc$ **do** $towerLoc \leftarrow SpawnTowerLoc$; **break**
end for
end if
if $towerLoc = null$ **then** $bestDist \leftarrow \infty$
for each $ally \in allies$ where $isAllyTower(ally.type)$ **do**
$d \leftarrow myLoc.distSq(ally.location)$
if $d < bestDist$ **then**$bestDist \leftarrow d$; $towerLoc \leftarrow ally.location$
end if
end for
end if
if $towerLoc = null$ **thenreturn**
end if
$isTowerInRange \leftarrow myLoc.distSq(towerLoc) < 20$
if $\neg isLowPaint$ **and** $\neg isTowerInRange$ **thenreturn**
end if
$msg \leftarrow encodeMsg(1, lastSeenEnemyTile.x, lastSeenEnemyTile.y)$
if $rc.canSendMessage(towerLoc, msg)$ **then**
$rc.sendMessage(towerLoc, msg)$
$lastReportedRound \leftarrow rc.getRoundNum()$; $lastSeenEnemyTile \leftarrow null$
end if

Procedure reportWeakTower(rc : RobotController, enemies : array of RobotInfo, allies : array of RobotInfo)

{I.S : Soldier mengindera robot musuh di sekitarnya}
{F.S : Jika ada tower musuh dengan HP di bawah threshold, kirim pesan ke tower sekutu terdekat}

myLoc, towerLoc	:	MapLocation
maxHp, anggapLemah, bestDist, d, msg	:	integer

if $rc.getRoundNum() - lastTowerReportRound < towerReportCooldown$ **thenreturn**
end if
for each $enemy \in enemies$ where $enemy.type.isTowerType()$ **do**
$maxHp \leftarrow enemy.type.health$
$anggapLemah \leftarrow (maxHp \times percantageWeakTower)/100$
if $enemy.health > anggapLemah$ **thencontinue**
end if
$towerLoc \leftarrow null$
if $SpawnTowerLoc \neq null$ **then for** each $ally \in allies$ where $ally.location = SpawnTowerLoc$ **do** $towerLoc \leftarrow SpawnTowerLoc$; **break**
end for
end if
if $towerLoc = null$ **then** $myLoc \leftarrow rc.getLocation()$; $bestDist \leftarrow \infty$
for each $ally \in allies$ where $isAllyTower(ally.type)$ **do**
$d \leftarrow myLoc.distSq(ally.location)$
if $d < bestDist$ **then**$bestDist \leftarrow d$; $towerLoc \leftarrow ally.location$
end if
end for
end if
if $towerLoc = null$ **thenreturn**
end if
$msg \leftarrow encodeMsg(msgTellWeakTow, enemy.location.x, enemy.location.y)$
if $rc.canSendMessage(towerLoc, msg)$ **then**
$rc.sendMessage(towerLoc, msg)$;
$lastTowerReportRound \leftarrow rc.getRoundNum()$
end if
return
end for

Function getBestSwingDirection(rc : RobotController, enemies : array of RobotInfo) → Direction

{I.S : Soldier sudah memilih ruin yang akan dibangun}
{F.S : Soldier menandai pola tower, mewarnai tile, dan menyelesaikan pembangunan jika pola sudah lengkap}

```
best                    :  Direction
bestCount, count        :  integer
step1, step2            :  MapLocation
```

{Mengembalikan arah kardinal yang menghasilkan mop swing mengenai paling banyak musuh dalam dua langkah ke depan}

$best \leftarrow null$; $bestCount \leftarrow 0$
for each $d \in fourDirections$ **do** $count \leftarrow 0$
$step1 \leftarrow rc.getLocation().add(d)$; $step2 \leftarrow step1.add(d)$
for each $e \in enemies$ **do if** $e.location = step1$ **or** $e.location = step2$
then$count \leftarrow count + 1$
end if
end for
if $count > bestCount$ **then**$bestCount \leftarrow count$; $best \leftarrow d$
end if
end for
if $bestCount > 0$ **then**
    return $best$
elsereturn $null$
end if

Function findRuinForBuilder(rc : RobotController, visible : array of MapInfo) → MapLocation

myLoc, bestRuin, ruinLoc	:	MapLocation
bestDist, myDist, closestSoldier	:	integer
allies	:	array of RobotInfo

{Mencari ruin terdekat yang belum diklaim lebih dari 1 soldier lain}

$myLoc \leftarrow rc.getLocation()$; $bestRuin \leftarrow null$; $bestDist \leftarrow \infty$
for each $tile \in visible$ where $tile.hasRuin()$ **do** $ruinLoc \leftarrow tile.location$
if $rc.senseRobotAtLocation(ruinLoc) \neq null$ **thencontinue**
end if
$myDist \leftarrow myLoc.distSq(ruinLoc)$; $closestSoldier \leftarrow 0$
$allies \leftarrow rc.senseNearbyRobots(kawan)$
for each $ally \in allies$ where $ally.type =$ SOLDIER **do if**
 $ally.location.distSq(ruinLoc) < myDist$
 then$closestSoldier \leftarrow closestSoldier + 1$
end if
end for
if $closestSoldier < 2$ **and** $myDist < bestDist$ **then**
 $bestDist \leftarrow myDist$; $bestRuin \leftarrow ruinLoc$
end if
end for
return $bestRuin$

Procedure handleRuinBuilding(rc : RobotController, ruinLoc : MapLocation)

```
towerType                    :  UnitType
needsPainting, useSecondary  :  boolean
if ruinLoc ≠ chosenRuinLoc then
  chosenTowerType ← chooseTowerType(rc);
  chosenRuinLoc ← ruinLoc
end if
towerType ← chosenTowerType
moveToward(rc, ruinLoc)
if rc.canMarkTowerPattern(towerType, ruinLoc)
  thenrc.markTowerPattern(towerType, ruinLoc)
end if
for each tile ∈ rc.senseNearbyMapInfos(ruinLoc, 8) do
  needsPainting ← tile.getMark() ≠ EMPTY and
  tile.getMark() ≠ tile.getPaint()
if needsPainting then
  useSecondary ← tile.getMark() = ALLY_SECONDARY
if rc.canAttack(tile.location)
  thenrc.attack(tile.location, useSecondary); break
end if
end if
end for
if rc.canCompleteTowerPattern(towerType, ruinLoc) then
  rc.completeTowerPattern(towerType, ruinLoc)
chosenRuinLoc ← null; chosenTowerType ← null
end if
if rc.canUpgradeTower(ruinLoc) thenrc.upgradeTower(ruinLoc)
end if
```

```
Function chooseTowerType(rc : RobotController) → UnitType
  allies                          :  array of RobotInfo
  paintTowers, moneyTowers,       :  integer
  defenseTowers
  t                               :  UnitType
{Memilih tipe tower berdasarkan jumlah tower yang sudah ada,
 menyeimbangkan paint dan money tower}
allies ← rc.senseNearbyRobots(kawan)
paintTowers ← 0; moneyTowers ← 0; defenseTowers ← 0
for each ally ∈ allies do t ← ally.getType()
if t ∈ {LEVEL_ONE_PAINT_TOWER,
 LEVEL_TWO_PAINT_TOWER,
 LEVEL_THREE_PAINT_TOWER} then
 paintTowers ← paintTowers + 1
end if
else if t ∈ {LEVEL_ONE_MONEY_TOWER,
 LEVEL_TWO_MONEY_TOWER,
 LEVEL_THREE_MONEY_TOWER} then
 moneyTowers ← moneyTowers + 1
end if
else if t ∈ {LEVEL_ONE_DEFENSE_TOWER,
 LEVEL_TWO_DEFENSE_TOWER,
 LEVEL_THREE_DEFENSE_TOWER} then
 defenseTowers ← defenseTowers + 1
end if
end for
if paintTowers < 2 thenreturn LEVEL_ONE_PAINT_TOWER
end if
if moneyTowers < 2 thenreturn LEVEL_ONE_MONEY_TOWER
end if
if defenseTowers < 1 thenreturn
 LEVEL_ONE_DEFENSE_TOWER
end if
if paintTowers ≤ moneyTowers thenreturn
 LEVEL_ONE_PAINT_TOWER
end if
return LEVEL_ONE_MONEY_TOWER
```

4.1.3 Pseudocode Strategi Alternatif 2

Kamus Global

MAX_CLAIMS	:	integer
CLAIM_TIMEOUT	:	integer
claimLoc	:	array of MapLocation $[0 \ldots MAX_CLAIMS - 1]$
claimOwner	:	array of integer $[0 \ldots MAX_CLAIMS - 1]$
claimRound	:	array of integer $[0 \ldots MAX_CLAIMS - 1]$
claimType	:	array of UnitType $[0 \ldots MAX_CLAIMS - 1]$
claimCount	:	integer
robotAssignedRuin	:	MapLocation
robotPrevLocation	:	MapLocation
robotExploreTarget	:	MapLocation
robotLastMove	:	Direction
robotBuildingLocked	:	boolean
robotNoProgressTurns	:	integer
robotExploreRefreshRound	:	integer
MAX_VISITED	:	integer
EXPLORE_TTL	:	integer
robotVisited	:	array of MapLocation $[0 \ldots MAX_VISITED - 1]$
robotVisitedCount	:	integer
robotSymInit	:	boolean
robotSymTgts	:	array of MapLocation $[0 \ldots 2]$
robotSymCount	:	integer
robotSymIdx	:	integer
PAINT_REFILL	:	integer
PAINT_BUILD_COMMIT	:	integer
PAINT_BUILD_LOCKED	:	integer
PAINT_DONATE_RESERVE	:	integer
PAINT_EXPLORE_PAINT	:	integer
rng	:	Random
dirs	:	array of Direction $[0 \ldots 7]$

Procedure initSym(input/output rc : RobotController)

I.S. Daftar target simetri robot mungkin belum pernah diinisialisasi.
F.S. Jika belum diinisialisasi, robot menyimpan hingga tiga target simetri berdasarkan posisi tower sekutu.
Proses. Menghasilkan kandidat titik simetri dari tower sekutu, membuang duplikasi, lalu menyimpannya sebagai target eksplorasi awal.

KAMUS LOKAL
w, h : integer
tmp : array of MapLocation
tc : integer
a : RobotInfo
candidates : array of MapLocation
m : MapLocation
dup : boolean
i : integer

ALGORITMA
if robotSymInit **then** return **end if**
w ← lebar peta
h ← tinggi peta
tc ← 0
for setiap robot sekutu a di sekitar **do if** a bukan tower **then** continue **end if**
bentuk tiga kandidat simetri dari posisi tower a
for setiap kandidat m **do** dup ← false
for i ← 0 **to** tc-1 **do if** tmp[i] = m **then** dup ← true
break **end if**
end for
if dup = false dan tc < 9 **then** tmp[tc] ← m
tc ← tc + 1 **end if**
end for
end for
for i ← 0 **to** tc-1 **do if** robotSymCount < 3 **then** robotSymTgts[robotSymCount] ← tmp[i]
robotSymCount ← robotSymCount + 1 **end if**
end for
robotSymInit ← true

Function getSymTarget(rc : RobotController) → MapLocation

I.S. Target simetri robot mungkin belum diinisialisasi atau robot belum memiliki indeks target.
F.S. Fungsi mengembalikan satu target simetri milik robot, atau null jika tidak ada target simetri yang tersedia.
Proses. Menginisialisasi daftar target simetri, lalu memilih satu target berdasarkan ID robot.

KAMUS LOKAL
tidak ada

ALGORITMA
initSym(rc)
if robotSymCount = 0 **then** return null **end if**

if robotSymIdx $<$ 0 **then** robotSymIdx $\leftarrow$ rc.getID() mod robotSymCount
end if
return robotSymTgts[robotSymIdx]

Procedure run(input/output rc : RobotController)

I.S. Robot aktif sedang berada di dalam permainan dan siap menjalankan perilaku sesuai tipenya.
F.S. Pada setiap ronde, robot menjalankan fungsi yang sesuai dengan tipenya lalu mengakhiri giliran.
Proses. Menjadi loop utama yang mendistribusikan kontrol ke SOLDIER, MOPPER, SPLASHER, atau TOWER.

KAMUS LOKAL
type : UnitType

ALGORITMA
while true **do** type $\leftarrow$ rc.getType()
if type = SOLDIER **then** runSoldier(rc) **end if**
else if type = MOPPER **then** runMopper(rc) **end if**
else if type = SPLASHER **then** runSplasher(rc) **end if**
else runTower(rc) **end if**
if terjadi kesalahan saat eksekusi **then** tuliskan *stack trace* **end if**
Clock.yield() **end while**

Procedure runTower(input/output rc : RobotController)

I.S. rc adalah tower aktif yang dapat menyerang musuh, upgrade, dan membangun unit baru.
F.S. Tower menyerang target prioritas, melakukan upgrade bila aman, lalu memproduksi unit yang paling sesuai dengan keadaan sekitar.
Proses. Menyerang musuh dengan HP terendah, menjaga ekonomi tower, lalu memilih dan membangun unit.

KAMUS LOKAL
enemies : array of RobotInfo
weakest : RobotInfo
lowestHp : integer
myType : UnitType
chipCost : integer
infos : array of MapInfo
spawn : UnitType
d : Direction
loc : MapLocation

ALGORITMA
enemies $\leftarrow$ semua musuh di sekitar tower

weakest ← null
lowestHp ← ∞
for setiap musuh r di enemies **do if** r.health < lowestHp **then** lowestHp ← r.health
weakest ← r **end if**
end for
if weakest ada dan tower dapat menyerangnya **then** serang weakest **end if**
myType ← tipe tower saat ini
if tidak ada musuh di sekitar dan myType adalah money tower level 1 atau level 2 **then if** myType = LEVEL_ONE_MONEY_TOWER **then** chipCost ← 2500 **end if**
else chipCost ← 5000 **end if**
if chip mencukupi, jumlah tower minimal 2, dan tower dapat di-upgrade **then** upgrade tower pada lokasi sekarang **end if**
end if
if tower tidak siap melakukan aksi **then** return **end if**
infos ← seluruh informasi petak di sekitar tower
spawn ← chooseSpawn(rc, enemies, infos)
for setiap arah d di dirs **do** loc ← lokasi tower ditambah arah d
if robot bertipe spawn dapat dibangun pada loc **then** bangun robot pada loc
return **end if**
end for

Function chooseSpawn(rc : RobotController, enemies : array of RobotInfo, infos : array of MapInfo) → UnitType

I.S. Tower mengetahui musuh di sekitarnya dan kondisi petak di sekelilingnya.
F.S. Fungsi mengembalikan tipe unit yang paling tepat untuk diproduksi pada kondisi saat itu.
Proses. Memilih unit berdasarkan ancaman musuh, banyaknya enemy paint, fase permainan, dan rasio jumlah SOLDIER terhadap MOPPER.

KAMUS LOKAL
enemyPaintTiles : integer
allies : array of RobotInfo
soldiers : integer
moppers : integer
a : RobotInfo
i : MapInfo

ALGORITMA
for setiap musuh e di enemies **do if** e bertipe MOPPER **then** return MOPPER **end if**

end for
if ada musuh di sekitar atau ada ruin klaim yang terkena enemy paint **then** return MOPPER **end if**
if round saat ini < 80 **then** return SOLDIER **end if**
enemyPaintTiles $\leftarrow 0$
for setiap petak i di infos **do if** i memiliki enemy paint **then** enemyPaintTiles $\leftarrow$ enemyPaintTiles $+ 1$ **end if**
end for
if enemyPaintTiles ≥ 4 **then** return MOPPER **end if**
allies $\leftarrow$ semua sekutu di sekitar tower
soldiers $\leftarrow 0$
moppers $\leftarrow 0$
for setiap sekutu a di allies **do if** a bertipe SOLDIER **then** soldiers $\leftarrow$ soldiers $+ 1$ **end if**
if a bertipe MOPPER **then** moppers $\leftarrow$ moppers $+ 1$ **end if**
end for
if moppers $= 0$ dan soldiers ≥ 3 **then** return MOPPER **end if**
if moppers $= 1$ dan soldiers ≥ 6 **then** return MOPPER **end if**
if jumlah tower minimal 4 dan round > 150 **then** return SPLASHER **end if**
return SOLDIER

Procedure runSoldier(input/output rc : RobotController)

I.S. rc adalah SOLDIER aktif yang dapat bergerak, mengecat, menyerang, refill paint, dan membangun tower.
F.S. Soldier menjalankan satu aksi prioritas pada ronde tersebut, seperti refill, membangun tower, sabotase ruin musuh, menyerang target, atau eksplorasi.
Proses. Mengutamakan penyelesaian build dan keselamatan paint, lalu beralih ke rush simetri, sabotase, pertarungan, atau eksplorasi.

KAMUS LOKAL
infos : array of MapInfo
enemies : array of RobotInfo
allies : array of RobotInfo
nearestEnemyTower : RobotInfo
sym : MapLocation
pt : MapLocation
enemyRuin : MapLocation
d : Direction
loc : MapLocation
inf : MapInfo
newRuin : MapLocation
towerLoc : MapLocation
adjacent : boolean
nearestFoe : RobotInfo

ALGORITMA
if robot sedang terkunci membangun dan robotAssignedRuin ada **then if** paint robot < PAINT_BUILD_LOCKED **then** robotBuildingLocked ← false
refill(rc)
return **end if**
doBuildTower(rc, robotAssignedRuin)
return **end if**
if paint robot < PAINT_REFILL **then** robotAssignedRuin ← null
robotBuildingLocked ← false
refill(rc)
return **end if**
paintUnder(rc)
infos ← seluruh informasi petak di sekitar
enemies ← semua musuh di sekitar
allies ← semua sekutu di sekitar
nearestEnemyTower ← nearestEnemy(rc, enemies, true)
if round saat ini ≤ 60 dan tidak ada enemy tower terdekat **then** sym ← getSymTarget(rc)
if sym ada dan jarak ke sym > 9 **then** move(rc, sym, false)
if aksi masih tersedia dan bytecode tersisa > 3000 **then** pt ← bestPaintTile(rc, infos)
if pt ada dan robot dapat menyerang pt **then** serang pt **end if**
end if
return **end if**
end if
enemyRuin ← findEnemyRuin(rc, infos)
if enemyRuin ada **then if** jarak ke enemyRuin > 8 **then** move(rc, enemyRuin, false)
return **end if**
for setiap arah d di dirs **do** loc ← enemyRuin ditambah arah d
if loc belum bisa di-sense atau loc tidak dapat diserang **then** continue **end if**
inf ← informasi petak loc
if loc tidak passable atau loc sudah memiliki ally paint **then** continue **end if**
serang loc
return **end for**
explore(rc, infos)
return **end if**
if robotAssignedRuin ada **then if** ruin target belum dapat di-sense **then** move(rc, robotAssignedRuin, false)
return **end if**
if sudah ada robot pada ruin target **then** releaseRuin(rc, robotAssignedRuin) **end if**

else if paint robot ≥ PAINT_BUILD_COMMIT **then** robotBuildingLocked ← true **end if**
doBuildTower(rc, robotAssignedRuin)
return **end if**
end if
newRuin ← findNearbyRuin(rc, infos, allies)
if newRuin ada **then if** paint robot ≥ PAINT_BUILD_COMMIT **then** robotBuildingLocked ← true **end if**
doBuildTower(rc, newRuin)
return **end if**
if nearestEnemyTower ada **then** towerLoc ← nearestEnemyTower.location
if round saat ini genap **then if** robot dapat menyerang towerLoc **then** serang towerLoc **end if**
else move(rc, towerLoc, false) **end if**
end if
else adjacent ← (jarak ke towerLoc ≤ 4)
move(rc, towerLoc, adjacent) **end if**
return **end if**
nearestFoe ← nearestEnemy(rc, enemies, false)
if nearestFoe ada **then if** robot dapat menyerang nearestFoe.location **then** serang nearestFoe.location **end if**
else if paint robot ≥ PAINT_EXPLORE_PAINT **then** move(rc, nearestFoe.location, false) **end if**
else move(rc, nearestFoe.location, true) **end if**
return **end if**
explore(rc, infos)

Procedure paintUnder(input/output rc : RobotController)

I.S. Robot sedang berdiri di satu petak dan mungkin masih memiliki aksi menyerang atau mengecat.
F.S. Jika perlu, robot mengecat atau membersihkan petak tempat ia berdiri.
Proses. Jika robot sedang dekat ruin build, robot mengikuti mark pola; jika tidak, robot membersihkan enemy paint atau mengecat petak netral di bawahnya.

KAMUS LOKAL
loc : MapLocation
info : MapInfo
mark : PaintType

ALGORITMA
if robot tidak siap melakukan aksi **then** return **end if**
loc ← lokasi robot saat ini
info ← informasi petak loc

if robotAssignedRuin ada dan jarak loc ke robotAssignedRuin paling jauh 8 **then** mark ← mark pada petak loc
if mark tidak kosong dan paint pada petak tidak sama dengan mark dan loc dapat diserang **then** serang loc dengan mode paint sesuai mark **end if**
return **end if**
if petak loc memiliki enemy paint **then if** loc dapat diserang **then** serang loc **end if**
return **end if**
if petak loc belum memiliki ally paint dan paint robot cukup dan loc dapat diserang **then** serang loc **end if**

Procedure runMopper(input/output rc : RobotController)

I.S. rc adalah MOPPER aktif yang dapat membersihkan paint, menyerang, bergerak, dan memberi paint ke sekutu.
F.S. Mopper melakukan refill, membantu soldier sekutu, mencegat rush musuh, membersihkan area kotor, mengejar musuh ber-paint rendah, atau eksplorasi.
Proses. Mopper berperan sebagai unit pembersih dan pendukung yang memprioritaskan ancaman di sekitar tower dan ruin.

KAMUS LOKAL
enemies : array of RobotInfo
allies : array of RobotInfo
infos : array of MapInfo
a : RobotInfo
give : integer
rushTarget : RobotInfo
ruin : MapLocation
dirty : MapLocation
lowestPaintEnemy : RobotInfo
lowestPaint : integer
nearestDirt : MapLocation
nearestDist : integer
i : MapInfo
d : integer

ALGORITMA
if paint robot < PAINT_REFILL **then** refill(rc)
return **end if**
enemies ← semua musuh di sekitar
allies ← semua sekutu di sekitar
infos ← seluruh informasi petak di sekitar
if robot masih punya aksi **then for** setiap sekutu a di allies **do if** a bukan SOLDIER **then** continue **end if**

if paint a sudah cukup **then** continue **end if**
give ← minimum dari 30 dan (paint robot - PAINT_DONATE_RESERVE)
if give > 0 dan paint dapat ditransfer ke a **then** transfer paint ke a
break **end if**
end for
end if
rushTarget ← antiRushTarget(rc, enemies, allies)
if rushTarget ada **then if** robot dapat menyerang rushTarget.location **then** serang rushTarget.location **end if**
else move(rc, rushTarget.location, false) **end if**
return **end if**
for setiap ruin pada {robotAssignedRuin, ruinWithEnemyPaint(infos)} **do if** ruin = null **then** continue **end if**
dirty ← enemyPaintNear(infos, rc, ruin)
if dirty ada **then if** robot dapat menyerang dirty **then** serang dirty **end if**
else move(rc, dirty, false) **end if**
return **end if**
end for
lowestPaintEnemy ← null
lowestPaint ← ∞
for setiap musuh e di enemies **do if** e.paintAmount < lowestPaint **then** lowestPaint ← e.paintAmount
lowestPaintEnemy ← e **end if**
end for
if lowestPaintEnemy ada **then if** robot dapat menyerang lowestPaintEnemy.location **then** serang lowestPaintEnemy.location **end if**
else move(rc, lowestPaintEnemy.location, false) **end if**
return **end if**
nearestDirt ← null
nearestDist ← ∞
for setiap petak i di infos **do if** i tidak memiliki enemy paint **then** continue **end if**
d ← jarak robot ke i
if d < nearestDist **then** nearestDist ← d
nearestDirt ← lokasi i **end if**
end for
if nearestDirt ada **then if** robot dapat menyerang nearestDirt **then** serang nearestDirt **end if**
else move(rc, nearestDirt, false) **end if**
return **end if**
explore(rc, infos)

Function antiRushTarget(rc : RobotController, enemies : array of RobotInfo, allies : array of RobotInfo) → RobotInfo

I.S. Robot mengetahui musuh dan sekutu di sekitarnya.
F.S. Fungsi mengembalikan target rush musuh terdekat yang layak dicegat, atau null jika tidak ada.
Proses. Memprioritaskan MOPPER musuh, atau SOLDIER musuh yang cukup dekat dengan tower sekutu.

KAMUS LOKAL
best : RobotInfo
bestDist : integer
e : RobotInfo
a : RobotInfo
nearOurTower : boolean
d : integer

ALGORITMA
best ← null
bestDist ← ∞
for setiap musuh e di enemies **do if** e bukan MOPPER dan e bukan SOLDIER **then** continue **end if**
if e bertipe SOLDIER **then** nearOurTower ← false
for setiap sekutu a di allies **do if** a bukan tower **then** continue **end if**
if jarak e ke tower sekutu paling jauh 20 **then** nearOurTower ← true
break **end if**
end for
if nearOurTower = false **then** continue **end if**
end if
d ← jarak robot ke e
if d < bestDist **then** bestDist ← d
best ← e **end if**
end for
return best

Function enemyPaintNear(infos : array of MapInfo, rc : RobotController, ruin : MapLocation) → MapLocation

I.S. Tersedia daftar petak di sekitar robot dan sebuah ruin acuan.
F.S. Fungsi mengembalikan petak enemy paint terdekat dari robot yang masih berada di sekitar ruin, atau null jika tidak ada.
Proses. Menyaring petak-petak ber-enemy paint dalam area ruin lalu memilih yang paling dekat.

KAMUS LOKAL
best : MapLocation
bestDist : integer

i : MapInfo
d : integer

ALGORITMA
best ← null
bestDist ← ∞
for setiap petak i di infos **do if** jarak i ke ruin > 8 atau i tidak memiliki enemy paint **then** continue **end if**
d ← jarak robot ke i
if d < bestDist **then** bestDist ← d
best ← lokasi i **end if**
end for
return best

Procedure runSplasher(input/output rc : RobotController)

I.S. rc adalah SPLASHER aktif yang dapat menyerang area, bergerak, dan refill paint.
F.S. Splasher memilih pusat splash terbaik, menjauh dari enemy paint di bawahnya, atau melakukan eksplorasi.
Proses. Memprioritaskan penghancuran pola enemy paint bertanda, lalu splash area padat, lalu reposition atau eksplorasi.

KAMUS LOKAL
infos : array of MapInfo
pb : MapLocation
best : MapLocation

ALGORITMA
if paint robot < PAINT_REFILL **then** refill(rc)
return **end if**
infos ← seluruh informasi petak di sekitar
if bytecode tersisa > 5000 **then** pb ← bestPatternBreak(infos, rc)
if pb ada dan robot dapat menyerang pb **then** serang pb
return **end if**
end if
if bytecode tersisa > 4000 **then** best ← bestSplash(infos, rc)
if best ada dan robot dapat menyerang best **then** serang best
return **end if**
end if
if petak tempat robot berdiri memiliki enemy paint **then** move(rc, lokasi robot saat ini, true)
return **end if**
explore(rc, infos)

Function bestPatternBreak(infos : array of MapInfo, rc : RobotController) → MapLocation

I.S. Splasher mengetahui petak di sekitarnya dan ingin memilih pusat serangan area terbaik.
F.S. Fungsi mengembalikan pusat splash terbaik untuk merusak tile musuh, terutama yang sudah bertanda pola.
Proses. Menilai setiap pusat splash 3x3 dengan bobot lebih besar untuk enemy paint yang juga memiliki mark.

KAMUS LOKAL
best : MapLocation
bestCount : integer
center : MapLocation
count : integer
i, m : MapInfo
dx, dy : integer

ALGORITMA
best ← null
bestCount ← 0
for setiap petak i di infos **do** center ← lokasi i
if robot tidak dapat menyerang center **then** continue **end if**
count ← 0
for setiap petak m di infos **do** dx ← center.x - m.x
dy ← center.y - m.y
if m berada di luar area splash 3x3 **then** continue **end if**
if m memiliki mark dan enemy paint **then** count ← count + 2 **end if**
else if m memiliki enemy paint **then** count ← count + 1 **end if**
end for
if count > bestCount **then** bestCount ← count
best ← center **end if**
end for
return best

Function bestSplash(infos : array of MapInfo, rc : RobotController) → MapLocation

I.S. Splasher ingin mencari pusat splash dengan cakupan enemy paint terbanyak.
F.S. Fungsi mengembalikan pusat splash 3x3 terbaik berdasarkan jumlah tile enemy paint.
Proses. Menghitung banyaknya enemy paint pada setiap area splash yang mungkin diserang.

KAMUS LOKAL

best : MapLocation
bestCount : integer
center : MapLocation
count : integer
i, m : MapInfo
dx, dy : integer

ALGORITMA
best ← null
bestCount ← 0
for setiap petak i di infos **do** center ← lokasi i
if robot tidak dapat menyerang center **then** continue **end if**
count ← 0
for setiap petak m di infos **do** dx ← center.x - m.x
dy ← center.y - m.y
if m berada di luar area splash 3x3 **then** continue **end if**
if m memiliki enemy paint **then** count ← count + 1 **end if**
end for
if count > bestCount **then** bestCount ← count
best ← center **end if**
end for
return best

Procedure move(input/output rc : RobotController, input tgt : MapLocation, input moveAway : boolean)

I.S. Robot memiliki target lokasi yang ingin didekati atau dijauhi, dan mungkin masih dapat bergerak.
F.S. Jika ada arah yang layak, robot bergerak ke arah dengan progres terbaik.
Proses. Menilai semua arah berdasarkan perubahan jarak, warna paint tujuan, serta pencegahan osilasi gerak.

KAMUS LOKAL
cur : MapLocation
best : Direction
bestFallback : Direction
bestDelta : integer
bestDeltaFB : integer
d : Direction
next : MapLocation
distNow, distNext : integer
delta : integer
pt : PaintType
before : MapLocation

ALGORITMA

if tgt = null atau robot tidak siap bergerak **then** return **end if**
cur ← lokasi robot saat ini
best ← null
bestFallback ← null
bestDelta ← $-\infty$
bestDeltaFB ← $-\infty$
for setiap arah d di dirs **do if** robot tidak dapat bergerak ke d **then** continue **end if**
next ← cur + d
if next dapat di-sense dan next tidak passable **then** continue **end if**
distNow ← jarak cur ke tgt
distNext ← jarak next ke tgt
if moveAway = true **then** delta ← distNext - distNow **end if**
else delta ← distNow - distNext **end if**
if next dapat di-sense **then** pt ← paint pada next
if pt adalah ally paint **then** delta ← delta + 1 **end if**
if pt adalah enemy paint **then** delta ← delta - 2 **end if**
end if
if delta > bestDeltaFB **then** bestDeltaFB ← delta
bestFallback ← d **end if**
if robotPrevLocation ada dan next = robotPrevLocation **then** continue **end if**
if robotLastMove ada dan d adalah kebalikan langkah terakhir **then** continue **end if**
if delta > bestDelta **then** bestDelta ← delta
best ← d **end if**
end for
if best = null **then** best ← bestFallback **end if**
before ← lokasi robot sebelum bergerak
if best ada dan robot dapat bergerak ke best **then** robotPrevLocation ← cur
robot bergerak ke best
robotLastMove ← best **end if**
if lokasi robot tidak berubah **then** robotNoProgressTurns ← robotNoProgressTurns + 1 **end if**
else robotNoProgressTurns ← 0 **end if**

Procedure explore(input/output rc : RobotController, input infos : array of MapInfo)

I.S. Robot tidak memiliki aksi prioritas lain dan dapat melakukan eksplorasi.
F.S. Robot bergerak menuju target eksplorasi aktif dan, bila memungkinkan, mengecat petak terbaik di sekitar.
Proses. Menyegarkan target eksplorasi jika macet, kadaluarsa, atau sudah sampai, lalu bergerak dan mengecat area baru.

KAMUS LOKAL
cur : MapLocation
expired : boolean
arrived : boolean
sym : MapLocation
pt : MapLocation

ALGORITMA
if robotNoProgressTurns ≥ 3 **then** robotExploreTarget $\leftarrow$ null
robotExploreRefreshRound $\leftarrow -999$
robotLastMove $\leftarrow$ null
robotPrevLocation $\leftarrow$ null
robotNoProgressTurns $\leftarrow 0$ **end if**
cur $\leftarrow$ lokasi robot saat ini
expired $\leftarrow$ umur target eksplorasi melebihi EXPLORE_TTL
arrived $\leftarrow$ robot sudah cukup dekat dengan robotExploreTarget
if robotExploreTarget = null atau expired atau arrived **then if** round saat ini ≤ 80 **then** sym $\leftarrow$ getSymTarget(rc) **end if**
else sym $\leftarrow$ null **end if**
if sym ada dan jarak ke sym > 16 **then** robotExploreTarget $\leftarrow$ sym **end if**
else robotExploreTarget $\leftarrow$ pickExploreTarget(rc) **end if**
robotExploreRefreshRound $\leftarrow$ round saat ini **end if**
move(rc, robotExploreTarget, false)
if aksi masih tersedia dan bytecode tersisa > 3000 **then** pt $\leftarrow$ bestPaintTile(rc, infos)
if pt ada dan robot dapat menyerang pt **then** serang pt **end if**
end if

Function pickExploreTarget(rc : RobotController) → MapLocation

I.S. Robot ingin memilih waypoint eksplorasi baru dari daftar waypoint tetap di peta.
F.S. Fungsi mengembalikan waypoint terbaik yang belum dikunjungi, atau fallback jika kandidat utama habis.
Proses. Memilih waypoint terdekat yang belum dikunjungi, dengan bias ke kuadran yang sesuai dengan ID robot.

KAMUS LOKAL
w, h, id : integer
waypoints : array of MapLocation
bestInQuadrant : MapLocation
bestAny : MapLocation
distInQuadrant : integer
distAny : integer
i : integer

dist : integer

ALGORITMA
bangun daftar waypoint tetap pada sudut, sisi, dan area tengah peta
if robotExploreTarget ada dan robot sudah dekat dengannya **then** visitedAdd(robotExploreTarget) **end if**
if robotVisitedCount $\geq$ banyak waypoint - 1 **then** robotVisitedCount $\leftarrow$ 0 **end if**
bestInQuadrant $\leftarrow$ null
bestAny $\leftarrow$ null
distInQuadrant $\leftarrow \infty$
distAny $\leftarrow \infty$
for setiap waypoint i **do if** waypoint i sudah pernah dikunjungi **then** continue **end if**
if waypoint i sama dengan target eksplorasi saat ini **then** continue **end if**
dist $\leftarrow$ jarak robot ke waypoint i
if dist $<$ distAny **then** distAny $\leftarrow$ dist
bestAny $\leftarrow$ waypoint i **end if**
if (i mod 4) = (id mod 4) dan dist $<$ distInQuadrant **then** distInQuadrant $\leftarrow$ dist
bestInQuadrant $\leftarrow$ waypoint i **end if**
end for
if bestInQuadrant ada **then** return bestInQuadrant **end if**
if bestAny ada **then** return bestAny **end if**
else return waypoint dengan indeks (id mod banyak waypoint) **end if**

Function visitedHas(loc : MapLocation) $\rightarrow$ boolean

I.S. Tersedia satu lokasi yang ingin dicek pada daftar waypoint yang sudah dikunjungi.
F.S. Fungsi mengembalikan true jika lokasi sudah ada pada daftar visited, dan false jika belum.
Proses. Menelusuri seluruh elemen daftar visited hingga menemukan lokasi yang sama.

KAMUS LOKAL
i : integer

ALGORITMA
if loc = null **then** return false **end if**
for i $\leftarrow$ 0 **to** robotVisitedCount-1 **do if** robotVisited[i] = loc **then** return true **end if**
end for
return false

Procedure visitedAdd(input loc : MapLocation)

I.S. Tersedia lokasi yang mungkin perlu dicatat sebagai waypoint yang sudah dikunjungi.
F.S. Jika lokasi valid dan belum tercatat, lokasi dimasukkan ke daftar visited.
Proses. Menolak null dan duplikasi, lalu menambahkan lokasi selama kapasitas daftar masih tersedia.

KAMUS LOKAL
tidak ada

ALGORITMA
if loc = null atau visitedHas(loc) **then** return **end if**
if robotVisitedCount < MAX_VISITED **then** robotVisited[robotVisitedCount] ← loc
robotVisitedCount ← robotVisitedCount + 1 **end if**

Function bestPaintTile(rc : RobotController, infos : array of MapInfo) → MapLocation

I.S. Robot ingin memilih petak terbaik untuk dibersihkan atau dicat.
F.S. Fungsi mengembalikan petak enemy paint terdekat, atau petak netral terdekat jika tidak ada enemy paint.
Proses. Menyaring petak yang bisa diserang dan tidak mengganggu area build ruin, lalu memprioritaskan enemy paint.

KAMUS LOKAL
bestEnemy : MapLocation
bestEmpty : MapLocation
bestEnemyD : integer
bestEmptyD : integer
i : MapInfo
loc : MapLocation
d : integer

ALGORITMA
bestEnemy ← null
bestEmpty ← null
bestEnemyD ← ∞
bestEmptyD ← ∞
for setiap petak i di infos **do** loc ← lokasi i
if loc tidak dapat diserang atau i tidak passable **then** continue **end if**
if robotAssignedRuin ada dan loc berada di dalam area build ruin **then** continue **end if**
d ← jarak robot ke loc
if loc memiliki enemy paint dan d < bestEnemyD **then** bestEnemyD ← d

bestEnemy ← loc **end if**
else if loc belum memiliki ally paint dan d < bestEmptyD **then** bestEmptyD ← d
bestEmpty ← loc **end if**
end for
if bestEnemy ada **then** return bestEnemy **end if**
else return bestEmpty **end if**

Procedure refill(rc : RobotController) → tidak ada

Procedure refill(input/output rc : RobotController)

I.S. Robot membutuhkan paint tambahan dan ingin mencari tower sekutu terbaik untuk refill.
F.S. Robot bergerak menuju tower refill terbaik atau mengambil paint jika sudah cukup dekat.
Proses. Memilih tower sekutu terdekat dengan preferensi lebih tinggi untuk paint tower.

KAMUS LOKAL
tower : MapLocation
bestDist : integer
a : RobotInfo
isPaintTower : boolean
d : integer
need : integer

ALGORITMA
tower ← null
bestDist ← ∞
for setiap sekutu a di sekitar **do if** a bukan tower **then** continue **end if**
tentukan apakah a adalah paint tower
d ← jarak robot ke a
if a adalah paint tower **then** d ← $d/2$ **end if**
if d < bestDist **then** bestDist ← d
tower ← lokasi a **end if**
end for
if tower = null **then** explore(rc, rc.senseNearbyMapInfos())
return **end if**
if jarak robot ke tower > 2 **then** move(rc, tower, false)
return **end if**
need ← kapasitas paint robot - paint robot saat ini
if need > 0 dan paint dapat diambil dari tower **then** ambil paint dari tower **end if**
else if robot masih bisa bergerak **then** move(rc, tower, false) **end if**

Procedure doBuildTower(input/output rc : RobotController, input ruin : MapLocation)

I.S. Robot memiliki ruin target yang ingin dibangun menjadi tower.
F.S. Robot bergerak ke ruin, memilih tipe tower, menandai pola, mengecat pola, atau menyelesaikan tower jika polanya sudah lengkap.
Proses. Menjaga seluruh alur pembangunan tower, mulai dari memilih tipe hingga menyelesaikan pattern.

KAMUS LOKAL
idx : integer
towerType : UnitType
alreadyMarked : boolean
i : MapInfo
l : MapLocation

ALGORITMA
if ruin = null **then** return **end if**
if paint robot < PAINT_BUILD_LOCKED **then** robotBuildingLocked ← false
refill(rc)
return **end if**
idx ← findClaimIdx(ruin)
if idx valid dan claimType[idx] sudah ada **then** towerType ← claimType[idx] **end if**
else towerType ← chooseTowerType(rc, ruin)
if idx valid **then** claimType[idx] ← towerType **end if**
end if
if jarak robot ke ruin > 2 **then** move(rc, ruin, false)
return **end if**
if sudah ada robot pada ruin **then** releaseRuin(rc, ruin)
return **end if**
if tower pattern sudah lengkap dan bisa diselesaikan **then** selesaikan tower
releaseRuin(rc, ruin)
return **end if**
alreadyMarked ← false
for setiap petak i di sekitar **do** l ← lokasi i
if l bukan ruin, l berada di dalam radius pola, dan i sudah memiliki mark **then** alreadyMarked ← true
break **end if**
end for
if belum ada mark pola **then if** tower pattern dapat ditandai **then** tandai tower pattern
if aksi masih tersedia dan bytecode cukup **then** paintTiles(rc, ruin, towerType) **end if**
end if
return **end if**

if aksi masih tersedia dan bytecode cukup **then** paintTiles(rc, ruin, towerType) **end if**

Procedure paintTiles(input/output rc : RobotController, input ruin : MapLocation, input towerType : UnitType)

I.S. Pola tower sudah diketahui dan robot ingin mengisi petak-petak pola yang masih salah warna.
F.S. Robot mengecat satu petak pola yang belum sesuai, atau menyelesaikan tower bila semua petak pola telah lengkap.
Proses. Memilih petak pola terdekat yang belum benar warnanya, lalu menyerangnya atau bergerak mendekatinya.

KAMUS LOKAL
toFill : MapLocation
bestDist : integer
useSecondary : boolean
i : MapInfo
l : MapLocation
mark : PaintType
paint : PaintType
d : integer

ALGORITMA
if robot tidak siap melakukan aksi **then if** jarak robot ke ruin $>$ 2 **then** move(rc, ruin, false) **end if**
return **end if**
toFill $\leftarrow$ null
bestDist $\leftarrow \infty$
useSecondary $\leftarrow$ false
for setiap petak i di sekitar **do** l $\leftarrow$ lokasi i
if l berada di luar pola, l adalah ruin, atau i tidak passable **then** continue **end if**
mark $\leftarrow$ mark pada l
paint $\leftarrow$ paint pada l
if mark kosong atau paint sudah sesuai mark **then** continue **end if**
d $\leftarrow$ jarak robot ke l
if d $<$ bestDist **then** bestDist $\leftarrow$ d
toFill $\leftarrow$ l
useSecondary $\leftarrow$ (mark = ALLY_SECONDARY) **end if**
end for
if toFill = null **then if** tower pattern sudah lengkap dan bisa diselesaikan **then** selesaikan tower
releaseRuin(rc, ruin) **end if**
else if jarak robot ke ruin $>$ 2 **then** move(rc, ruin, false) **end if**
else releaseRuin(rc, ruin) **end if**

return **end if**
if robot dapat menyerang toFill **then** serang toFill dengan mode paint yang sesuai **end if**
else move(rc, toFill, false) **end if**

Function chooseTowerType(rc : RobotController, ruin : MapLocation) → UnitType

---

I.S. Robot ingin menentukan tipe tower yang paling cocok untuk sebuah ruin.
F.S. Fungsi mengembalikan tipe tower level satu yang sesuai dengan fase permainan, keseimbangan ekonomi, dan posisi ruin.
Proses. Mengutamakan money tower di early game, menjaga minimal satu paint tower, lalu memilih defense tower untuk chokepoint atau menyeimbangkan money dan paint tower.

KAMUS LOKAL
paintTowers : integer
moneyTowers : integer
a : RobotInfo
t : UnitType
wallCount : integer
d : Direction
n : MapLocation

ALGORITMA
if round saat ini $<$ 120 **then** return LEVEL_ONE_MONEY_TOWER **end if**
paintTowers ← 0
moneyTowers ← 0
for setiap sekutu a di sekitar **do** t ← tipe a
if t adalah paint tower **then** paintTowers ← paintTowers + 1 **end if**
if t adalah money tower **then** moneyTowers ← moneyTowers + 1 **end if**

end for
if paintTowers = 0 **then** return LEVEL_ONE_PAINT_TOWER **end if**

wallCount ← 0
for setiap arah kardinal d **do** n ← ruin + d
if n tidak dapat di-sense atau n tidak passable **then** wallCount ← wallCount + 1 **end if**
end for
if wallCount $\geq$ 2 **then** return LEVEL_ONE_DEFENSE_TOWER **end if**
if moneyTowers $>$ paintTowers **then** return LEVEL_ONE_PAINT_TOWER **end if**

else return LEVEL_ONE_MONEY_TOWER **end if**

Function findClaimIdx(r : MapLocation) → integer

I.S. Tersedia satu ruin yang ingin dicari pada daftar claim aktif.
F.S. Fungsi mengembalikan indeks claim jika ditemukan, atau -1 jika tidak ada.
Proses. Menelusuri seluruh claim aktif dan membandingkan lokasinya.

KAMUS LOKAL
i : integer

ALGORITMA
for i ← 0 **to** claimCount-1 **do if** claimLoc[i] ada dan claimLoc[i] = r **then** return i **end if**
end for
return -1

Procedure cleanClaims(input/output rc : RobotController)

I.S. Daftar claim mungkin masih berisi claim lama yang sudah kedaluwarsa.
F.S. Semua claim yang melebihi batas waktu dihapus dari daftar aktif.
Proses. Menyapu seluruh claim dan mengganti claim kedaluwarsa dengan elemen terakhir yang masih aktif.

KAMUS LOKAL
now : integer
i : integer

ALGORITMA
now ← round saat ini
for i ← 0 **to** claimCount-1 **do if** claimLoc[i] kosong atau umur claim belum melebihi CLAIM_TIMEOUT **then** continue **end if**
gantikan claim ke-i dengan claim terakhir
kosongkan elemen terakhir
claimCount ← claimCount - 1
i ← i - 1 **end for**

Function canClaim(rc : RobotController, r : MapLocation) → boolean

I.S. Robot ingin mengetahui apakah sebuah ruin masih boleh diklaim.
F.S. Fungsi mengembalikan true jika ruin bebas, claim lama sudah timeout, atau ruin sudah diklaim oleh robot ini sendiri.
Proses. Mengecek status claim ruin berdasarkan indeks claim, umur

claim, dan pemilik claim.

KAMUS LOKAL
i : integer

ALGORITMA
i ← findClaimIdx(r)
if i < 0 **then** return true **end if**
if round saat ini - claimRound[i] > CLAIM_TIMEOUT **then** return true **end if**
if claimOwner[i] = ID robot saat ini **then** return true **end if**
return false

Procedure claimRuin(input/output rc : RobotController, input r : MapLocation)

I.S. Robot ingin menandai sebuah ruin sebagai target build miliknya.
F.S. Ruin tercatat sebagai claim aktif milik robot dan robotAssignedRuin diperbarui.
Proses. Menambah claim baru jika ruin belum ada di daftar, atau memperbarui claim lama jika ruin sudah pernah diclaim.

KAMUS LOKAL
i : integer

ALGORITMA
i ← findClaimIdx(r)
if i < 0 **then if** claimCount ≥ MAX_CLAIMS **then** return **end if**
i ← claimCount
claimCount ← claimCount + 1 **end if**
claimLoc[i] ← r
claimOwner[i] ← ID robot saat ini
claimRound[i] ← round saat ini
robotAssignedRuin ← r

Procedure releaseRuin(input/output rc : RobotController, input r : MapLocation)

I.S. Robot mungkin masih memegang claim terhadap sebuah ruin.
F.S. Jika ruin memang diklaim robot ini, claim dihapus dan status build robot di-reset.
Proses. Menghapus claim aktif milik robot pada ruin tersebut lalu melepaskan target build.

KAMUS LOKAL
i : integer

ALGORITMA

if r = null **then** return **end if**
i ← findClaimIdx(r)
if i valid dan claimOwner[i] = ID robot saat ini **then** gantikan claim ke-i dengan claim terakhir
kosongkan elemen terakhir
claimCount ← claimCount - 1 **end if**
robotAssignedRuin ← null
robotBuildingLocked ← false

Function ruinWithEnemyPaint(infos : array of MapInfo) → MapLocation

---

I.S. Tersedia daftar petak sekitar yang mungkin memuat ruin dan enemy paint.
F.S. Fungsi mengembalikan ruin klaim yang di sekitarnya terdapat enemy paint, atau null jika tidak ada.
Proses. Mencari ruin aktif yang sedang terancam oleh paint musuh di area polanya.

KAMUS LOKAL
i, o : MapInfo

ALGORITMA
for setiap petak i di infos **do if** i tidak memiliki ruin atau ruin tersebut bukan claim aktif **then** continue **end if**
for setiap petak o di infos **do if** o berada di sekitar ruin dan o memiliki enemy paint **then** return lokasi ruin **end if**
end for
end for
return null

Function findNearbyRuin(rc : RobotController, infos : array of MapInfo, allies : array of RobotInfo) → MapLocation

---

I.S. Robot ingin mencari ruin kosong terdekat yang masih layak diklaim.
F.S. Fungsi mengembalikan ruin terbaik yang berhasil diklaim robot, atau null jika tidak ada kandidat valid.
Proses. Membersihkan claim lama lalu memilih ruin terdekat yang bebas, belum ditempati, dan tidak kalah dekat dengan soldier sekutu lain.

KAMUS LOKAL
best : MapLocation
closestDist : integer
info : MapInfo
ruin : MapLocation
myDist : integer
closerAllyExists : boolean

a : RobotInfo

ALGORITMA
cleanClaims(rc)
best ← null
closestDist ← ∞
for setiap petak info di infos **do if** info tidak memiliki ruin **then** continue **end if**
ruin ← lokasi ruin
if ruin tidak bisa diklaim atau ruin sudah ditempati robot **then** continue **end if**
myDist ← jarak robot ke ruin
closerAllyExists ← false
for setiap sekutu a di allies **do if** a bertipe SOLDIER dan jarak a ke ruin < myDist **then** closerAllyExists ← true
break **end if**
end for
if closerAllyExists **then** continue **end if**
if myDist < closestDist **then** closestDist ← myDist
best ← ruin **end if**
end for
if best ada **then** claimRuin(rc, best) **end if**
return best

Function findEnemyRuin(rc : RobotController, infos : array of MapInfo) → MapLocation

I.S. Robot ingin mencari ruin musuh yang paling layak disabotase.
F.S. Fungsi mengembalikan ruin dengan progres build musuh tertinggi, atau null jika tidak ada kandidat yang cukup penting.
Proses. Menilai setiap ruin berdasarkan banyaknya mark, enemy paint, dan ally paint di sekitarnya.

KAMUS LOKAL
best : MapLocation
mostProgress : integer
info, o : MapInfo
ruin, l : MapLocation
enemyPaint : integer
marks : integer
allyPaint : integer
progress : integer

ALGORITMA
best ← null
mostProgress ← 0

for setiap petak info di infos **do if** info tidak memiliki ruin **then** continue **end if**
ruin ← lokasi ruin
if ruin sudah ditempati robot atau ruin sama dengan target build kita sendiri **then** continue **end if**
enemyPaint ← 0
marks ← 0
allyPaint ← 0
for setiap petak o di infos **do** l ← lokasi o
if l adalah ruin atau l berada di luar radius ruin **then** continue **end if**
if o memiliki enemy paint **then** enemyPaint ← enemyPaint + 1 **end if**
if o memiliki mark **then** marks ← marks + 1 **end if**
if o memiliki ally paint **then** allyPaint ← allyPaint + 1 **end if**
end for
progress ← 2 $\times marks + enemyPaint - allyPaint$
if progress ≥ 1 dan progress > mostProgress **then** mostProgress ← progress
best ← ruin **end if**
end for
return best

Function nearestEnemy(rc : RobotController, enemies : array of RobotInfo, wantTower : boolean) → RobotInfo

I.S. Robot ingin mencari musuh terdekat dari kategori tower atau non-tower.
F.S. Fungsi mengembalikan musuh terdekat yang sesuai kategori, atau null jika tidak ada.
Proses. Menyaring musuh berdasarkan apakah ia tower atau bukan, lalu memilih yang jaraknya paling kecil.

KAMUS LOKAL
best : RobotInfo
bestDist : integer
e : RobotInfo
d : integer

ALGORITMA
best ← null
bestDist ← ∞
for setiap musuh e di enemies **do if** status tower e tidak sesuai dengan wantTower **then** continue **end if**
d ← jarak robot ke e
if d < bestDist **then** bestDist ← d
best ← e **end if**
end for
return best

4.2 Struktur Program

4.2.1 Struktur Data

4.2.2 Fungsi

4.2.3 Prosedur

4.3 Pengujian Program

4.3.1 Skenario Pengujian

4.3.2 Hasil dan Analisis Pengujian

1. **Analisis Bot Main vs Alternatif 1**:

   Ringkasan Hasil Pertandingan

   Dari total 6 pertandingan yang dijalankan, **Main** berhasil memenangkan 5 match ($\approx 83\%$) dan **Alternatif 1** memenangkan 1 match ($\approx 17\%$).

Bot	Total Menang	Win Rate
Main	5	$\approx 83\%$
Alternatif 1	1	$\approx 17\%$
Total Match	**5**	

   Main unggul secara signifikan dan konsisten di hampir semua kondisi peta. Kemenangan **Main** bersifat situasional dan sangat bergantung pada karakteristik spesifik peta.

   Strategi Inti

   Main Algoritma **Main** memiliki strategi yang bersifat agresif dan koordinatif antara unit robot dan tower. Setiap unit soldier memiliki prioritas utama yaitu mengisi tile kosong dengan paint tim kemudian mengirimkan informasi mengenai tile musuh kembali ke tower. Tower juga berfungsi untuk

   Alternatif 1 — Reactive Coordination **Alternatif 1** membangun strategi di atas fondasi *reactive coordination* dengan tower sebagai relay pusat. Tidak ada perencanaan global; setiap unit bertindak berdasarkan observasi lokal, lalu mengirim laporan ke tower yang mendistribusikan perintah ke unit spesialis.

   Filosofi: *Soldier menemukan masalah → lapor ke tower → tower dispatch unit spesialis → spesialis tangani.*

   Sistem pesan menggunakan encoding bitpacked (4-bit type + 6-bit x + 6-bit y), mendukung 4 jenis pesan:

   (a) `msgEnemyTile` — soldier lapor lokasi tile musuh ke tower

   (b) `msgTellMopper` — tower dispatch mopper ke koordinat tile musuh

(c) `msgTellWeakTow` — soldier lapor tower musuh dengan HP < 40%

(d) `msgTellSplasher` — tower dispatch splasher ke tower musuh lemah

Perbandingan Algoritma

Tabel berikut merangkum perbedaan utama antara kedua implementasi dari sisi desain algoritma.

Table 1: Perbandingan Algoritma Kedua Bot

Aspek	**Main**	**Alternatif 1**
Eksplorasi	4 endpoint utama sebagai target eksplorasi global. Berfokus terutama pada ruin yang berada pada jangkauan penglihatan.	8 waypoint sekuensial, tidak ada memori. Bisa revisit area yang sama.
Navigasi	Fungsi `robotMove()` mencoba satu direction kemudian terus menerus rotate kanan sampai `canMove()` bernilai true. Rentan menyebabkan gerak maju mundur.	`moveToward()`: coba arah langsung, rotate kiri/kanan hingga 3x. Rentan stuck di koridor.
Target tower	Tidak ada prioritas utama, jika terdeteksi tower ataupun robot musuh maka akan langsung menyerang yang mana saja.	Weakest-first: serang musuh dengan HP terendah. Lebih efisien dalam kill rate.
Manajemen paint	Prioritas pembangunan Money Tower terlebih dahulu untuk memastikan bahwa chip memadai seiring bertambahnya tower	Threshold darurat (< 15), refill (< 50). Splasher threshold lebih tinggi (< 80).
Spawning unit	Bergilir kecuali jika tower mendeteksi adanya tile musuh dalam jangkauan penglihatan	Resource-gated: hard threshold chip + paint. Splasher hanya jika ada intel aktif.
Komunikasi	Menggunakan message yang mengandung koordinat tile musuh pertama yang terdeteksi, komunikasi hanya antar soldier -> tower -> mopper	4-tipe message relay via tower. Koordinasi lintas-sensor radius.
Agresivitas	Secara default bersifat agresif kecuali menemukan tile musuh	Reaktif — agresif hanya jika dipicu pesan atau deteksi langsung.

Analisis Per Peta

Berikut adalah analisis peta-peta yang diobservasi langsung dari hasil pertandingan, dipisah berdasarkan pemenang.

Table 2: Peta dengan Kemenangan Main

Nama Peta	Karakteristik	Catatan Observasi
AlarmClock	Ukuran medium dengan 8 buah tower di tengah yang dikelilingi wall	Main berhasil mengambil sebagian besar tower di wilayahnya lebih cepat dari Alt. 1 sehingga memiliki keunggulan lebih dalam menguasai wilayah tengah
Barcode	Koridor memanjang horizontal dengan warna tile bergilir	Main lebih cepat menguasai wilayah tengah dan langsung menyerang wilayah Alt. 1 sehingga meraih lebih banyak keunggulan
Mirage	Peta besar dengan wall cukup signifikan	Wilayah yang besar memberi ruang bagi bot Main untuk mengambil alih peta dengan cepat mendahului bot Alt. 1, kasus yang serupa dengan dua pertandingan sebelumnya.
Racetrack	Peta medium, 2 wilayah terbagi sama besar oleh tembok	Pertandingan yang sengit di bagian diagonal utama karena kedua tim saling bertemu dan sama-sama membangun dan menghancurkan tower. Bot Main berhasil menang dengan 49.6% peta terpenuhi melawan Alt. 1 dengan 46.1% peta

Table 3: Peta dengan Kemenangan Alternatif 1

Nama Peta	Karakteristik	Catatan Observasi
Four Corners	Peta medium dengan jumlah tower dan wilayah terbatas	Pertandingan yang sengit di bagian tengah peta, Alt. 1 dapat memenangkan lebih banyak wilayah dibandingkan bot Main.

Pola umum yang teramati:

- Main menang konsisten di peta dengan wilayah yang luas dan ruin yang banyak karena kecepatan pengambilan wilayah oleh unit soldier dan mopper.
- Alt. 1 dapat menang di peta yang memaksa kedua tim bertemu di tengah dengan obstacle padat dan tanpa wilayah yang cukup luas untuk *expand*

Kelemahan yang Terverifikasi

Kelemahan Main

- **Rentan terhadap peta kecil** Algoritma pengambilalih tile musuh masih belum memadai untuk jumlah tile yang banyak sehingga strategi Main kalah pada peta kecil di mana kedua tim langsung berhadapan
- **Lemah jika ruin sedikit** Karena fokus awal strategi Main adalah membentuk Money Tower, jumlah spawn robot menjadi cenderung lebih sedikit jika jumlah ruin di map sedikit.
- **Rentan terhadap wall** Unit robot pada strategi Main cenderung rentan untuk berjalan ke arah tembok dan tersangkut

Kelemahan Alternatif 1

- **Navigasi lemah di terrain kompleks.** `moveToward()` gagal melewati obstacle non-trivial. Terkonfirmasi di Oasis, Maze, dan DefaultHuge.
- **Eksplorasi tidak efisien di peta besar.** Tidak ada visited set, robot bisa mengunjungi area yang sama berulang kali.
- **Lemah di peta vertikal.** Waypoint berbasis lebar peta tidak optimal untuk geometri tinggi.
- **Chain pesan rentan putus.** Laporan tile musuh hanya terkirim saat paint rendah atau dekat tower — bisa gagal di momen kritis.

Kesimpulan

Secara keseluruhan, Bot Main adalah implementasi yang lebih tangguh dan konsisten untuk menang secara umum (win rate $\approx$ 83%). , asalkan peta memberikan ruang dan sumber daya (ruin) yang cukup untuk melancarkan taktik ekspansi masifnya. Sementara itu, Alternatif 1 memiliki desain manajemen unit dan target pertempuran yang lebih cerdas, namun potensi penuhnya sangat tertahan oleh kurangnya sistem navigasi pergerakan dan eksplorasi buta tanpa memori.

2. **Analisis Bot Alternatif 1 vs Alternatif 2**:

Ringkasan Hasil Pertandingan

Dari total 75 pertandingan yang dijalankan, **Alternatif 2** berhasil memenangkan 52 match ($\approx$ 69%) dan **Alternatif 1** memenangkan 23 match ($\approx$ 31%).

Bot	Total Menang	Win Rate
Alternatif 1	23	$\approx$ 31%
Alternatif 2	52	$\approx$ 69%
Total Match	**75**	

Alternatif 2 unggul secara signifikan dan konsisten di hampir semua

kondisi peta. Kemenangan **Alternatif 1** bersifat situasional dan sangat bergantung pada karakteristik spesifik peta.

Strategi Inti

Alternatif 1, Reactive Coordination **Alternatif 1** membangun strategi di atas fondasi *reactive coordination* dengan tower sebagai relay pusat. Tidak ada perencanaan global; setiap unit bertindak berdasarkan observasi lokal, lalu mengirim laporan ke tower yang mendistribusikan perintah ke unit spesialis.

Filosofi: *Soldier menemukan masalah → lapor ke tower → tower dispatch unit spesialis → spesialis tangani.*

Sistem pesan menggunakan encoding bitpacked (4-bit type + 6-bit x + 6-bit y), mendukung 4 jenis pesan:

(a) `msgEnemyTile` — soldier lapor lokasi tile musuh ke tower

(b) `msgTellMopper` — tower dispatch mopper ke koordinat tile musuh

(c) `msgTellWeakTow` — soldier lapor tower musuh dengan HP $< 40\%$

(d) `msgTellSplasher` — tower dispatch splasher ke tower musuh lemah

Alternatif 2, Hierarchical Priority Evaluation **Alternatif 2** membangun strategi di atas *priority list* yang ketat per unit, tanpa komunikasi antar unit sama sekali. Setiap unit mengevaluasi kondisi dari prioritas tertinggi ke terendah dan langsung mengeksekusi aksi pertama yang terpenuhi.

Filosofi: *Setiap unit sudah tahu apa yang harus dilakukan dalam situasi apapun, tanpa perlu bertanya ke siapapun.*

Koordinasi terjadi secara implisit melalui *claim system* untuk ruin (maks. 20 klaim, timeout 25 ronde), mencegah duplikasi tugas antar soldier.

Perbandingan Algoritma

Tabel berikut merangkum perbedaan utama antara kedua implementasi dari sisi desain algoritma.

Table 4: Perbandingan Algoritma Kedua Bot

Aspek	**Alternatif 1**	**Alternatif 2**
Eksplorasi	8 waypoint sekuensial, tidak ada memori. Bisa revisit area yang sama.	12 waypoint dengan visited set. ID-biased per kuadran. 60 ronde awal untuk symmetry rush.
Navigasi	`moveToward()`: coba arah langsung, rotate kiri/kanan hingga 3x. Rentan stuck di koridor.	Scoring-based: pilih arah dengan delta jarak terbaik, bonus tile ally (+1), penalti tile musuh (−2). Ada oscillation guard dan auto-reset.
Target tower	First-found: serang musuh pertama yang terdeteksi.	Weakest-first: serang musuh dengan HP terendah. Lebih efisien dalam kill rate.
Manajemen paint	Threshold darurat (< 15), refill (< 50). Splasher threshold lebih tinggi (< 80).	Threshold terstruktur: `REFILL=25`, `BUILD_COMMIT=70`, `BUILD_LOCKED=40`. Building lock mencegah tower setengah jadi.
Spawning unit	Resource-gated: hard threshold chip + paint. Splasher hanya jika ada intel aktif.	Threat-reactive: langsung spawn mopper jika ada ancaman. Splasher setelah stabil (4+ tower, ronde > 150).
Komunikasi	4-tipe message relay via tower. Koordinasi lintas-sensor radius.	Tidak ada pesan. Koordinasi via claim system dan pembacaan state lokal.
Agresivitas	Reaktif — agresif hanya jika dipicu pesan atau deteksi langsung.	Proaktif — sabotage ruin musuh, synchronized attack, anti-rush aktif.

Analisis Kejadian Unik di Beberapa Peta

Berikut adalah analisis peta-peta yang diobservasi langsung dari hasil pertandingan, dipisah berdasarkan pemenang.

Table 5: Peta dengan Kemenangan Alternatif 1

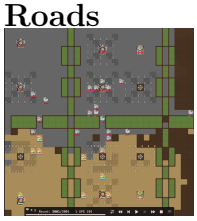
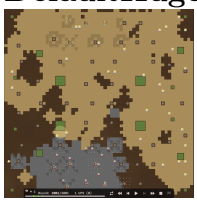
Nama Peta	Karakteristik	Catatan Observasi
Restart	Jalur sempit, area tengah terbuka	Alt. 1 mengecat tile di jalur utama sehingga robot Alt. 2 menghindar secara otomatis dan terisolasi dari wilayah kosong.
Roads 	Koridor memanjang horizontal	Pola sama seperti Restart, Alt. 1 memblokir jalur sempit dengan paint. Alt. 1 cenderung lebih kuat di peta horizontal dibanding vertikal.
Snowman	Peta kecil, banyak dinding	Wilayah terlalu terbatas sehingga keunggulan eksplorasi Alt. 2 tidak relevan. Alt. 1 lebih cepat menguasai posisi strategis di awal.
Batsignal	Orientasi horizontal dominan	Sistem waypoint Alt. 1 yang berbasis lebar peta lebih cocok dengan geometri peta ini.

Table 6: Peta dengan Kemenangan Alternatif 2

Nama Peta	Karakteristik	Catatan Observasi
Thirds	Peta medium, obstacle sedang	Alt. 2 menguasai eksplorasi lebih cepat dan efisien, mencat lebih banyak tile sebelum Alt. 1 sempat membangun cukup tower.
DefaultHuge 	Peta luas, sedikit obstacle	Alt. 2 unggul berkat *visited set* (tidak revisit) dan ID-biased quadrant. Alt. 1 sering mengulang area yang sudah dicat.
Oasis	Obstacle alami berkluster, terrain tidak rata	Navigasi scoring-based Alt. 2 jauh lebih efektif. Alt. 1 sering stuck karena `moveToward()` tidak memperhitungkan biaya tile.

Pola umum yang teramati:

- Alt. 1 menang konsisten ketika berhasil **memblokir jalur** Alt. 2 dengan paint. Ini mengeksploitasi penalti -2 movement scoring Alt. 2 untuk tile musuh.
- Alt. 2 menang konsisten di peta **luas atau kompleks** karena kualitas eksplorasi dan navigasi yang lebih baik.
- Peta kecil dengan obstacle padat hasilnya lebih ketat, sedikit berpihak ke Alt. 1.
- Peta dengan **paint bawaan** lawan di dekat base cenderung menguntungkan Alt. 1 karena mopper dispatch langsung aktif sejak awal (contoh: Barcode, Sandy Beach).

Kelemahan yang Terverifikasi

Kelemahan Alternatif 1

- **Navigasi lemah di terrain kompleks.** `moveToward()` gagal melewati obstacle non-trivial. Terkonfirmasi di Oasis, Maze, dan DefaultHuge.
- **Eksplorasi tidak efisien di peta besar.** Tidak ada visited set, robot bisa mengunjungi area yang sama berulang kali.
- **Lemah di peta vertikal.** Waypoint berbasis lebar peta tidak optimal untuk geometri tinggi.
- **Chain pesan rentan putus.** Laporan tile musuh hanya terkirim

saat paint rendah atau dekat tower — bisa gagal di momen kritis.

Kelemahan Alternatif 2

- **Rentan terhadap blocking paint.** Penalti −2 untuk tile musuh membuat robot Alt. 2 terlalu mudah diblokir — Alt. 1 cukup mengecat jalur utama untuk membatasi mobilitas.
- **Tidak ada respons lintas-jarak.** Tanpa pesan, ancaman di luar sensor radius tidak direspons sampai ada unit yang melintas ke area tersebut.
- **Synchronized attack memangkas DPS.** Serangan hanya di giliran genap memberi tower musuh waktu recovery di giliran ganjil.
- **Lemah di peta kecil + obstacle padat.** Overhead evaluasi scoring justru kontraproduktif ketika ruang gerak sangat terbatas.

Kesimpulan

Berdasarkan data 75 pertandingan, **Alternatif 2** adalah bot yang lebih kuat secara keseluruhan (win rate ≈ 69%). Keunggulan ini bersumber dari kualitas per-turn decision yang lebih tinggi: navigasi lebih andal, eksplorasi lebih sistematis, dan manajemen resource lebih efisien.

Alternatif 1 tidak lebih lemah secara fundamental, tetapi memiliki *niche* yang lebih sempit. Keunggulan utamanya terletak pada satu mekanisme yang tidak ada di **Alternatif 2**: sistem relay pesan yang memungkinkan mopper dikirim langsung ke koordinat tile musuh, sehingga mobilitas lawan bisa diblokir secara aktif dengan paint.

- **Alternatif 2** menang karena **konsistensi kualitas keputusan per unit**, bukan satu fitur dominan.
- **Alternatif 1** menang karena **satu mekanisme spesifik**: blocking paint + mopper dispatch.
- Perbaikan paling impactful untuk **Alternatif 1**: adopsi visited set dan scoring movement.
- Perbaikan paling impactful untuk **Alternatif 2**: kurangi penalti tile musuh saat mode eksplorasi (−1 bukan −2).

3. **Analisis Bot Main vs Alternatif 2**:

Ringkasan Hasil Pertandingan

Pengujian dilakukan melalui total **75 pertandingan** antara bot utama (**Main**) dan bot alternatif (**Alternatif 2**) pada berbagai peta Battlecode.

Bot	Total Menang	Win Rate
Main	54	≈ 72%
Alternatif 2	21	≈ 28%
Total Match	**75**	

Secara keseluruhan, bot **Main** menunjukkan performa yang lebih stabil dan konsisten. Keunggulan utama bot ini terletak pada kemampuan membangun tower lebih cepat dan mempertahankan wilayah melalui produksi unit yang berkelanjutan.

Sebaliknya, **Alternatif 2** menunjukkan performa yang lebih agresif pada tahap eksplorasi awal, namun sering mengalami kesulitan dalam mempertahankan wilayah yang telah ditemukan karena pembangunan tower yang lebih lambat.

Hasil ini menunjukkan bahwa strategi greedy eksplorasi yang digunakan oleh Alternatif 2 tidak selalu menghasilkan solusi optimal pada semua kondisi peta.

Analisis Per Peta

Berikut merupakan analisis pada beberapa peta yang menunjukkan perilaku strategi yang berbeda antara kedua bot.

Table 7: Analisis Pertandingan pada Map Barcode

Aspek	**Bot Dominan**	**Observasi**
Eksplorasi awal	Alternatif 2	Soldier Alternatif 2 melakukan eksplorasi lurus menuju wilayah lawan dan menemukan ruin lebih cepat.
Pembangunan tower	Main	Bot main membangun tower lebih stabil dari wilayah awal sehingga produksi unit meningkat lebih cepat.
Produksi unit	Main	Setelah tower aktif, main dapat melakukan spawn unit secara konsisten dan mulai mendominasi wilayah.
Hasil akhir	Main	Alternatif 2 terlalu jauh dari sumber daya sehingga tidak mampu mempertahankan wilayah yang telah dieksplorasi.

Map Barcode **Kesimpulan:** Strategi greedy eksplorasi Alternatif 2 tidak optimal pada peta ini karena pembangunan tower yang terlambat menyebabkan kekurangan unit untuk mempertahankan wilayah.

Table 8: Analisis Pertandingan pada Map Snowman

Aspek	**Bot Dominan**	**Observasi**
Klaim ruin awal	Alternatif 2	Alternatif 2 berhasil mengklaim 4 dari 5 ruin lebih awal dibandingkan bot main.
Gangguan pembangunan tower	Alternatif 2	Alternatif 2 mencoba mengganggu pembangunan tower lawan.
Produksi unit	Main	Main berhasil menyelesaikan tower lebih cepat sehingga dapat melakukan spawn unit secara berkelanjutan.
Penguasaan wilayah	Main	Wilayah yang awalnya diklaim Alternatif 2 akhirnya diambil alih oleh bot main.

Map Snowman Kesimpulan: Alternatif 2 unggul pada eksplorasi awal, namun gagal mempertahankan wilayah setelah bot main berhasil membangun tower dan meningkatkan jumlah unit.

Table 9: Analisis Pertandingan pada Map Gears

Aspek	**Bot Dominan**	**Observasi**
Eksplorasi wilayah	Alternatif 2	Alternatif 2 mampu menjangkau area peta yang lebih luas sejak awal permainan.
Pergerakan unit	Alternatif 2	Bot main mengalami kesulitan keluar dari wilayah awal karena cenderung menghindari tile lawan.
Penemuan ruin	Alternatif 2	Alternatif 2 menemukan lebih banyak ruin dan membangun tower tambahan lebih cepat.
Penguasaan wilayah	Alternatif 2	Dengan lebih banyak tower aktif, Alternatif 2 dapat memperluas wilayah hingga akhir permainan.

Map Gears Kesimpulan: Pada peta dengan banyak jalur eksplorasi terbuka seperti Gears, strategi greedy eksplorasi Alternatif 2 mampu menghasilkan hasil yang lebih optimal dibandingkan bot main.

Kesimpulan Pengujian

Berdasarkan hasil pengujian, bot **Main** memiliki performa yang lebih stabil karena fokus pada pembangunan tower dan produksi unit secara konsisten. Hal ini memungkinkan bot untuk mempertahankan wilayah dan mendominasi permainan dalam jangka panjang.

Sebaliknya, strategi greedy pada **Alternatif 2** lebih efektif pada kondisi peta yang memungkinkan eksplorasi luas dan ekspansi wilayah cepat. Namun strategi ini kurang optimal pada peta yang menuntut pembangunan tower yang stabil dan produksi unit yang berkelanjutan.

Dengan demikian dapat disimpulkan bahwa strategi greedy eksplorasi memberikan keuntungan pada peta tertentu, tetapi tidak selalu menghasilkan solusi optimal dalam semua kondisi permainan.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan pada bot Battlecode menggunakan pendekatan algoritma greedy, dapat diperoleh beberapa kesimpulan sebagai berikut.

1. Permasalahan Battlecode dapat dimodelkan menggunakan elemen-elemen algoritma greedy, yaitu himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, fungsi solusi, dan fungsi objektif. Pada setiap giliran, unit memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi lokal yang diamati.
2. Strategi greedy memungkinkan setiap unit robot mengambil keputusan secara cepat tanpa memerlukan perencanaan global yang kompleks. Hal ini penting dalam Battlecode karena setiap robot memiliki batasan waktu komputasi pada setiap ronde.
3. Berdasarkan hasil pengujian sebanyak 75 pertandingan antara bot utama dan bot alternatif, bot utama memperoleh 54 kemenangan (sekitar 72%) sedangkan bot alternatif memperoleh 21 kemenangan (sekitar 28%). Hal ini menunjukkan bahwa strategi yang digunakan oleh bot utama lebih stabil dan efektif dalam berbagai kondisi peta.
4. Bot utama memiliki keunggulan pada manajemen pembangunan tower dan produksi unit yang lebih konsisten, sehingga mampu mempertahankan wilayah dan mendominasi permainan dalam jangka panjang.
5. Strategi eksplorasi agresif yang digunakan pada bot alternatif dapat memberikan keuntungan pada kondisi tertentu, khususnya pada peta yang luas dan memiliki banyak jalur eksplorasi. Namun strategi tersebut kurang optimal pada peta dengan ruang terbatas atau ketika pembangunan tower menjadi faktor utama kemenangan.

5.2 Saran

Berdasarkan hasil analisis dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan bot pada penelitian selanjutnya.

1. Mengembangkan sistem navigasi yang lebih adaptif, misalnya dengan mempertimbangkan biaya pergerakan atau jalur alternatif sehingga robot tidak mudah terjebak pada area yang terhalang atau dikelilingi tile musuh.
2. Menambahkan mekanisme koordinasi antar robot yang lebih efektif, misalnya dengan sistem komunikasi yang lebih kaya sehingga informasi ancaman atau target dapat disebarkan dengan lebih cepat ke seluruh unit.
3. Mengombinasikan pendekatan greedy dengan strategi lain seperti heu-

ristik berbasis evaluasi wilayah atau perencanaan jangka pendek agar keputusan robot tidak hanya berdasarkan kondisi lokal tetapi juga mempertimbangkan kondisi global permainan.

4. Melakukan pengujian pada lebih banyak variasi peta dan skenario permainan untuk memperoleh evaluasi performa bot yang lebih komprehensif.
5. Mengoptimalkan parameter heuristik seperti prioritas aksi, ambang batas sumber daya, dan strategi eksplorasi sehingga performa bot dapat lebih stabil pada berbagai kondisi permainan.

Daftar Pustaka

Gubskiy, T., & Nguyen, A. (2025). *Just Woke Up – Battlecode 2025 Postmortem.* Technical report.

Hahn, M. (2025). *Battlecode 2025 Postmortem.* Team Confused.

Sha, M., Erranki, S., Ji, E., & Gao, J. (2025). *SPAARK Battlecode 2025 Postmortem.* Team SPAARK.

Sharma, C., & Gagushin, E. (2025, February). *Money Is All You Need: Battlecode 2025 Postmortem.* Team Om Nom.

The Kragle Team. (2025). *Battlecode 2025 Postmortem.* Technical report.

Munir, Rinaldi. (2026), *Algoritma Greedy (Bagian 1)*, 2025. https://informatika.stei.itb.ac.id/ rinaldi.m 2026/04-Algoritma-Greedy-(2026)-Bag1.pdf.

Munir, Rinaldi. (2026), *Algoritma Greedy (Bagian 2)*, 2025. https://informatika.stei.itb.ac.id/ rinaldi.m 2026/05-Algoritma-Greedy-(2026)-Bag2.pdf

Munir, Rinaldi. (2026), *Algoritma Greedy (Bagian 3)*, 2025. https://informatika.stei.itb.ac.id/ rinaldi.m 2026/06-Algoritma-Greedy-(2026)-Bag3.pdf

GeeksforGeeks. (2022, October 4). Greedy Algorithm Tutorial. GeeksforGeeks. https://www.geeksforge to-greedy-algorithm-data-structures-and-algorithm-tutorials/

Lampiran

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Josh Reinhart Zidik

Arina Azka

Angelina Andra Alanna

Tautan Repositori GitHub

Seluruh kode sumber program dapat diakses melalui repositori berikut: